

Project Title

Job Tracker Job Application Management System

By

Azka Hameed

Roll No: RBSIT-21-07

Session: 2021-2025

Supervisor: Mr. Ghulam Ghos

**Department of Information Technology, Bahauddin Zakariya
University Multan**



January 2026

BAHAUDDIN ZAKARIYA UNIVERSITY, MULTAN

Department of Information Technology

Job Tracker Job Application Management System

(Job Tracker)

Degree of Bachelor of Science in Information Technology

By

AZKA HAMEED

Roll No: BSIT-21-07

Session: 2021 – 2025

Supervisor: Mr. Ghulam Ghos



January 2026

Certificate of Approval

This is to certify that the project titled "**Job Tracker Job Application Management System**" submitted by **AZKA HAMEED (Roll No: RBSIT-21-07)** in partial fulfillment of the requirements for the degree of Bachelor of Science in Information Technology has been examined and approved.

Name: Mr. Ghulam Ghos

Name: Dr. Maroof Pasha

Signature: _____

Signature: _____

Date: _____

Date: _____

Declaration

I hereby declare that this project titled "**Job Tracker Job Application Management System**" is my own work and has not been submitted elsewhere for any other degree or qualification

Name: AZKA HAMEED

Roll No: RBSIT-21-07

Signature: _____

Date: _____

Acknowledgements

All praise, thanks, glorification, and gratitude are for **ALLAH** Almighty. I achieved my goal by **ALLAH's** infinite mercy and grace. I say Alhamdulillah for HIS uncountable blessings. After **ALLAH**, thanks to last Prophet **Muhammad** (ﷺ) who guided us toward the right path. Beloved prophet (ﷺ) says “Education is equally important for men and women”.

Very special and humble thanks to my father. He is my main supporter to continue the studies. My father instilled iron determination in me. I just want to say, he is great, he is genius. I also express special thanks to my mother. She always prayed for me and improves my confidence to continue the higher study. My mother has always been very caring about my health, studies, and career. I would like to pay my heartiest gratitude to my sister and brothers for being a great source of support and encouragement to complete this work. Especially, thanks to my brothers who guided and helped me all along the way.

I would like to express my sincere gratitude to my supervisor **Mr. Ghulam Ghos** for the continuous support of my research work, for their patience, motivation, enthusiasm, and immense knowledge. Their guidance helped me during all the time of research and writing of this thesis. I could not have imagined having a better advisor and mentor for my research work.

Secondly, I would like to say thanks to **Mam Aliya Bodla**, principle and director of Regional Institute of Allied Health Sciences and Higher Education, Mian Channu, whose support and concern towards students is highly valuable.

In addition, I would like to acknowledge important role of **Mr. Usman**, whose help and assistance remained a source of motivation for me throughout this task.

AZKA HAMEED
RBSIT-21-07

Abstract

Job seekers today apply to a large number of positions across multiple platforms, making it difficult to track application status, deadlines, and follow-ups in an organized manner. Existing solutions are often either overly generic spreadsheet-based trackers or expensive enterprise tools that are not tailored to individual users. This project presents the design and development of a web-based Job Application Tracking System that enables users to systematically manage their job search process from a single, centralized platform.

The system is built on a three-tier architecture with a RESTful API backend, a document-oriented database, and a responsive React-based frontend. It provides secure user authentication using JWT (JSON Web Tokens), full CRUD functionality for job applications, and a status pipeline that allows users to track applications through stages such as Applied, Interview, Offer, and Rejected. A background reminder service notifies users of upcoming deadlines and follow-ups, while an analytics module provides statistical insights into application trends and success rates. The system also incorporates a tiered subscription model, offering both free and premium access levels with corresponding feature limits.

The application was developed using modern web technologies and evaluated through unit, integration, security, performance, and usability testing to ensure reliability and a smooth user experience. Functional testing confirmed that core modules — authentication, application management, reminders, and analytics — operate correctly under expected use cases, and performance testing validated acceptable API response times.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION	1
1.1 Project Overview	1
1.2 Problem Statement	1
1.3 Project Motivation	2
1.4 Aims and Objectives	3
1.5 Project Scope	5
1.6 Document Structure	7
Chapter 2: Literature Review & Background Study	9
2.1 Introduction to Domain	9
2.2 Existing Systems Analysis	11
2.3 Comparative Analysis of Existing Systems	12
2.4 Identified Research Gaps	13
2.5 Proposed Solution Overview	15
2.6 Summary of Related Background Research	16
Chapter 3: Requirement Analysis & Specification	18
3.1 Introduction to Requirements	18
3.2 Stakeholder Analysis	18
3.3 Functional Requirements	19
3.4 Non-Functional Requirements	24
3.5 Use Case Modeling	24
3.6 User Stories	26
3.7 Hardware and Software Requirements	27
Chapter 4: System Design	30
4.1 Design Principles and Methodology	30
4.2 System Architecture	31
4.3 Database Design	33
4.4 API Design	36
4.5 Process Design	39
4.6 Security Design	46
4.7 UI/UX Design	47

Chapter 5: Implementation & Development.....	51
5.1 Development Environment Setup.....	51
5.2 Module Implementation	53
5.3 Key Code Explanations	60
5.4 Frontend Implementation.....	63
5.5 API Integration Between Frontend and Backend.....	65
5.6 Database Integration.....	66
5.7 Challenges Faced and Solutions	66
Chapter 6: Testing & Validation	68
6.1 Testing Strategy	68
6.2 Test Environment	69
6.3 Unit Testing	69
6.4 Integration Testing	70
6.5 Functional Test Cases and Results.....	71
6.6 Security Testing	74
6.7 Performance Testing	75
6.8 UI and Usability Testing.....	77
6.9 Bug Report and Resolution	77
6.10 Testing Summary.....	78
Chapter 7: Results & Discussion	80
7.1 System Implementation Results.....	80
7.2 Achievement of Objectives	86
7.3 Gap Resolution Assessment.....	87
7.4 Performance Evaluation	88
7.5 Discussion.....	88
7.6 Limitations Encountered	89
Chapter 8: Conclusion & Future Work	91
8.1 Conclusion.....	91
8.2 Limitations.....	92
8.3 Future Enhancements	92
8.4 Social Impact and Applications	94
References	96

Appendix.....	97
Appendix A - Installation and Setup Guide	97
Appendix B - API Endpoint Reference	98
Appendix C - Database Schema Reference	98

List of Tables

Table 1.1 - Project Objectives.....	4
Table 1.2 - Project Scope Summary	6
Table 2.1 - Existing Tools Feature Comparison.....	12
Table 2.2 - Identified Research Gaps.....	14
Table 2.3 - Background Research Summary.....	16
Table 3.1 - User Authentication Functional Requirements	19
Table 3.2 - Job Application Management Requirements	20
Table 3.3 - Status Pipeline Requirements	21
Table 3.4 - Reminder Module Functional Requirements	21
Table 3.5 - Analytics Module Functional Requirements	22
Table 3.6 - Premium Subscription Module Requirements	23
Table 3.7 - Non-Functional Requirements	24
Table 3.8 - User Stories	26
Table 3.9 - Hardware Requirements	27
Table 3.10 - Software and Technology Stack.....	27
Table 4.1 - Database Indexes.....	36
Table 4.2 - Authentication API Endpoints	36
Table 4.3 - Job Application API Endpoints.....	37
Table 5.1 - Backend Development Tools and Versions.....	51
Table 5.2 - Challenges Faced and Solutions	66
Table 6.1 - Authentication Functional Test Cases	71
Table 6.2 - Job Application CRUD Test Cases.....	72
Table 6.3 - Free Tier Limit Test Cases	73
Table 6.4 - Statistics and Reminders Test Cases	73
Table 6.5 - Frontend Functional Test Cases.....	74
Table 6.6 - Security Test Cases	75
Table 6.7 - API Performance Results	76
Table 6.8 - Bug Report and Resolution Log	77
Table 6.9 - Testing Summary	78
Table 7.1 - Objectives Achievement Matrix	86

List of FIGURES

Figure 1.1 - Document Structure Flow Diagram	8
Figure 2.1 - Job Tracking Tools Evolution Timeline	10
Figure 2.2 - Existing Tools Feature Comparison	13
Figure 2.3 - Research Gap to Solution Mapping	15
Figure 3.1 - System Level Use Case Diagram	25
Figure 3.2 - Authentication Use Case Diagram	25
Figure 3.3 - Application Management Use Case Diagram	25
Figure 3.4 - Reminder Management Use Case Diagram.....	26
Figure 3.5 - Technology Stack Architecture Diagram	28
Figure 4.1 - High Level Architecture Diagram	31
Figure 4.2 - Three Tier Architecture Diagram	32
Figure 4.3 - Background Reminder Service Flow	33
Figure 4.4 - User Document Structure	34
Figure 4.5 - Job Application Document Structure.....	35
Figure 4.6 - JWT Authentication Flow Diagram	38
Figure 4.7 - API Response Structure Diagram.....	39
Figure 4.8 - DFD Level 0 Context Diagram	40
Figure 4.9 - DFD Level 1 Diagram.....	41
Figure 4.10 - Add Application Sequence Diagram	42
Figure 4.11 - Reminder Processing Sequence Diagram	43
Figure 4.12 - User Login Sequence Diagram	44
Figure 4.13 - Status Update Activity Diagram.....	45
Figure 4.14 - Dashboard Page Design	48
Figure 4.15 - Applications List Page Design	49
Figure 4.16 - Add Application Form Design	49
Figure 4.17 - Reminders Page Design	50
Figure 5.1 - Project Directory Structure.....	53
Figure 5.2 - Authentication Module Implementation Diagram	55
Figure 5.3 - Job Controller Function Map	57
Figure 5.4 - Reminder Service Flow Code	59
Figure 5.5 - JWT Token Generation Snippet.....	60
Figure 5.6 - Free Tier Limit Check Snippet	61
Figure 5.7 - Axios Instance Configuration Snippet.....	62
Figure 7.1 - Landing Page Screenshot.....	80
Figure 7.2 - Registration Page Screenshot	81
Figure 7.3 - Login Page Screenshot.....	81
Figure 7.4 - Dashboard Page Screenshot	82
Figure 7.5 - Applications List Page Screenshot	83

Figure 7.6 - Add Application Page Screenshot	84
Figure 7.7 - Edit Application Page Screenshot	84
Figure 7.8 - Reminders Page Screenshot	85
Figure 7.9 - Settings Page Screenshot.....	85
Figure 7.10 - Pricing Page Screenshot.....	86

CHAPTER 1: INTRODUCTION

1.1 Project Overview

The Job Tracker is a full-stack web application developed to modernize and simplify the process of managing job applications for individuals actively engaged in job searching. The system is built using a Node.js and Express.js backend with a React.js frontend, integrating a clean and responsive user interface to address the shortcomings of traditional, manual job-search tracking methods such as spreadsheets, notebooks, or disconnected digital notes.

The system provides a single authenticated portal for registered users. Through this portal, users can add new job applications, update their current status, set follow-up reminders, manage salary information, apply tags, and monitor their overall job search progress from a unified dashboard. The application is designed to serve job seekers at every stage of their search - from wishlist management to offer acceptance - by centralizing all application data in one organized, accessible location.

The backend is powered by Express.js and connects to a MongoDB database through Mongoose ORM, exposing a fully documented RESTful API consumed by the React.js frontend. Authentication is handled through JSON Web Tokens (JWT), ensuring that each user's data remains private and secure. The frontend is styled using Tailwind CSS with a modern dark-themed design system, providing a visually polished and professional user experience. Follow-up reminders are managed through a background cron job service that runs at scheduled intervals on the server.

1.2 Problem Statement

1.2.1 Existing Challenges in Manual Job Application Tracking

The process of applying for jobs in today's competitive market often involves submitting applications to dozens - sometimes hundreds - of positions across multiple platforms such as LinkedIn, Indeed, Rozee.pk, and company-specific career portals. Despite the scale and complexity of this process, most job seekers rely on highly informal methods to track their applications.

The most common approach is the use of spreadsheets, either in Microsoft Excel or Google Sheets. While spreadsheets offer basic organizational capability, they require entirely manual data entry, provide no reminders or notifications, lack status-based visual organization, and offer no analytics or progress reporting. As the number of applications grows, spreadsheets become increasingly difficult to maintain and navigate.

Other job seekers rely on email folders, browser bookmarks, or simple text notes. These approaches are even more fragmented. There is no single view of all active applications, no

way to track which applications require follow-up, and no historical record of rejected or withdrawn applications to inform future strategy.

The consequences of these limitations are significant. Job seekers frequently forget to follow up on submitted applications, missing the opportunity to demonstrate proactive interest. They lose track of interview schedules, offer deadlines, and negotiation windows. They have no visibility into patterns in their job search - for instance, which types of roles yield more interview callbacks or which salary ranges they are targeting. Without such insights, the job search process becomes reactive rather than strategic.

For students and fresh graduates in Pakistan - particularly those from institutions such as Bahauddin Zakariya University who are entering the job market for the first time - this problem is especially acute. The absence of career management tools tailored to their context means many applicants underperform not due to lack of qualifications but due to poor organizational management of their opportunities.

1.2.2 Need for a Dedicated Application Management Tool

A dedicated, web-based job application tracking tool addresses all of the limitations identified above. By providing a structured data model for each application - capturing company name, position, location, status, salary expectations, application date, job type, work mode, and personal notes - such a system eliminates the ambiguity and manual overhead of spreadsheet-based tracking.

Status-based tracking with clearly defined stages (Wishlist, Applied, Interviewing, Offer, Accepted, Rejected, Withdrawn) gives users an immediate visual understanding of where each application stands in the hiring pipeline. Priority flags (Low, Medium, High) allow users to focus their energy on the opportunities that matter most.

A reminder system integrated directly into the application platform ensures that follow-up actions are never missed. Rather than setting external calendar reminders that are disconnected from the application record, users can set a reminder date on each job entry and receive alerts when those dates approach. This tightly coupled approach keeps all relevant information in one place.

An analytics dashboard that displays application counts by status, weekly and monthly activity trends, and interview conversion rates gives users the strategic visibility needed to continuously improve their job search approach. This data-driven perspective transforms job searching from a series of individual, disconnected actions into a manageable, measurable campaign.

1.3 Project Motivation

The motivation for developing the Job Tracker emerged from direct observation of the challenges faced by university students and fresh graduates during their job search journeys.

As a student at the Department of Information Technology, Bahauddin Zakariya University Multan, the developer witnessed first-hand how disorganized application tracking contributed to missed opportunities and unnecessary stress among peers navigating the transition from academic to professional life.

Several specific observations provided the impetus for this project. Students applying to multiple organizations simultaneously had no reliable way to remember which applications had been submitted, which required follow-up, and which had resulted in rejections versus silence. The absence of a centralized tracking mechanism meant that interview preparation was often reactive - triggered by an unexpected callback rather than planned in advance.

At the national level, Pakistani job seekers lack access to affordable, locally relevant career management tools. International platforms such as Huntr or Teal offer job tracking features but are designed for Western job markets, require paid subscriptions for full functionality, and are not optimized for the application workflows common on Pakistani platforms such as Rozee.pk and LinkedIn Pakistan.

The motivation was therefore both personal and practical: to build a free, accessible, and functional tool that directly addresses the job-search management needs of Pakistani students and early-career professionals, demonstrating that effective productivity software can be developed indigenously to serve local needs.

1.4 Aims and Objectives

1.4.1 Primary Objectives

The primary objectives of the Job Tracker project are as follows:

- **Objective 1** - Full-Stack Web Application Development: To design and develop a production-quality job application tracking system with a Node.js and Express.js REST backend and a React.js frontend, supporting the complete job application lifecycle from initial wishlist entry through to offer acceptance or rejection.
- **Objective 2** - Secure User Authentication and Authorization: To implement a JWT-based authentication system that allows users to register, log in, and maintain secure, persistent sessions, ensuring that each user's application data is private and accessible only to them.
- **Objective 3** - Comprehensive Job Application Management: To develop a full CRUD (Create, Read, Update, Delete) interface for job applications, capturing all relevant fields including company, position, location, status, job type, work mode, salary range, currency, priority, tags, applied date, job URL, description, and personal notes.
- **Objective 4** - Status and Priority Tracking: To implement a structured status pipeline (Wishlist → Applied → Interviewing → Offer → Accepted / Rejected / Withdrawn) and a three-level priority system (Low, Medium, High) that enables users to categorize and filter their applications meaningfully.

- **Objective 5** - Follow-Up Reminder System: To build a server-side cron job-based reminder service that tracks upcoming follow-up dates set by users on their applications, surfacing due reminders through the dashboard and dedicated reminders page.

1.4.2 Secondary Objectives

- **Objective 6** - Analytics Dashboard: To develop a statistics dashboard that aggregates application data by status, calculates weekly and overall application counts, and provides users with a clear summary of their job search progress.
- **Objective 7** - Search, Filter, and Sort Functionality: To implement server-side search by company name or position, filtering by application status, and sorting by date, company name, or priority, enabling users to efficiently navigate large application datasets.
- **Objective 8** - Premium Subscription Model: To implement a freemium model with a free tier limited to twenty-five applications and a one-time premium upgrade that unlocks unlimited applications and additional features, demonstrating a commercially viable product architecture.
- **Objective 9** - Responsive and Polished User Interface: To design and implement a fully responsive dark-themed user interface using Tailwind CSS and Framer Motion animations, providing a professional and visually consistent experience across desktop and mobile devices.

Table 1.1 - Project Objectives

Obj. ID	Objective Description	Priority	Status
OBJ-01	Full-Stack Web Application with Node.js, Express.js & React.js	High	✓
OBJ-02	Secure User Authentication and Authorization using JWT	High	✓
OBJ-03	Comprehensive Job Application Management (Full CRUD)	High	✓
OBJ-04	Status and Priority Tracking with Seven-Stage Pipeline	High	✓
OBJ-05	Follow-Up Reminder System with Server-Side Cron Processing	High	✓
OBJ-06	Analytics Dashboard with Application Statistics	Medium	✓
OBJ-07	Search, Filter, and Sort Functionality	Medium	✓

Obj. ID	Objective Description	Priority	Status
OBJ-08	Premium Subscription Model with Free Tier Limit	Medium	✓
OBJ-09	Responsive and Polished UI with Dark Theme	Medium	✓

1.5 Project Scope

1.5.1 In-Scope Features

The Job Tracker encompasses a comprehensive set of features covering the core needs of a job application management system. The following functional areas are implemented within the scope of this project:

User Authentication and Account Management: The system supports user registration with name, email, and password, login with JWT token-based session management, profile update functionality, and a premium upgrade mechanism. Passwords are hashed using bcryptjs before storage.

Job Application CRUD Operations: Users can create new job application records capturing all relevant fields, view a paginated and filterable list of all their applications, edit any application to update status or details, and delete applications they no longer wish to track.

Status Pipeline Management: Applications move through a seven-stage status pipeline. Each status is visually represented with a distinct color scheme in the interface, providing instant recognition of application states at a glance.

Salary and Currency Tracking: The system supports salary range entry (minimum and maximum) with a searchable currency selector covering thirty major world currencies including PKR, USD, EUR, AED, GBP, SAR, and others commonly relevant to Pakistani job seekers targeting both local and international opportunities.

Follow-Up Reminder System: Users can set a follow-up reminder date on any application. A background cron service runs hourly on the server, identifies reminders that are due, marks them as notified, and surfaces upcoming reminders (within the next seven days) through the reminders API endpoint.

Analytics and Statistics: The dashboard displays total application count, applications added in the current week and month, and a breakdown of application counts by each status category, giving users an instant overview of their job search activity.

Search, Filter, and Sort: The applications list supports text search across company and position fields, status-based filtering, and sorting by newest, oldest, company name alphabetically, and priority level.

Tags System: Users can attach comma-separated tags to any application (for example: react, remote, startup) for custom categorization and future filtering.

Responsive Design: The interface is fully responsive and functional across desktop, tablet, and mobile screen sizes.

1.5.2 Out-of-Scope Features

The following features are explicitly excluded from the current scope of the project:

Email Notifications: While the server-side reminder service identifies due reminders, no email notification is sent to users. Email delivery integration (via services such as SendGrid or Nodemailer with SMTP) is identified as a future enhancement.

Calendar Integration: The system does not integrate with external calendar applications such as Google Calendar or Outlook Calendar for reminder synchronization.

Resume and Document Management: Users cannot upload or store CV files, cover letters, or supporting documents within the system. Document management is identified as a future feature.

Social Features: There is no social or community layer - users cannot share applications, compare statistics with peers, or access a community forum.

Mobile Application: No native iOS or Android application has been developed. The system is accessible via responsive web design on mobile browsers but a dedicated mobile app is not part of this release.

Job Board Integration: The system does not automatically import job listings from external platforms such as LinkedIn, Indeed, or Rozee.pk. All application data is entered manually by the user.

Interview Scheduling Module: While interview status can be tracked, there is no dedicated interview preparation or scheduling sub-module within the current scope.

Table 1.2 - Project Scope Summary

In-Scope Features	Out-of-Scope Features
User Authentication with JWT	Email / Push Notifications
Job Application CRUD Operations	Calendar Integration
Seven-Stage Status Pipeline	Resume and Document Management
Salary Tracking with 30 Currencies	Social or Community Features
Follow-Up Reminder System	Mobile Application (iOS/Android)

In-Scope Features	Out-of-Scope Features
Analytics and Statistics Dashboard	Job Board Integration (LinkedIn etc.)
Search, Filter, and Sort	Interview Scheduling Module
Tags System	Payment Gateway Integration
Responsive Dark-Theme UI	Data Export (CSV/PDF)
Premium Subscription Model	Offline Capability

1.6 Document Structure

This report is organized into eight chapters, each addressing a distinct phase of the software development lifecycle, following the standard format prescribed by the Department of Information Technology, Bahauddin Zakariya University, Multan.

- **Chapter 1** provides the project overview, problem statement, motivation, objectives, and scope - establishing the context and rationale for the development of the Job Tracker application.
- **Chapter 2** reviews existing job tracking tools and platforms, performs a comparative feature analysis, identifies the research and product gaps that motivated this project, and surveys relevant background literature on web application development and career management tools.
- **Chapter 3** presents the complete functional and non-functional requirements, stakeholder analysis, use case diagrams with detailed specifications, user stories, and the hardware and software stack used in development.
- **Chapter 4** covers the full system architecture, MongoDB database schema design, RESTful API design with endpoint documentation, data flow diagrams, sequence diagrams for key workflows, and the UI/UX design principles applied throughout the application.
- **Chapter 5** describes the development environment setup, module-by-module implementation details, key code explanations for critical components including the authentication system, reminder service, and API structure, and a log of challenges encountered with their solutions.
- **Chapter 6** presents the testing strategy, functional test cases and results across all major modules, security testing, performance measurements, and an overall testing summary.
- **Chapter 7** presents final system screenshots across all application pages, an objectives achievement matrix confirming the completion of all defined goals, and a discussion of results and limitations encountered during development.

- **Chapter 8** summarizes the project contributions, acknowledges current limitations, and outlines planned future enhancements including email notifications, calendar integration, resume management, and mobile application development.

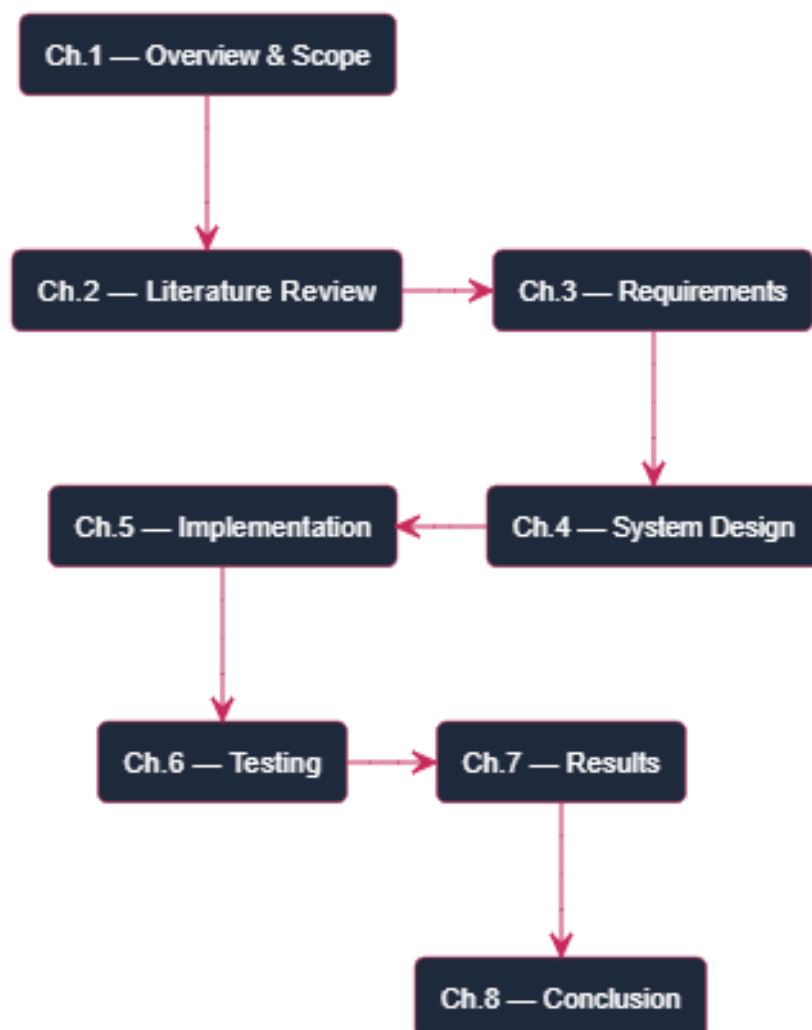


Figure 1.1 - Document Structure Flow Diagram

Chapter 2: Literature Review & Background Study

2.1 Introduction to Domain

2.1.1 Evolution of Job Application Tracking Tools

The concept of tracking job applications is not new. In the early days of digital job searching - roughly the late 1990s and early 2000s - candidates who applied for positions through emerging online job boards such as Monster.com and CareerBuilder would manually record their activity in Microsoft Word documents or printed spreadsheets. The volume of applications in that era was relatively low, and informal tracking methods were adequate.

As internet penetration grew and online job boards proliferated globally, the volume of applications a single candidate could submit in a week increased dramatically. By the mid-2000s, platforms such as LinkedIn had introduced professional networking into the job search process, adding a new layer of relationship management on top of application tracking. The number of touchpoints a job seeker needed to manage - initial application, LinkedIn connection, recruiter message, phone screening, technical interview, HR interview, offer negotiation - grew far beyond what an unstructured list could handle effectively.

The 2010s saw the emergence of dedicated applicant tracking systems (ATS) on the employer side. Tools such as Greenhouse, Lever, and Workday allowed recruiters and hiring managers to manage large volumes of incoming applications efficiently. However, the candidate side of this equation received comparatively little attention. Job seekers continued to rely on spreadsheets while employers invested heavily in their own tracking infrastructure, creating a significant asymmetry in the hiring process.

It was not until the late 2010s and early 2020s that candidate-facing job tracking tools began to emerge as a distinct product category. Platforms such as Huntr, Teal, and Jobscan introduced browser extensions, Kanban boards, and resume optimization tools specifically designed for job seekers. Despite this progress, these tools remained predominantly focused on Western job markets and carried subscription costs that made them inaccessible to students and early-career professionals in developing countries such as Pakistan.

The Job Tracker developed in this project represents a response to this gap - a purpose-built, locally relevant, and freely accessible tool that brings structured application management to Pakistani job seekers at no cost.

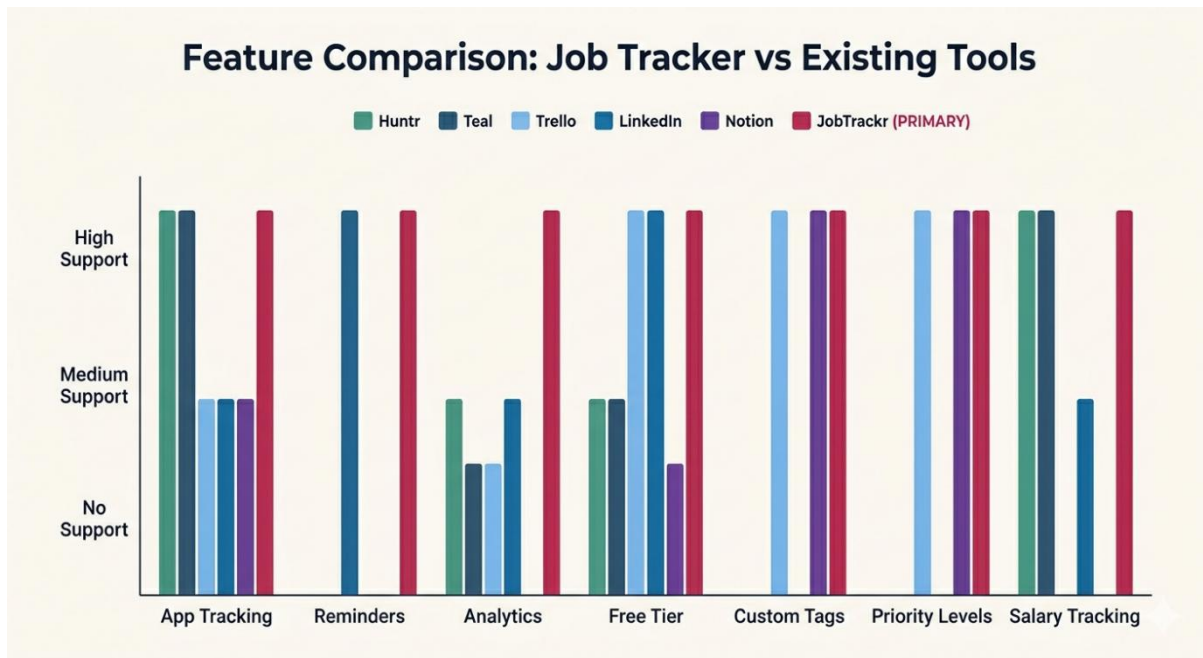


Figure 2.1 - Job Tracking Tools Evolution Timeline

2.1.2 Role of Web Applications in Career Management

Modern web applications have transformed career management by making powerful organizational tools accessible from any device with an internet connection, without requiring software installation or platform-specific compatibility. The shift from desktop software to browser-based applications - enabled by frameworks such as React.js and backend technologies such as Node.js - has lowered the barrier to building and deploying sophisticated productivity tools significantly.

For career management specifically, web applications offer several advantages over traditional alternatives. First, data is stored centrally on a server rather than on a local device, meaning it is accessible from any location and is not lost if a device is replaced or damaged. Second, web applications can integrate background services - such as cron-based reminder checking - that operate continuously on the server regardless of whether the user has the application open. Third, modern web frameworks allow the development of highly interactive interfaces with real-time feedback, smooth animations, and responsive layouts that adapt gracefully to different screen sizes.

The combination of these capabilities - persistent cloud storage, background processing, and responsive interfaces - makes web applications the ideal delivery mechanism for a job tracking tool designed to serve users across varying devices and usage contexts.

2.2 Existing Systems Analysis

2.2.1 Microsoft Excel and Google Sheets

Spreadsheets remain the most widely used job application tracking tool globally, primarily because of their universal availability and zero learning curve. A typical job tracking spreadsheet includes columns for company name, position, application date, status, contact name, and notes. Users update rows manually as their application progresses through the pipeline.

Despite their ubiquity, spreadsheets suffer from several critical limitations in the job tracking context. There is no built-in status pipeline with defined stages - users must create and enforce their own conventions, which leads to inconsistency. There are no reminders or notifications of any kind. There is no search functionality beyond basic CTRL+F text matching. There is no analytics layer - generating a count of applications by status requires the user to manually write COUNTIF formulas. Collaboration is possible in Google Sheets but creates privacy concerns. Most critically, spreadsheets provide no visual hierarchy or priority management, making it difficult to focus on high-priority opportunities in a large list.

2.2.2 Huntr

Huntr is a dedicated job tracking web application that presents applications as cards on a Kanban board organized by pipeline stage. It supports the addition of contacts, notes, tasks, and documents to each application card. A browser extension allows users to save job listings from any website directly into their Huntr board.

Huntr's Kanban interface is visually intuitive and well-suited to users who prefer a board-style view. However, the free tier is limited to forty cards total, after which a paid subscription is required. The platform is designed primarily for the North American job market, and its browser extension is optimized for LinkedIn and Indeed US rather than platforms commonly used by Pakistani job seekers. There is no reminder system, no salary tracking with multi-currency support, and no analytics dashboard in the free tier.

2.2.3 Teal

Teal is a comprehensive career management platform that includes a job tracker, resume builder, resume-job description matching tool, and professional profile management. The job tracker component supports status tracking, note-taking, and contact management for each application.

Teal is the most feature-complete tool reviewed, but this comprehensiveness comes with significant trade-offs. The platform requires a paid subscription for full access to the job tracker and resume features. The resume builder and matching tools, while valuable, are designed for American resume conventions and ATS systems that differ from hiring practices in Pakistani organizations. The interface, while polished, is complex and has a steeper learning

curve than simpler tools. Like Huntr, Teal has no multi-currency salary tracking and no reminder notification system.

2.2.4 Jobscan

Jobscan is primarily a resume optimization tool that analyzes how well a resume matches a specific job description's keywords. It includes a basic job tracker as a secondary feature. The tracker supports adding applications with status, date, and notes but offers no reminder system, no analytics, and no salary tracking. The core resume scanning functionality requires a paid subscription.

Jobscan is most useful as a supplementary tool in combination with a dedicated tracker rather than as a standalone solution. Its relevance is further limited in the Pakistani context because its ATS optimization is calibrated for US-based hiring systems that differ substantially from local recruitment processes.

2.2.5 Notion and Similar Productivity Tools

Many job seekers use general-purpose productivity and note-taking platforms such as Notion, Airtable, or Trello as makeshift job trackers. These tools offer greater flexibility than spreadsheets - Notion supports rich text notes and linked databases, Airtable supports relational data with custom views, and Trello provides a Kanban board interface.

However, all of these tools require significant manual configuration to function as effective job trackers. Users must build their own templates, define their own status labels, and create their own views. None provides built-in reminder systems, salary tracking, or job-search-specific analytics. The configuration overhead is a significant barrier for users who want to start tracking immediately without investing time in setup.

2.3 Comparative Analysis of Existing Systems

The table below provides a direct feature-by-feature comparison of the most relevant existing tools against the Job Tracker developed in this project.

Table 2.1 - Existing Tools Feature Comparison

Feature	Excel/Sheets	Huntr	Teal	Jobscan	Notion	This System
Status Pipeline	No	Yes	Yes	Basic	Manual	Yes
Reminder System	No	No	No	No	No	Yes
Salary with Currency	No	No	No	No	No	Yes (30)

Feature	Excel/Sheets	Huntr	Teal	Jobscan	Notion	This System
Analytics Dashboard	No	Paid	Paid	No	No	Yes
Search and Filter	Basic	Yes	Yes	No	Yes	Yes
Tags System	Manual	No	No	No	Yes	Yes
Free Full Access	Yes	Limited	Limited	Limited	Yes	Yes
Pakistani Market	No	No	No	No	No	Yes
Mobile Responsive	No	Yes	Yes	Yes	Yes	Yes

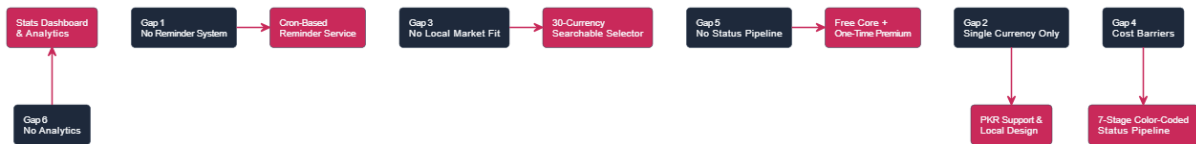


Figure 2.2 - Existing Tools Feature Comparison

2.4 Identified Research Gaps

The comparative analysis reveals six distinct gaps in existing tools that this project directly addresses.

- **Gap 1 - No Integrated Reminder System:** Not a single reviewed tool provides a server-side reminder system that proactively surfaces due follow-up actions to the user. Huntr supports task creation but requires the user to manually check tasks. Spreadsheets, Notion, and Teal have no reminder functionality at all. The absence of proactive reminders is one of the primary reasons job seekers miss follow-up opportunities - a problem directly solved by the cron-based reminder service in this project.
- **Gap 2 - No Multi-Currency Salary Tracking:** All reviewed tools that support salary fields default to a single currency, typically USD. For Pakistani job seekers who may be targeting roles paying in PKR, AED, SAR, GBP, or EUR depending on whether they are applying locally or to international remote positions, single-currency salary fields are insufficient. The Job Tracker provides a searchable currency selector covering thirty major currencies with PKR prominently included.
- **Gap 3 - No Pakistani Market Adaptation:** All reviewed tools are designed for Western job markets. The currency defaults, platform integrations, and market assumptions built into these tools do not reflect the reality of job searching in Pakistan, where platforms such as Rozee.pk are dominant and salary norms differ significantly from

North American benchmarks. A locally developed tool inherently addresses this gap by being designed from the outset for the Pakistani context.

- **Gap 4 - Cost Barriers to Full Functionality:** Every commercial tool reviewed - Huntr, Teal, and Jobscan - places significant functionality behind a paid subscription. For university students and fresh graduates in Pakistan, subscription costs in USD are effectively prohibitive. The Job Tracker provides its core functionality entirely free, with a one-time optional premium upgrade at a locally accessible price point.
- **Gap 5 - No Comprehensive Status Pipeline with Visual Differentiation:** Spreadsheets and general productivity tools require users to define and maintain their own status conventions. Dedicated tools such as Huntr use a Kanban board that is visually engaging but lacks the list-based density preferred for managing large numbers of applications efficiently. The Job Tracker provides a structured seven-stage pipeline with color-coded status badges that deliver immediate visual clarity in a list-based interface optimized for large application sets.
- **Gap 6 - No Unified Analytics for Job Search Progress:** None of the free tools reviewed provides a meaningful analytics dashboard. Users of spreadsheets, Huntr's free tier, and Notion have no automated way to see how many applications they have submitted this week, what proportion have progressed to interview stage, or how their application activity has trended over time. The Job Tracker's statistics endpoint and dashboard provide these insights automatically.

Table 2.2 - Identified Research Gaps

Gap ID	Identified Gap	Impact on User	Solution in This Project
GAP-01	No integrated reminder system in any free tool	Missed follow-up opportunities	Server-side cron reminder service
GAP-02	No multi-currency salary tracking	Cannot track international salaries	Searchable 30-currency selector
GAP-03	No Pakistani market adaptation	Irrelevant defaults and pricing	PKR/AED/SAR defaults, free tier
GAP-04	Cost barriers to full functionality	Students cannot afford paid tools	Free core features, one-time premium
GAP-05	No visual status pipeline differentiation	Difficulty identifying application states	Seven-stage color-coded status badges
GAP-06	No unified job search analytics	No visibility into search progress	Statistics endpoint and dashboard

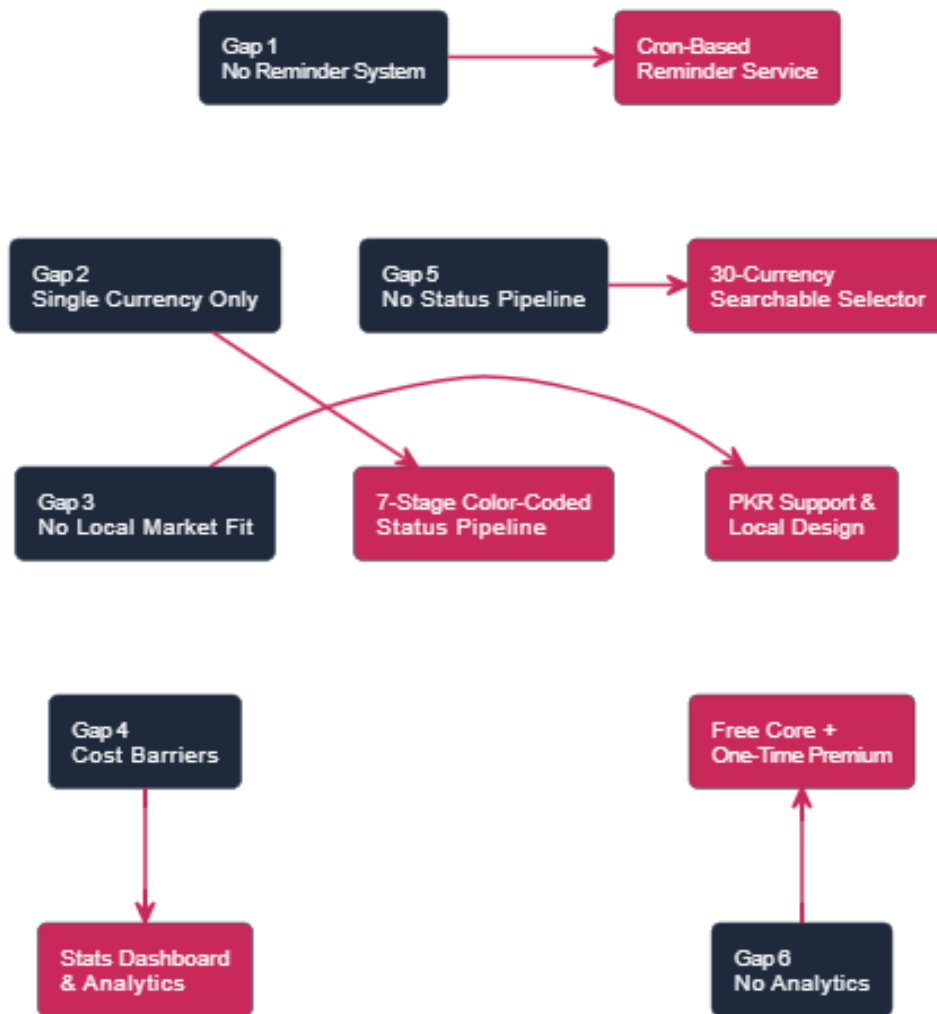


Figure 2.3 - Research Gap to Solution Mapping

2.5 Proposed Solution Overview

2.5.1 Solution Architecture

The Job Tracker addresses all six identified gaps through a two-tier web application architecture. The presentation tier is a React.js single-page application styled with Tailwind CSS and animated with Framer Motion, providing a responsive and visually polished interface for all application management tasks. The backend tier is a Node.js and Express.js server exposing a RESTful API, connected to a MongoDB database through Mongoose ODM for flexible, document-based data storage.

A background services layer operates within the backend process, running a node-cron scheduled task that checks for due reminders on an hourly basis. This service operates independently of user interaction, ensuring that reminder processing occurs reliably without requiring the user to have the application open.

Authentication is handled end-to-end through JSON Web Tokens. The frontend stores the JWT in localStorage and injects it into every API request via an Axios request interceptor. The backend validates the token on every protected endpoint before processing the request.

2.5.2 Key Innovation Points

The key technical and product innovations of this system relative to existing tools are:

- ✓ **Innovation 1:** A server-side cron job reminder service that runs independently of user sessions, identifies applications with due follow-up dates, and marks them as notified - providing reliable, background-driven reminder processing without requiring third-party notification services.
- ✓ **Innovation 2:** A searchable currency selector supporting thirty international currencies, making the salary tracking feature genuinely useful for Pakistani job seekers targeting both local and international remote opportunities.
- ✓ **Innovation 3:** A structured seven-stage status pipeline with color-coded visual differentiation, combined with a three-level priority system, that gives users both pipeline visibility and urgency management in a single unified list view.
- ✓ **Innovation 4:** A statistics aggregation endpoint that computes application counts by status, weekly activity, and monthly activity in a single database query using MongoDB aggregation, delivering an instant overview of job search progress without any manual calculation.
- ✓ **Innovation 5:** A freemium model with a completely free core tier and a simple one-time premium upgrade that unlocks unlimited applications - a pricing structure deliberately chosen to make the tool accessible to Pakistani students while establishing a path to sustainability.

2.6 Summary of Related Background Research

The following table summarizes key concepts, technologies, and prior work that informed the design and development decisions made in this project.

Table 2.3 - Background Research Summary

Reference Topic	Key Concepts	Relevance to This Project
JWT Authentication in Web Apps	Stateless auth, token expiry, Bearer scheme	Used for all API authentication
RESTful API Design with Express.js	Resource-based URLs, HTTP methods, consistent responses	Backend API architecture
MongoDB Schema Design	Document model, embedded subdocuments, indexes	Database design for applications

Reference Topic	Key Concepts	Relevance to This Project
React.js SPA Architecture	Component hierarchy, Context API, React Router	Frontend application structure
Cron Job Scheduling in Node.js	node-cron, scheduled tasks, background processing	Reminder service implementation
Tailwind CSS Design Systems	Utility-first CSS, responsive breakpoints	UI styling and responsive layout
Freemium Pricing Models	Free tier limits, one-time vs subscription pricing	Premium subscription model design
Job Search Behavior Research	Application volume, follow-up importance, tracking needs	Feature prioritization rationale

Chapter 3: Requirement Analysis & Specification

3.1 Introduction to Requirements

Requirements elicitation for the Job Tracker was conducted through a combination of methods. Direct analysis of the existing codebase - including the frontend page components, backend API controllers, database models, and service layers - provided the primary and most authoritative source of requirements, as the implemented system represents the fully realized specification. Comparative analysis of existing job tracking tools reviewed in Chapter 2 provided a benchmark against which functional completeness was assessed. User experience principles drawn from modern productivity application design informed the non-functional requirements related to responsiveness, animation, and interface feedback.

The requirements are organized into functional requirements, which describe what the system must do, and non-functional requirements, which describe how well the system must perform those functions. All requirements are traceable to the project objectives defined in Chapter 1.

3.2 Stakeholder Analysis

3.2.1 Registered User (Job Seeker)

The registered user is the sole primary actor in the Job Tracker system. This stakeholder is an individual actively engaged in a job search - typically a student, fresh graduate, or working professional seeking a career transition. The registered user interacts with every functional area of the system: creating and managing application records, monitoring their pipeline status, setting follow-up reminders, reviewing analytics, and managing their account settings.

The registered user's primary goals are to maintain an organized record of all active and past applications, to never miss a follow-up opportunity, and to maintain a clear picture of their job search progress at all times. Their secondary goals include being able to quickly locate specific applications through search and filtering, and to present their job search data to themselves in a meaningful, motivating way through the dashboard.

3.2.2 Premium User

The premium user is a registered user who has completed the one-time upgrade payment. In addition to all standard user capabilities, the premium user benefits from unlimited job application storage - the free tier is capped at twenty-five applications - and access to any additional features that may be designated as premium in future releases. The premium upgrade is a one-time transaction with no recurring charges, reflecting a product philosophy of accessible pricing over subscription dependence.

3.2.3 System (Background Services)

The system itself acts as a non-human stakeholder through its background reminder service. The cron job process runs on the server at scheduled intervals, queries the database for applications with due or upcoming follow-up reminders, and marks those reminders as notified. This stakeholder requires no human interaction and operates autonomously as long as the server is running.

3.3 Functional Requirements

3.3.1 User Authentication and Account Management Module

This module covers all aspects of user identity management, including registration, login, session management, profile updates, and the premium upgrade pathway.

Table 3.1 - User Authentication Functional Requirements

Req. ID	Description	Actor	Priority
FR-AUTH-01	Register with name, email, and password	Admin/User	High
FR-AUTH-02	Authenticate user and issue JWT token	All	High
FR-AUTH-03	Enforce JWT-based session management	All	High
FR-AUTH-04	Get current authenticated user profile	All	High
FR-AUTH-05	Update user name and email	Registered User	Medium
FR-AUTH-06	Upgrade free account to premium	Registered User	Medium
FR-AUTH-07	Hash passwords using bcryptjs before storage	System	High
FR-AUTH-08	Expire tokens after 30 days	System	High

The registration endpoint validates that the provided email address is not already associated with an existing account before creating a new user record. Passwords are hashed using bcryptjs before storage and are never returned in any API response. The login endpoint returns both a JWT token and the user object, allowing the frontend to immediately populate the user context without a separate profile fetch.

The premium upgrade endpoint checks whether the user is already a premium member before processing the upgrade, preventing duplicate upgrade requests. The upgrade is recorded with a timestamp in the premiumPurchaseDate field, providing an audit trail of when the upgrade occurred.

3.3.2 Job Application Management Module

This module is the core functional area of the system, covering the full lifecycle of a job application record from creation through deletion.

Table 3.2 - Job Application Management Requirements

Req. ID	Description	Actor	Priority
FR-JOB-01	Create application with all required and optional fields	Registered User	High
FR-JOB-02	Retrieve paginated application list with filters	Registered User	High
FR-JOB-03	Retrieve single application by ID	Registered User	High
FR-JOB-04	Update any application field	Registered User	High
FR-JOB-05	Delete an application record	Registered User	High
FR-JOB-06	Enforce 25-application limit for free users	System	High
FR-JOB-07	Search by company or position text	Registered User	Medium
FR-JOB-08	Filter by application status	Registered User	Medium
FR-JOB-09	Sort by date, company, or priority	Registered User	Medium
FR-JOB-10	Return total count and pagination metadata	System	Medium

The application creation endpoint accepts a comprehensive data payload including company name, position title, location, status, job type, work mode, job URL, description, personal notes, salary minimum, salary maximum, salary currency, priority level, tags array, applied date, and optional follow-up reminder configuration. All fields except company name and position title are optional, allowing users to create a minimal record quickly and enrich it with additional detail later.

The free tier enforcement is implemented in the create endpoint, which counts the user's existing applications before processing a new creation request. If the count has reached or exceeded twenty-five and the user does not have premium status, a 403 response is returned with a message directing the user to upgrade.

3.3.3 Status Pipeline Management Module

The status pipeline defines the structured set of stages through which a job application can move. The system enforces a controlled vocabulary for application status rather than allowing free-text entry, ensuring consistency across all records and enabling meaningful aggregation.

Table 3.3 - Status Pipeline Requirements

Status Value	Display Label	Intended Use	Visual Color
wishlist	Wishlist	Opportunities saved for future consideration	Slate Gray
applied	Applied	Application submitted to employer	Blue
interviewing	Interviewing	Active interview process underway	Amber
offer	Offer	Job offer received from employer	Emerald Green
rejected	Rejected	Application declined by employer	Red
accepted	Accepted	Offer accepted by user	Green
withdrawn	Withdrawn	Application withdrawn by user	Gray

Status transitions are not constrained by the system - a user may move an application from any status to any other status at any time, reflecting the unpredictable nature of real hiring processes where, for example, a rejected application may be reconsidered or a withdrawn application may be resubmitted.

3.3.4 Follow-Up Reminder Module

The reminder module enables users to attach a specific follow-up date to any application and provides server-side processing to identify when those dates have been reached.

Table 3.4 - Reminder Module Functional Requirements

Req. ID	Description	Actor	Priority
FR-REM-01	Enable reminder with date on application creation	Registered User	High
FR-REM-02	Enable reminder with date on application update	Registered User	High
FR-REM-03	Retrieve upcoming reminders within next 7 days	Registered User	High
FR-REM-04	Cron job runs hourly to check due reminders	System	High

Req. ID	Description	Actor	Priority
FR-REM-05	Mark reminders as notified after processing	System	High
FR-REM-06	Dedicated reminder update endpoint	Registered User	Medium
FR-REM-07	Display reminder count on dashboard	System	Medium

The reminder retrieval endpoint returns all applications belonging to the authenticated user that have a reminder enabled, a reminder date within the next seven days from the current date, and a notified status of false. This endpoint powers both the dashboard reminder widget and the dedicated reminders page.

The cron service, implemented using the node-cron library, is scheduled with the expression `0 * * * *`, which triggers execution at the start of every hour. On each execution, the service queries for all applications across all users where the reminder is enabled, the reminder date is in the past or present, and the notified flag is false. Each matching application is updated to set notified to true, and a log entry is written to the server console for operational visibility.

3.3.5 Analytics and Statistics Module

The statistics module aggregates application data to provide users with a meaningful overview of their job search activity and progress.

Table 3.5 - Analytics Module Functional Requirements

Req. ID	Description	Actor	Priority
FR-STAT-01	Return total application count	System	High
FR-STAT-02	Return applications added this week	System	High
FR-STAT-03	Return applications added this month	System	Medium
FR-STAT-04	Return count by each of the seven status values	System	High
FR-STAT-05	Single API endpoint returning all statistics	System	High
FR-STAT-06	Statistics scoped to authenticated user only	System	High

The statistics endpoint uses MongoDB aggregation to compute all required metrics in a single database round trip. The aggregation pipeline matches documents belonging to the authenticated user, groups them by status to produce per-status counts, and runs parallel

computations for the weekly and monthly counts using date comparison operators. The result is a structured statistics object returned to the frontend for rendering on the dashboard.

3.3.6 Search, Filter, and Sort Module

The search, filter, and sort module enables users to efficiently navigate their application list regardless of its size.

All search, filter, and sort operations are implemented server-side in the `getJobs` controller, ensuring that only the relevant subset of data is transmitted to the frontend rather than loading all records and filtering client-side. This approach maintains performance as the number of applications grows and reduces unnecessary data transfer over the network.

3.3.7 Tags Management Module

The tags system provides users with a flexible, user-defined categorization mechanism that complements the structured status and priority fields.

Tags are stored as an array of strings on each application document in MongoDB. During application creation and update, the frontend sends tags as a comma-separated string in the form field, which is split, trimmed, and filtered to remove empty entries before storage. On the frontend, tags are rendered as small badge elements within each application card, providing visual categorization at a glance.

3.3.8 Premium Subscription Module

The premium subscription module manages the distinction between free and premium user tiers, enforcing feature limitations for free users and providing a straightforward upgrade pathway.

Table 3.6 - Premium Subscription Module Requirements

Req. ID	Description	Actor	Priority
FR-PREM-01	Free tier limited to 25 applications	System	High
FR-PREM-02	403 response when limit reached with upgrade message	System	High
FR-PREM-03	One-time upgrade endpoint available	Registered User	High
FR-PREM-04	isPremium flag stored on user document	System	High
FR-PREM-05	premiumPurchaseDate recorded on upgrade	System	Medium
FR-PREM-06	Premium status reflected in user profile response	System	Medium

Req. ID	Description	Actor	Priority
FR-PREM-07	Frontend displays upgrade prompt when limit approached	System	Medium

3.4 Non-Functional Requirements

Non-functional requirements define the quality attributes that the system must exhibit in addition to its functional capabilities.

Table 3.7 - Non-Functional Requirements

NFR ID	Category	Requirement	Metric
NFR-01	Performance	API response time for standard operations	< 500ms
NFR-02	Security	Passwords hashed using bcryptjs	bcrypt salt rounds >= 10
NFR-03	Security	JWT access tokens with defined expiry	30-day TTL
NFR-04	Scalability	MongoDB document model supports data growth	No schema migration needed
NFR-05	Usability	Fully responsive across all screen sizes	Mobile, tablet, desktop
NFR-06	Availability	Server runs continuously with cron active	Cron fires every hour
NFR-07	Maintainability	RESTful API with consistent response structure	Consistent JSON format
NFR-08	Reliability	All user data scoped by user ID	No cross-user data leakage

3.5 Use Case Modeling

3.5.1 System-Level Use Case Diagram

The system-level use case diagram presents the two primary actors - the Registered User and the System (cron service) - and their interactions with the five major functional modules of the Job Tracker: Authentication, Application Management, Reminder Management, Analytics, and Account Settings.

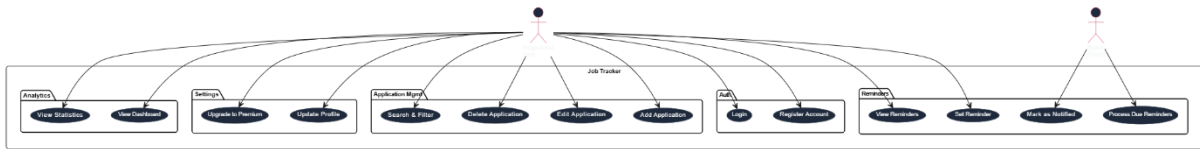


Figure 3.1 - System Level Use Case Diagram

3.5.2 Authentication Use Case Diagram

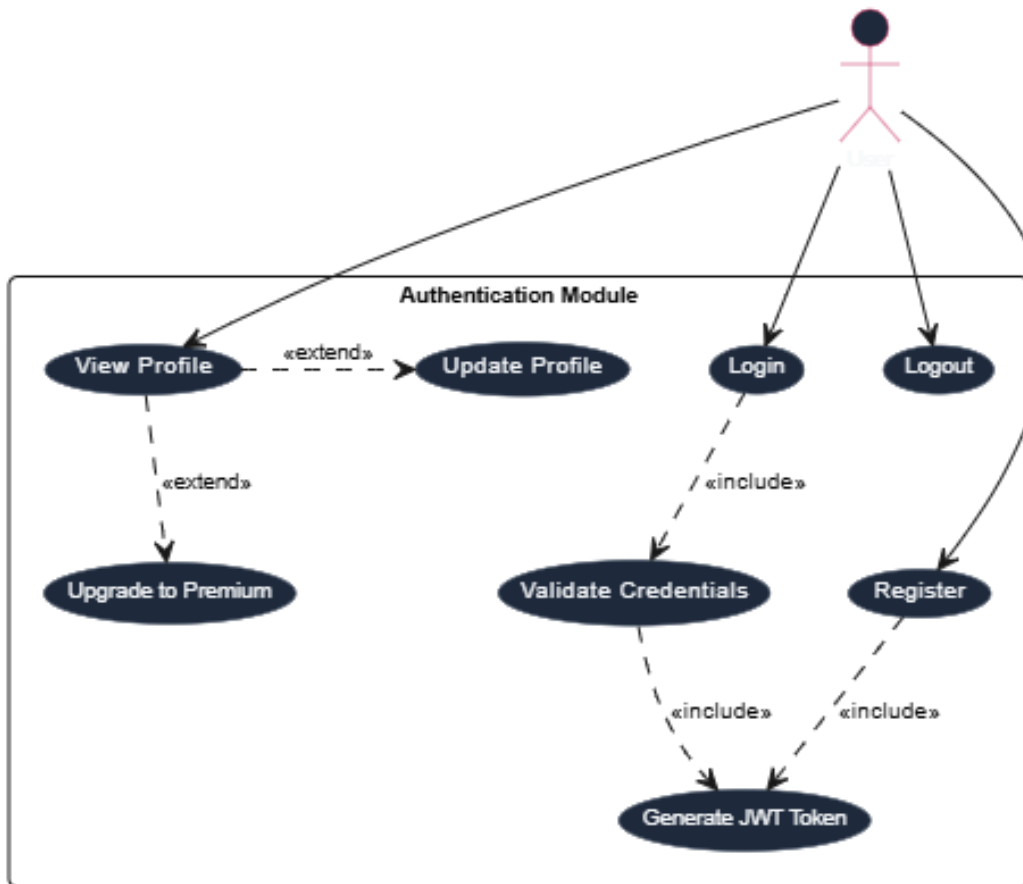


Figure 3.2 - Authentication Use Case Diagram

3.5.3 Application Management Use Case Diagram

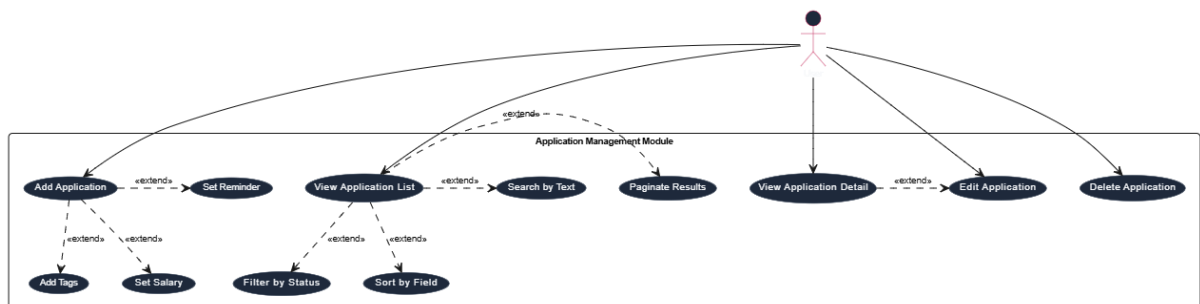


Figure 3.3 - Application Management Use Case Diagram

3.5.4 Reminder Management Use Case Diagram

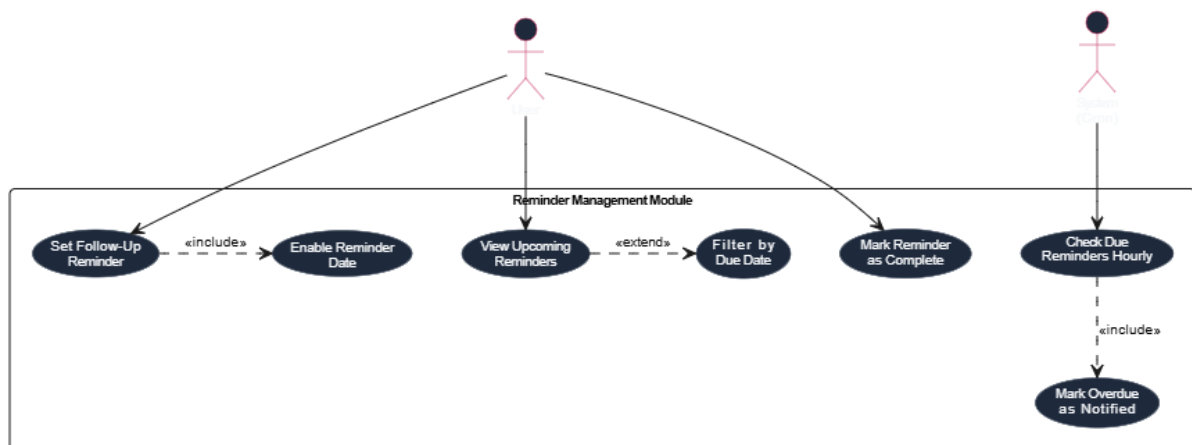


Figure 3.4 - Reminder Management Use Case Diagram

3.5.5 Detailed Use Case Specifications

Use Case UC-01: Add New Job Application

Use Case UC-02: Set and View Follow-Up Reminder

Use Case UC-03: Search and Filter Applications

3.6 User Stories

User stories capture the functional requirements from the end user's perspective, providing a human-centered framing of each feature alongside its acceptance criteria.

Table 3.8 - User Stories

ID	Role	User Story	Acceptance Criteria
US-01	Job Seeker	I want to add a new application with all details	Form accepts all fields, mandatory validation, success toast
US-02	Job Seeker	I want to see all applications with current status	List shows status badge, company, position, date
US-03	Job Seeker	I want to set a follow-up reminder on any application	Reminder date can be set, reminders page shows upcoming
US-04	Job Seeker	I want to search for a specific application by name	Case-insensitive search returns results within 500ms
US-05	Job Seeker	I want to see a dashboard with my statistics	Dashboard shows total, weekly count, and per-status counts

ID	Role	User Story	Acceptance Criteria
US-06	Job Seeker	I want to upgrade to premium for unlimited applications	Upgrade button available, isPremium set, unlimited access

3.7 Hardware and Software Requirements

3.7.1 Hardware Requirements

Table 3.9 - Hardware Requirements

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i3 / Dual-Core 2GHz	Intel Core i5 / Quad-Core 2.5GHz+
RAM	4 GB	8 GB or more
Storage	20 GB free space	SSD 50 GB+
Network	5 Mbps Internet	20 Mbps+
Operating System	Windows 10 / Ubuntu 20.04	Ubuntu 22.04 LTS

3.7.2 Software and Technology Stack

Table 3.10 - Software and Technology Stack

Layer	Technology	Version	Purpose
Frontend	React.js	18.x	Single Page Application
Frontend	Vite	5.x	Build Tool and Dev Server
Frontend	Tailwind CSS	3.x	Utility-First Styling
Frontend	Framer Motion	10.x	UI Animations and Transitions
Frontend	Axios	1.x	HTTP Client with Interceptors
Frontend	React Router DOM	6.x	Client-Side Routing
Frontend	React Hot Toast	2.x	Toast Notification System
Frontend	Lucide React	0.303	Icon Library

Layer	Technology	Version	Purpose
Frontend	date-fns	3.x	Date Formatting and Comparison
Backend	Node.js	18+	Server Runtime
Backend	Express.js	4.x	REST API Framework
Backend	Mongoose	8.x	MongoDB ODM
Backend	jsonwebtoken	9.x	JWT Authentication
Backend	bcryptjs	2.x	Password Hashing
Backend	node-cron	3.x	Scheduled Background Tasks
Backend	dotenv	16.x	Environment Variable Management
Backend	cors	2.x	Cross-Origin Request Handling
Backend	express-validator	7.x	Request Input Validation
Database	MongoDB	6.x	NoSQL Document Database

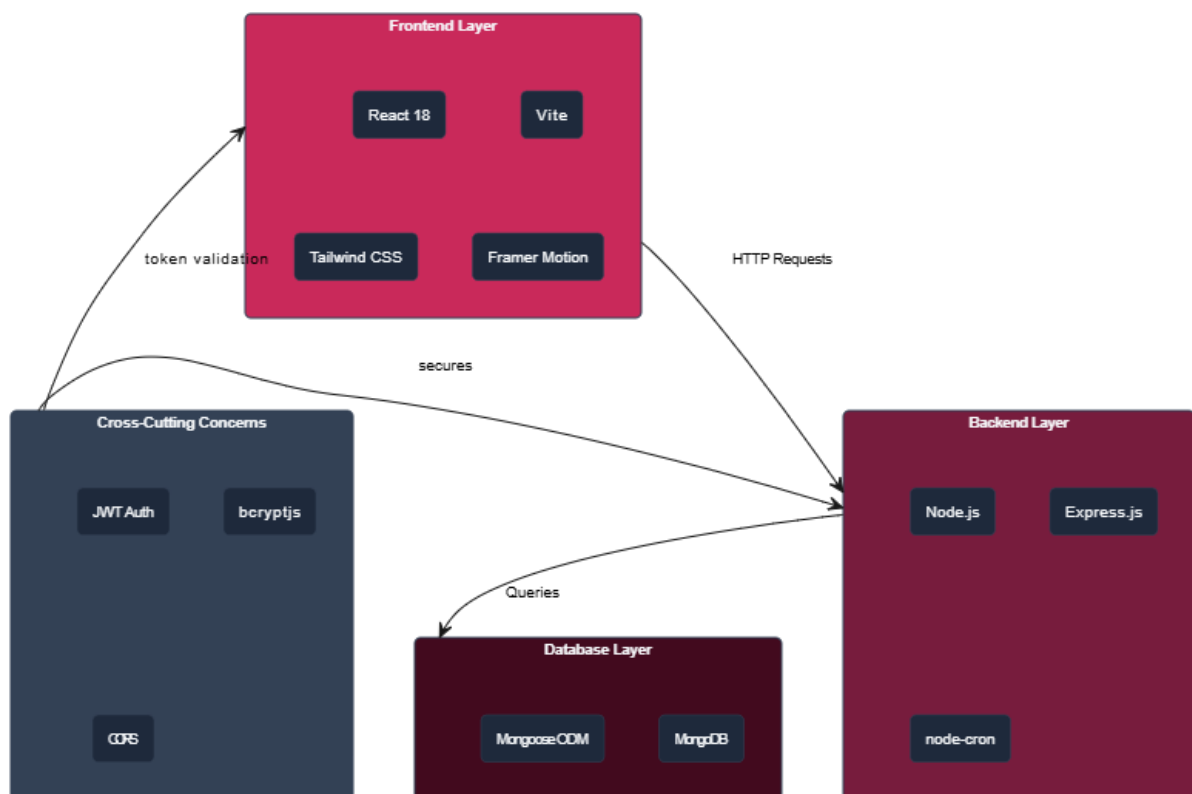


Figure 3.5 - Technology Stack Architecture Diagram

3.8 Assumptions and Constraints

Table 3.11 - Assumptions and Constraints

Type	Item ID	Description
Assumption	A-01	All users have access to a modern web browser with JavaScript enabled
Assumption	A-02	Stable internet connectivity is available for API communication
Assumption	A-03	MongoDB instance is accessible locally or via cloud provider
Assumption	A-04	Users are responsible for entering accurate application data
Constraint	C-01	System is designed for individual user deployment only
Constraint	C-02	Premium upgrade simulates payment without real gateway integration
Constraint	C-03	Reminder system notifies via application interface only - no email
Constraint	C-04	Free tier is limited to 25 applications, enforced server-side
Constraint	C-05	System requires Node.js 18+ and a running MongoDB instance

Chapter 4: System Design

4.1 Design Principles and Methodology

4.1.1 Design Approach

The Job Tracker follows a layered architectural approach based on the separation of concerns principle. Each layer of the system carries a clearly defined responsibility and communicates with adjacent layers through well-defined interfaces. The presentation layer handles only UI rendering, user interaction, and client-side state management. The business logic layer handles all application rules, data validation, authentication enforcement, and background processing. The data layer handles only persistence and retrieval of documents through the Mongoose ODM.

This separation ensures that changes in one layer do not propagate uncontrollably into others, making the codebase maintainable and extensible. The design also follows the Single Responsibility Principle at the module level - each route handler, controller function, middleware, and service is responsible for one well-defined operation. For example, the authentication middleware is solely responsible for validating the JWT token and attaching the user object to the request, while the job controller is solely responsible for executing CRUD operations on application documents.

4.1.2 API-First Design

All system functionality is exposed through a RESTful API before any frontend implementation considerations influence the backend structure. This API-first approach means the backend is independently testable and documented, and the frontend consumes the API as an external client rather than having its concerns mixed into the server layer. The consistent request and response structure maintained across all endpoints - with success, data, message, and error fields following a predictable pattern - ensures that the frontend Axios client can handle responses uniformly through shared interceptors and error handlers.

4.1.3 Document-Oriented Data Design

The choice of MongoDB as the database reflects a deliberate design decision to leverage document-oriented storage for the job application domain. Each job application is a self-contained document with nested objects for salary and follow-up reminder data, and an array field for tags. This structure maps naturally to the way application data is created, read, and updated - as a single coherent unit rather than as multiple related rows across joined tables. The document model also allows optional fields to be absent entirely from a document rather than stored as null values, reducing storage overhead for records where optional fields are not provided.

4.2 System Architecture

4.2.1 High-Level Architecture

The Job Tracker implements a classic two-tier client-server architecture with a background services component operating within the server tier. The client tier is a React.js single-page application served as static files. The server tier is a Node.js and Express.js process that handles both API requests and background cron job execution. The data tier is a MongoDB instance accessed exclusively through the server tier.

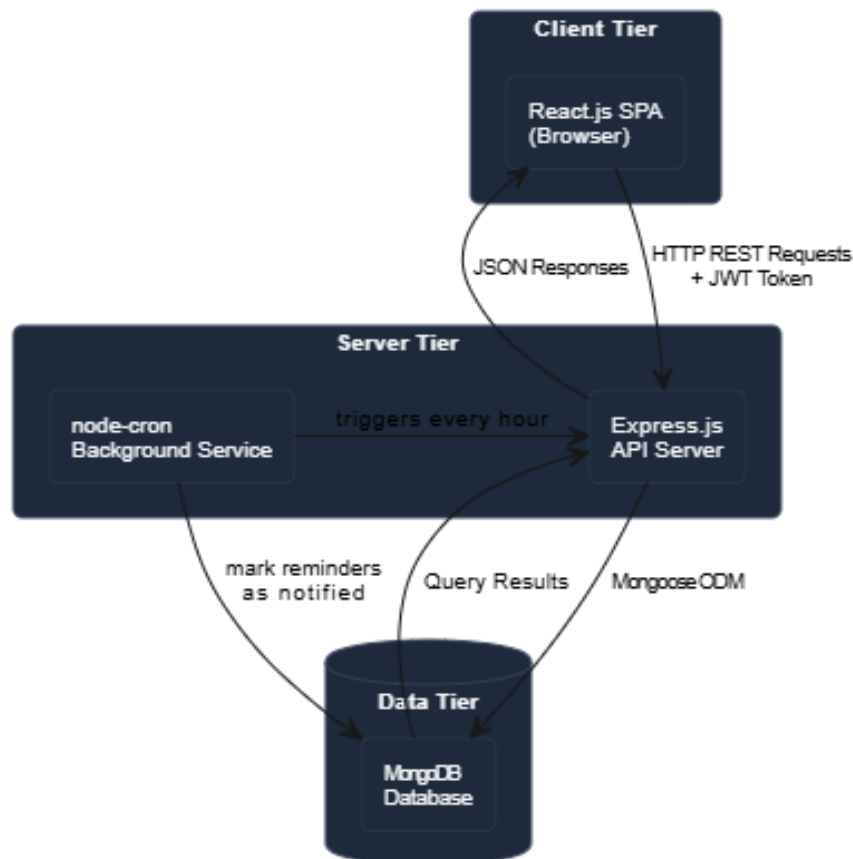


Figure 4.1 - High Level Architecture Diagram

4.2.2 Three-Tier Architecture Detail

Tier 1 - Presentation (React.js Frontend): The React.js single-page application is built with Vite and structured around a component hierarchy managed by React Router DOM for client-side navigation. Authentication state is managed through a React Context provider that stores the current user object and JWT token, making them available to all components without prop drilling. API communication is centralized through a shared Axios instance with request and response interceptors. The dark-themed Tailwind CSS design system and Framer Motion animation library provide visual consistency and interactivity across all pages.

Tier 2 - Business Logic (Express.js Backend): The Express.js server is organized into routes, controllers, middleware, models, and services following a clean separation of responsibilities. Routes define the URL patterns and HTTP methods. Controllers contain the request handling logic for each endpoint. Middleware functions handle cross-cutting concerns such as authentication, error handling, and request validation. Models define the MongoDB document schemas through Mongoose. The reminder service operates as a background module started at server initialization.

Tier 3 - Data (MongoDB with Mongoose): MongoDB stores all application data as documents within two primary collections: users and jobapplications. Mongoose schemas enforce field types, validation rules, default values, and indexes at the application level. All database interactions are performed through Mongoose model methods, ensuring consistent query construction and automatic application of schema-level defaults and middleware hooks.

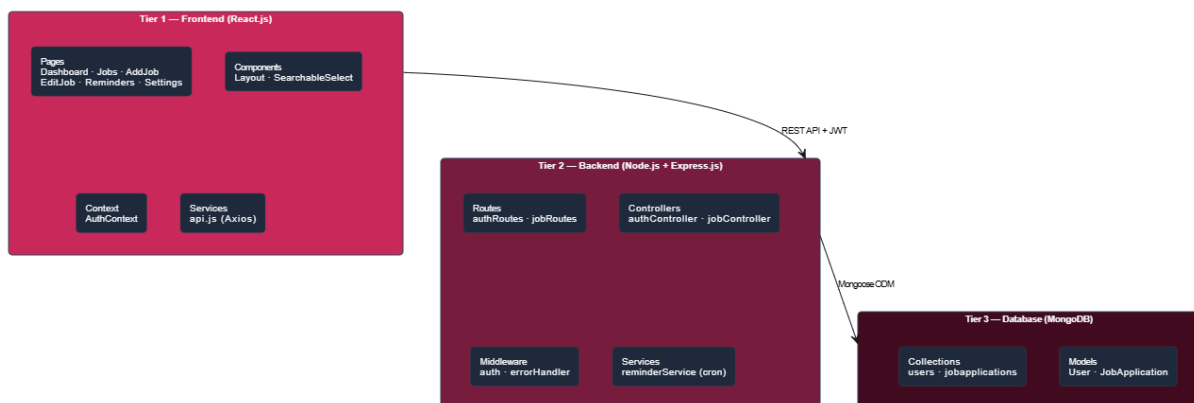


Figure 4.2 - Three Tier Architecture Diagram

4.2.3 Background Services Architecture

The reminder service operates as an autonomous background component within the Express.js server process. It is initialized by calling `startReminderService()` from `server.js` immediately after the server begins listening on its configured port. The service registers a cron job with node-cron using the schedule expression `0 * * * *`, which triggers execution at minute zero of every hour.

On each execution, the service performs a single MongoDB query that matches all `JobApplication` documents where `followUpReminder.enabled` is `true`, `followUpReminder.date` is less than or equal to the current timestamp, and `followUpReminder.notified` is `false`. For each matching document, the service sets `followUpReminder.notified` to `true` and saves the document. A console log records the number of reminders processed in each execution cycle for operational visibility.

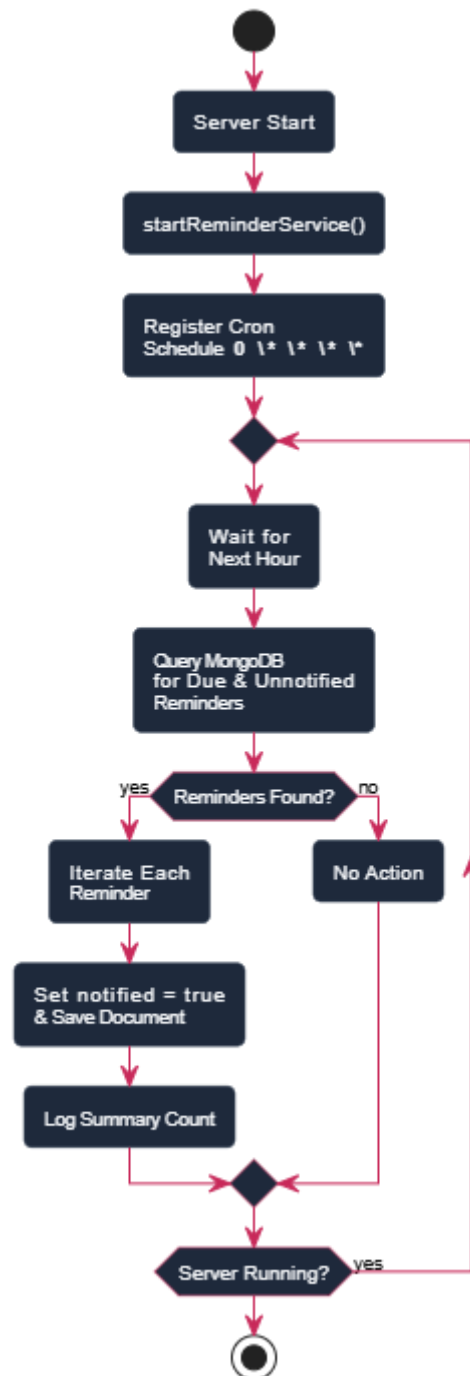


Figure 4.3 - Background Reminder Service Flow

4.3 Database Design

4.3.1 Collections Overview

The Job Tracker MongoDB database contains two primary collections. The users collection stores account information, authentication credentials, and subscription status for each registered user. The jobapplications collection stores all job application records, each linked to its owner through a user reference field.

The relationship between these collections is a one-to-many relationship - one user document is associated with many job application documents. This relationship is implemented through a reference field (user: ObjectId) on the JobApplication schema rather than through embedding, because the number of applications per user can grow large and applications require independent querying, pagination, and filtering.

4.3.2 User Schema Design

The User schema captures the minimum information required for authentication and account management. The name field stores the user's display name as a trimmed string with a maximum length of fifty characters. The email field is stored in lowercase with a uniqueness constraint enforced at both the Mongoose schema level and the MongoDB index level, preventing duplicate account registration. The password field stores the bcryptjs hash and is excluded from query results by default through the select: false option, ensuring that hashed passwords are never inadvertently returned in API responses.

The isPremium boolean flag defaults to false and is set to true by the upgrade endpoint. The premiumPurchaseDate field records the timestamp of the upgrade transaction. The createdAt field records the account creation timestamp.

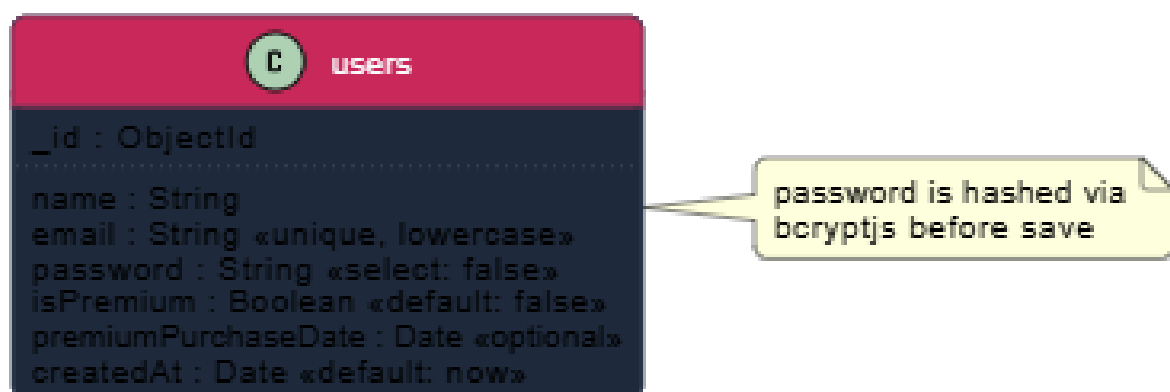


Figure 4.4 - User Document Structure

4.3.3 JobApplication Schema Design

The JobApplication schema is the central data structure of the system. It is designed to capture all information a job seeker would want to record about a single application opportunity, organized into logical field groups.

The identification fields include company (required string, max 100 characters) and position (required string, max 100 characters), which together uniquely identify the opportunity in the user's mind. The location field is an optional string defaulting to Remote. The jobUrl field stores the URL of the original job posting for reference.

The classification fields include status (enum of seven values, default applied), jobType (enum of five values: full-time, part-time, contract, internship, freelance, default full-time),

workMode (enum of three values: remote, onsite, hybrid, default remote), and priority (enum of three values: low, medium, high, default medium).

The salary subdocument contains min (Number), max (Number), and currency (String, default USD) fields. All three are optional, allowing users to record salary information when known without requiring it on every application.

The content fields include description (String, max 2000 characters) for the job posting content and notes (String, max 1000 characters) for personal observations and reminders.

The followUpReminder subdocument contains enabled (Boolean, default false), date (Date), and notified (Boolean, default false). The enabled field controls whether the reminder is active. The date field stores the target follow-up date. The notified field is managed by the cron service and prevents repeated processing of the same reminder.

The tags field is an array of strings allowing flexible user-defined categorization. The appliedDate field records when the application was submitted, defaulting to the current timestamp. The user field is a required ObjectId reference linking the application to its owner.

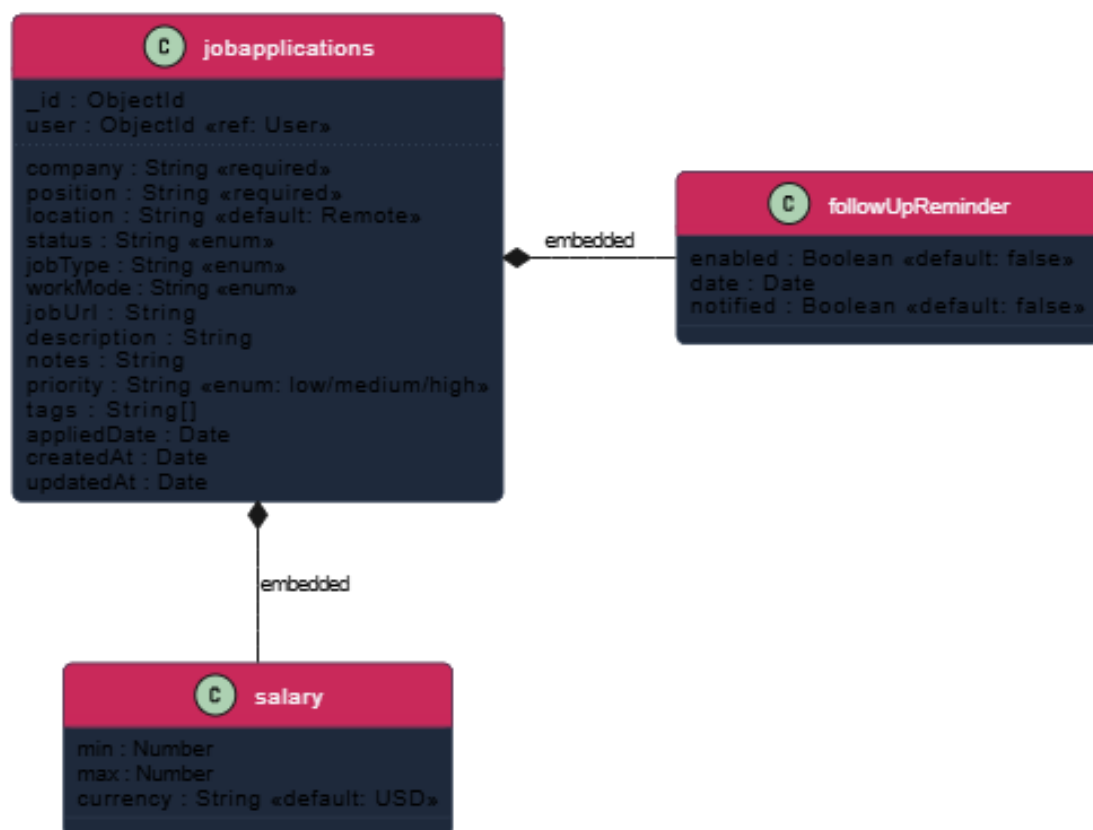


Figure 4.5 - Job Application Document Structure

4.3.4 Database Indexes

MongoDB indexes are defined to optimize the most frequent query patterns in the application. A compound index on the user and status fields accelerates the filtered application list query,

which is the most frequently executed read operation in the system. A compound index on user and createdAt with descending sort order accelerates the default newest-first list retrieval. A compound index on followUpReminder.enabled, followUpReminder.date, and followUpReminder.notified accelerates the cron service query, which runs every hour and scans the entire collection for matching reminders.

Table 4.1 - Database Indexes

Index Name	Fields	Sort Order	Purpose
idx_user_status	user, status	ASC, ASC	Filtered application list queries
idx_user_createdat	user, createdAt	ASC, DESC	Default sorted application list
idx_reminder_processing	followUpReminder.enabled, followUpReminder.date, followUpReminder.notified	ASC	Cron service hourly query

4.4 API Design

4.4.1 RESTful API Design Principles

The API follows REST architectural constraints throughout. All resources are identified by noun-based URLs organized into two resource groups: /api/auth for authentication and account management operations, and /api/jobs for job application resource operations. HTTP methods carry their standard semantic meanings - GET for retrieval, POST for creation, PUT for full replacement update, and DELETE for removal. All request and response bodies use JSON format with the Content-Type: application/json header. Authentication is enforced on all protected endpoints through a Bearer token in the Authorization header, validated by the protect middleware before the request reaches the controller.

Error responses follow a consistent structure with a success field set to false and a message field providing a human-readable explanation. Validation errors from express-validator include an errors array with field-level detail. Success responses include a success field set to true and a data payload appropriate to the operation.

4.4.2 Authentication API Endpoints

Table 4.2 - Authentication API Endpoints

Method	Endpoint	Description	Auth Required	Response
POST	/api/auth/register	Register new user	No	201 token + user object

Method	Endpoint	Description	Auth Required	Response
POST	/api/auth/login	Login with credentials	No	200 token + user object
GET	/api/auth/me	Get current user profile	Yes	200 user object
PUT	/api/auth/profile	Update name and email	Yes	200 updated user object
POST	/api/auth/upgrade	Upgrade to premium	Yes	200 user with isPremium true

4.4.3 Job Application API Endpoints

Table 4.3 - Job Application API Endpoints

Method	Endpoint	Description	Query Params	Response
GET	/api/jobs	Get paginated application list	status, search, sort, page, limit	jobs array + pagination
POST	/api/jobs	Create new application	None	201 created job document
GET	/api/jobs/stats	Get application statistics	None	stats object
GET	/api/jobs/reminders	Get upcoming reminders	None	reminders array
GET	/api/jobs/:id	Get single application	None	job document
PUT	/api/jobs/:id	Update application	None	updated job document
DELETE	/api/jobs/:id	Delete application	None	success message
PUT	/api/jobs/:id/reminder	Update reminder settings	None	updated job document

4.4.4 Authentication Flow Detail

The JWT authentication flow operates as follows. During registration, the user submits name, email, and password. The server checks for an existing account with the provided email address. If none exists, the password is hashed using bcryptjs with a salt factor of ten and a new User document is created in MongoDB. A JWT token is signed with the user's MongoDB ObjectId as the subject claim, using the JWT_SECRET environment variable and an expiry of thirty days. The token and user object are returned to the client.

During subsequent requests, the client attaches the token as a Bearer token in the Authorization header. The protect middleware extracts the token, verifies its signature and expiry using jsonwebtoken, decodes the user ID from the subject claim, and queries MongoDB to retrieve the full user object. The user object is attached to the request as req.user and made available to the downstream controller. If the token is missing, malformed, or expired, a 401 Unauthorized response is returned immediately without reaching the controller.

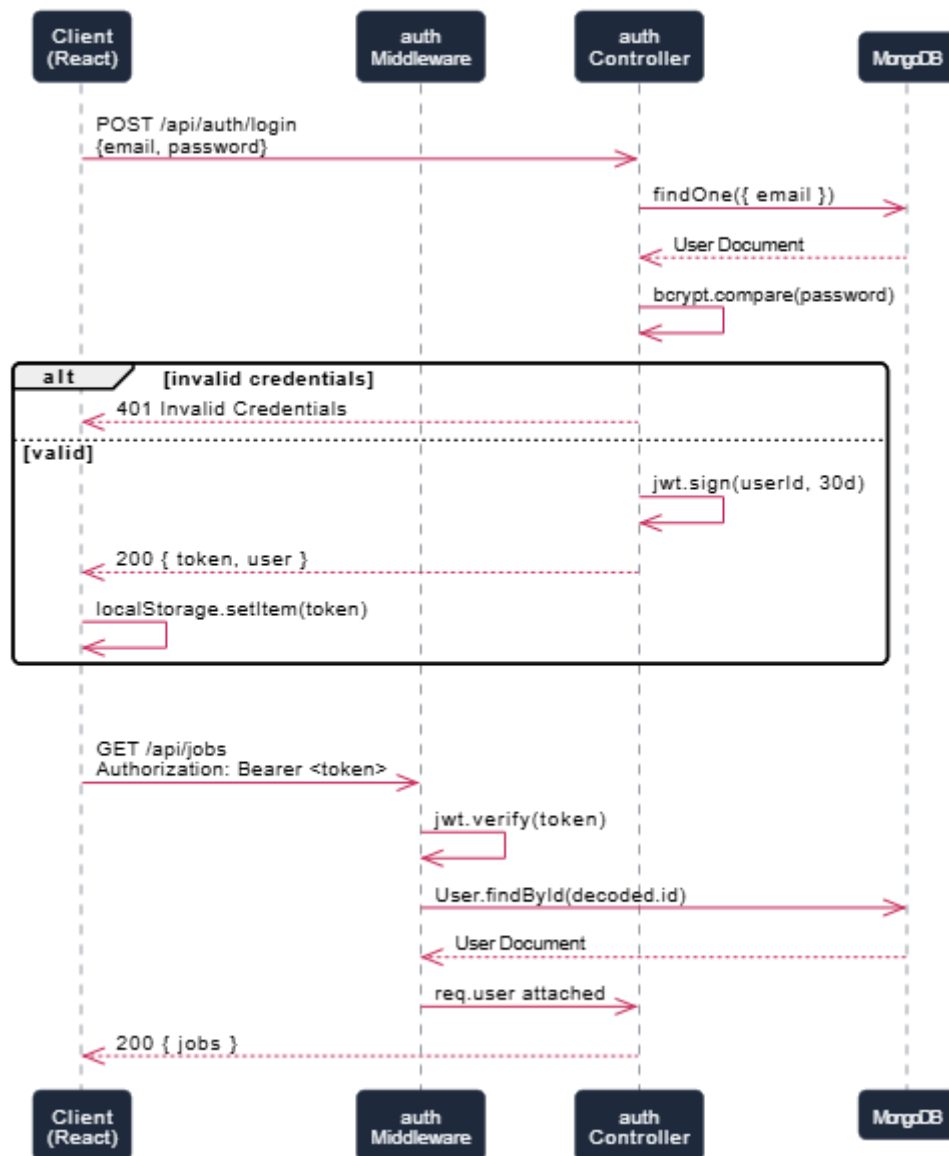


Figure 4.6 - JWT Authentication Flow Diagram

4.4.5 API Response Structure Standards

All API responses follow a consistent JSON structure to enable uniform handling on the frontend. Success responses for single-resource operations return a success flag set to true alongside the resource data. Success responses for list operations return a success flag, the array of results, and pagination metadata including total count, current page, and total pages.

Error responses return a success flag set to false and a descriptive message. Validation error responses additionally include an errors array with per-field error details.

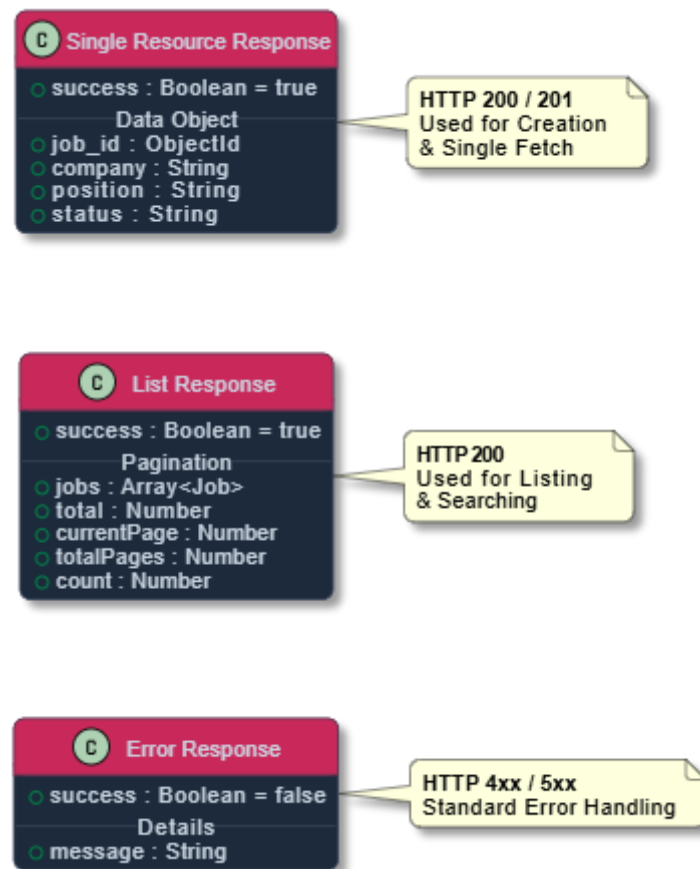


Figure 4.7 - API Response Structure Diagram

4.5 Process Design

4.5.1 Data Flow Diagram - Level 0 (Context Diagram)

The context diagram represents the Job Tracker as a single system process interacting with two external entities. The Registered User provides authentication credentials, application data, and filter parameters, and receives authenticated sessions, application records, statistics, and reminder alerts in return. The Cron Scheduler is an external time-based trigger that initiates the reminder processing operation at scheduled intervals, with the system returning reminder status updates in response.

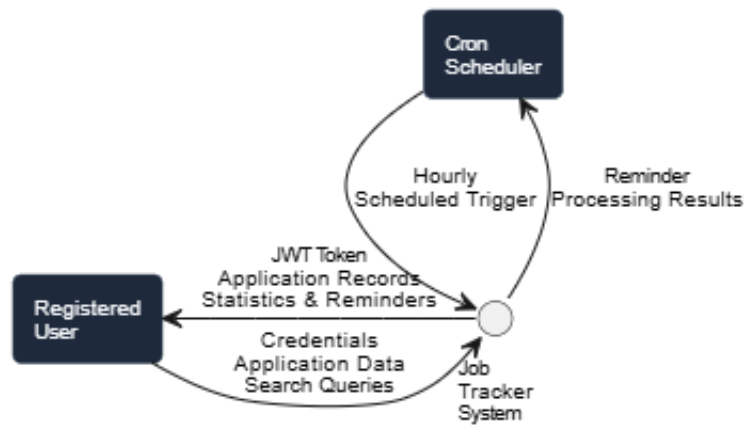


Figure 4.8 - DFD Level 0 Context Diagram

4.5.2 Data Flow Diagram - Level 1

The Level 1 DFD decomposes the Job Tracker system into five major internal processes: User Authentication, Application Management, Reminder Management, Statistics Aggregation, and Background Reminder Processing. Each process interacts with the relevant data stores - the Users data store and the Job Applications data store - through labeled read and write data flows.

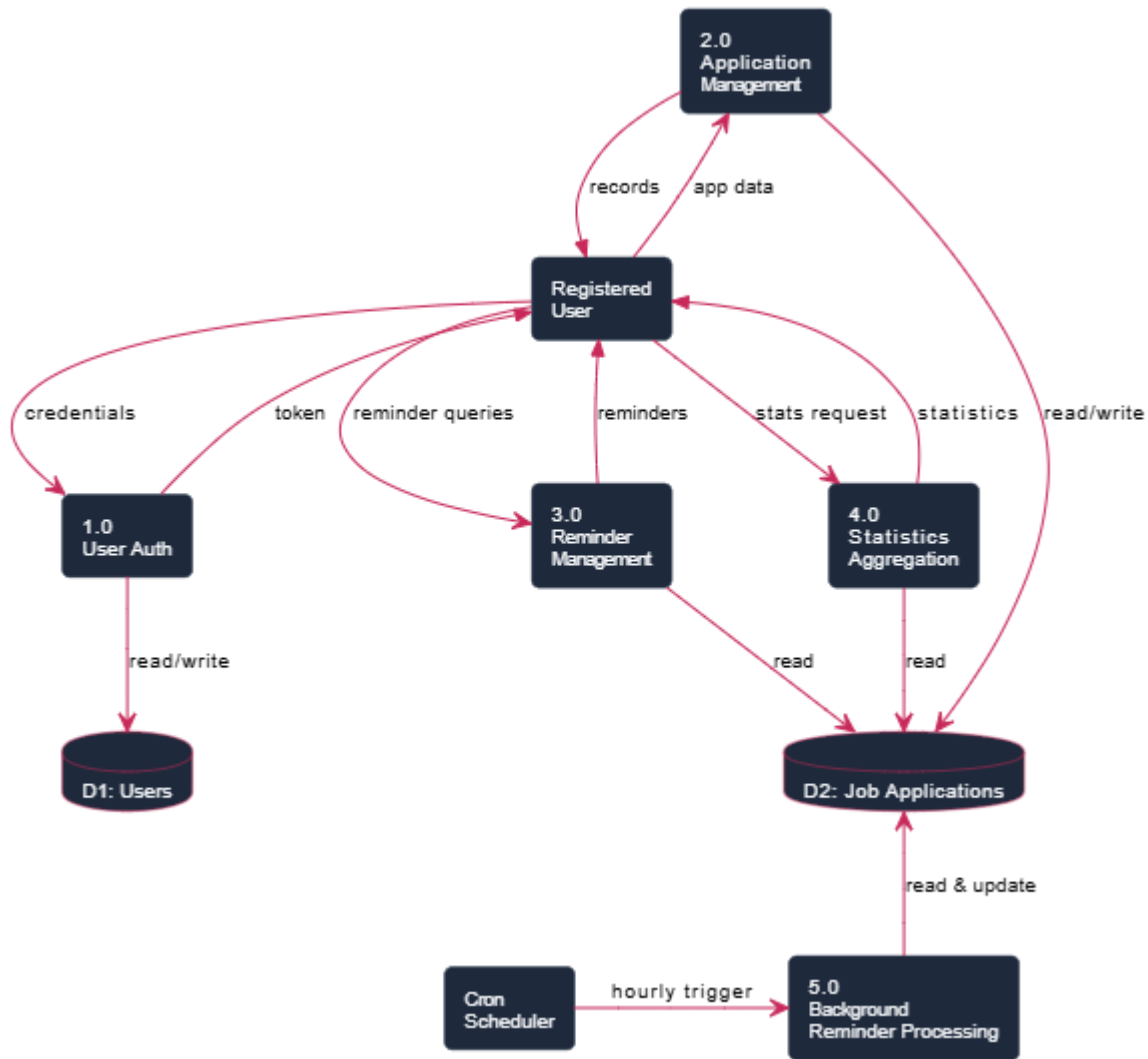


Figure 4.9 - DFD Level 1 Diagram

4.5.3 Sequence Diagram - Add New Job Application Flow

When a user submits the Add Application form, the React frontend collects all form field values and constructs a data object. The Axios client appends the JWT Bearer token to the request headers through the request interceptor and sends a POST request to `/api/jobs`. The Express router directs the request to the `createJob` controller through the `protect` middleware. The middleware validates the token and attaches the user object to the request. The controller checks the user's `isPremium` status and current application count - if the free tier limit has been reached, a 403 response is returned immediately. Otherwise, the controller sets `req.body.user` to the authenticated user's ID and calls `JobApplication.create()` with the full request body. The newly created document is returned with a 201 status code. The frontend receives the success response, displays a toast notification, and navigates the user to the applications list page.

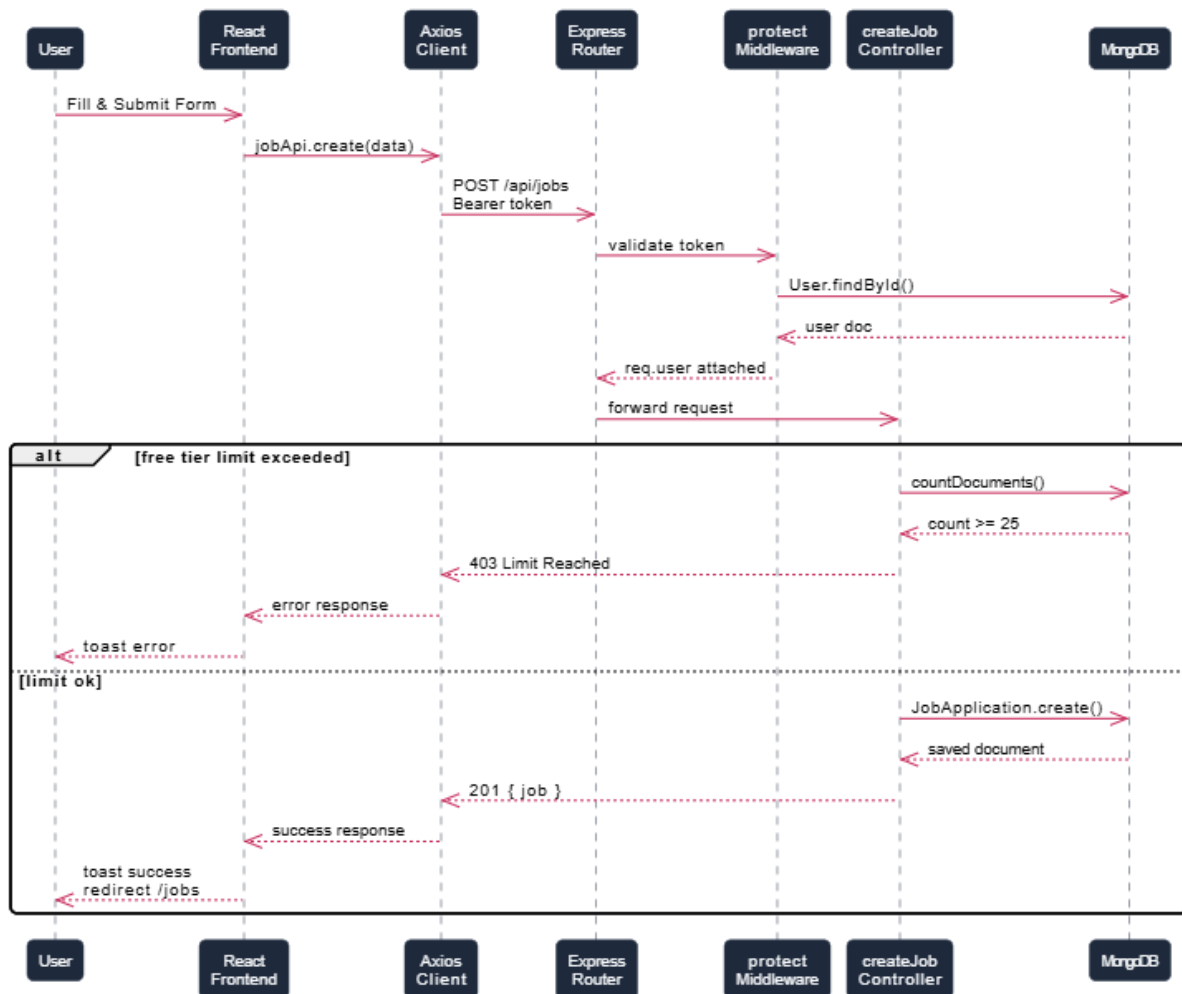


Figure 4.10 - Add Application Sequence Diagram

4.5.4 Sequence Diagram - Reminder Processing Flow

The cron job fires at the start of each hour as scheduled by node-cron. The startReminderService function's scheduled callback begins execution. A MongoDB query is constructed using the JobApplication model, filtering for documents where followUpReminder.enabled is true, followUpReminder.date is less than or equal to the current Date object, and followUpReminder.notified is false. The query returns an array of matching documents, each populated with the associated user's email and name fields for logging purposes. The service iterates over each matching document, sets followUpReminder.notified to true, and calls save() on the document. After all documents are processed, the service logs the total count of processed reminders to the server console.

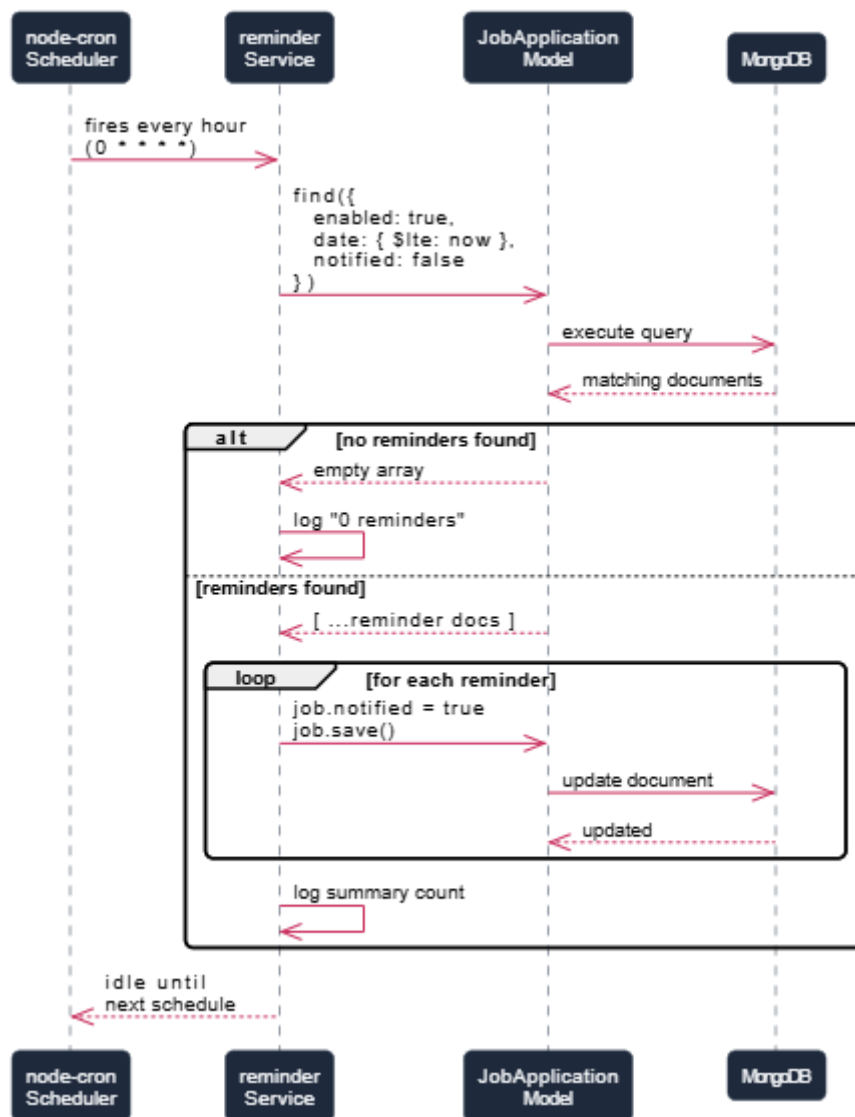


Figure 4.11 - Reminder Processing Sequence Diagram

4.5.5 Sequence Diagram - User Login and Token Storage Flow

The user navigates to the login page and submits their email and password. The frontend Login component calls the login function from AuthContext, which calls the `api.post('/auth/login')` Axios method. The Express server receives the request on the `POST /api/auth/login` route and passes it to the login controller. The controller queries MongoDB for a user with the provided email, including the password field which is excluded by default. If no user is found, a 401 response is returned. If a user is found, `bcryptjs.compare()` is called to verify the provided password against the stored hash. If the comparison fails, a 401 response is returned. If successful, the `generateToken` function signs a JWT with the user's ID and thirty-day expiry. The token and user object are returned in the response body. The AuthContext login function stores the token in `localStorage`, updates the token state, and sets the user state. The App component's route guard detects the authenticated state and navigates the user to the dashboard.

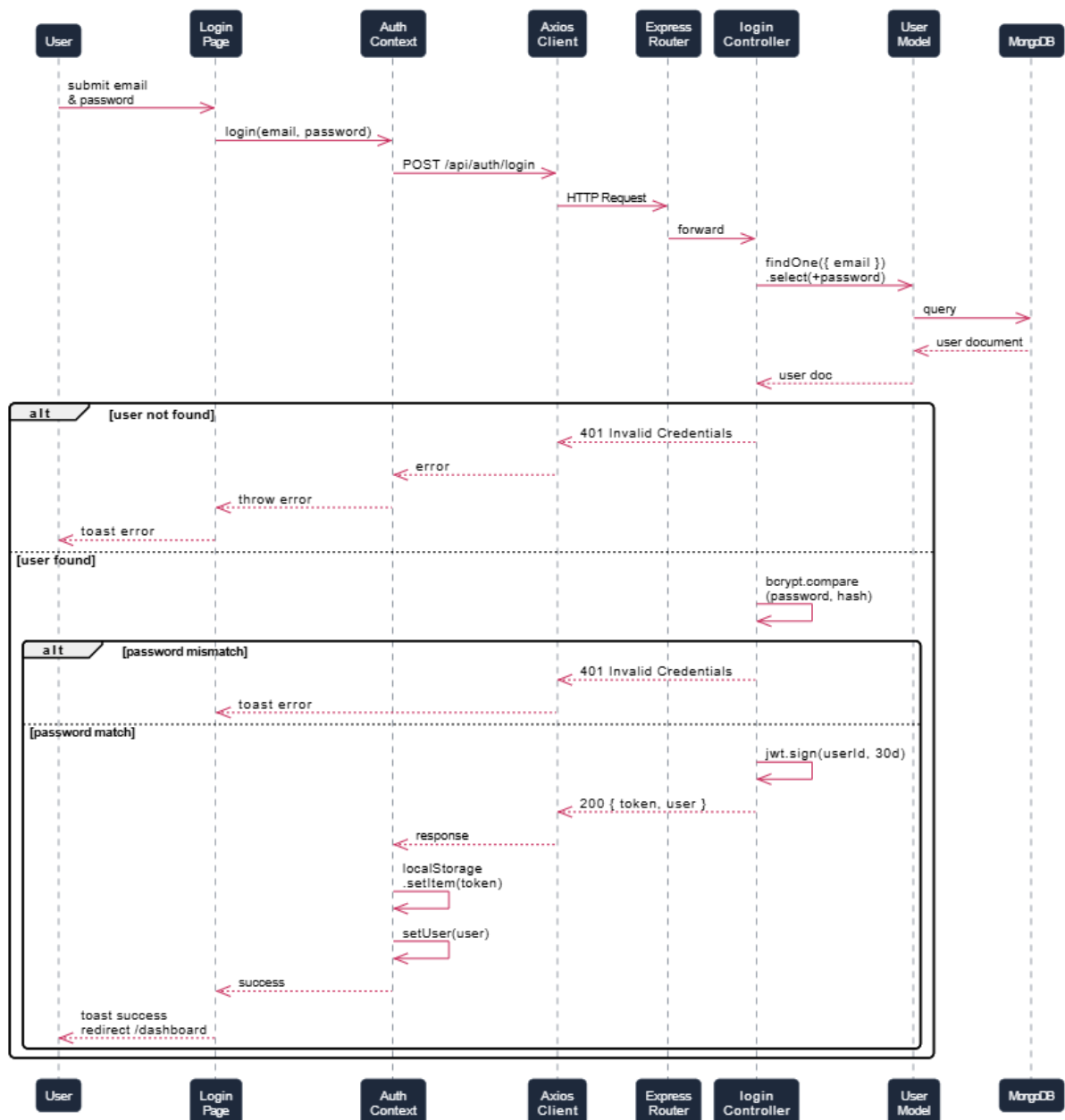


Figure 4.12 - User Login Sequence Diagram

4.5.6 Activity Diagram - Job Application Status Update Flow

The status update flow begins when a user opens an existing application in the edit form. The current status is pre-populated in the status dropdown. The user selects a new status from the seven available options. The user saves the form, which triggers the handleSubmit function. The frontend constructs the updated data object and sends a PUT request to /api/jobs/:id. The protect middleware validates the token. The updateJob controller queries for the application document matching both the provided ID and the authenticated user's ID, preventing a user from modifying another user's records. If the document is not found, a 404 response is returned. If found, findByIdAndUpdate is called with the new status value and the

new: true option to return the updated document. The updated document is returned to the frontend, which displays a success toast and navigates back to the applications list.

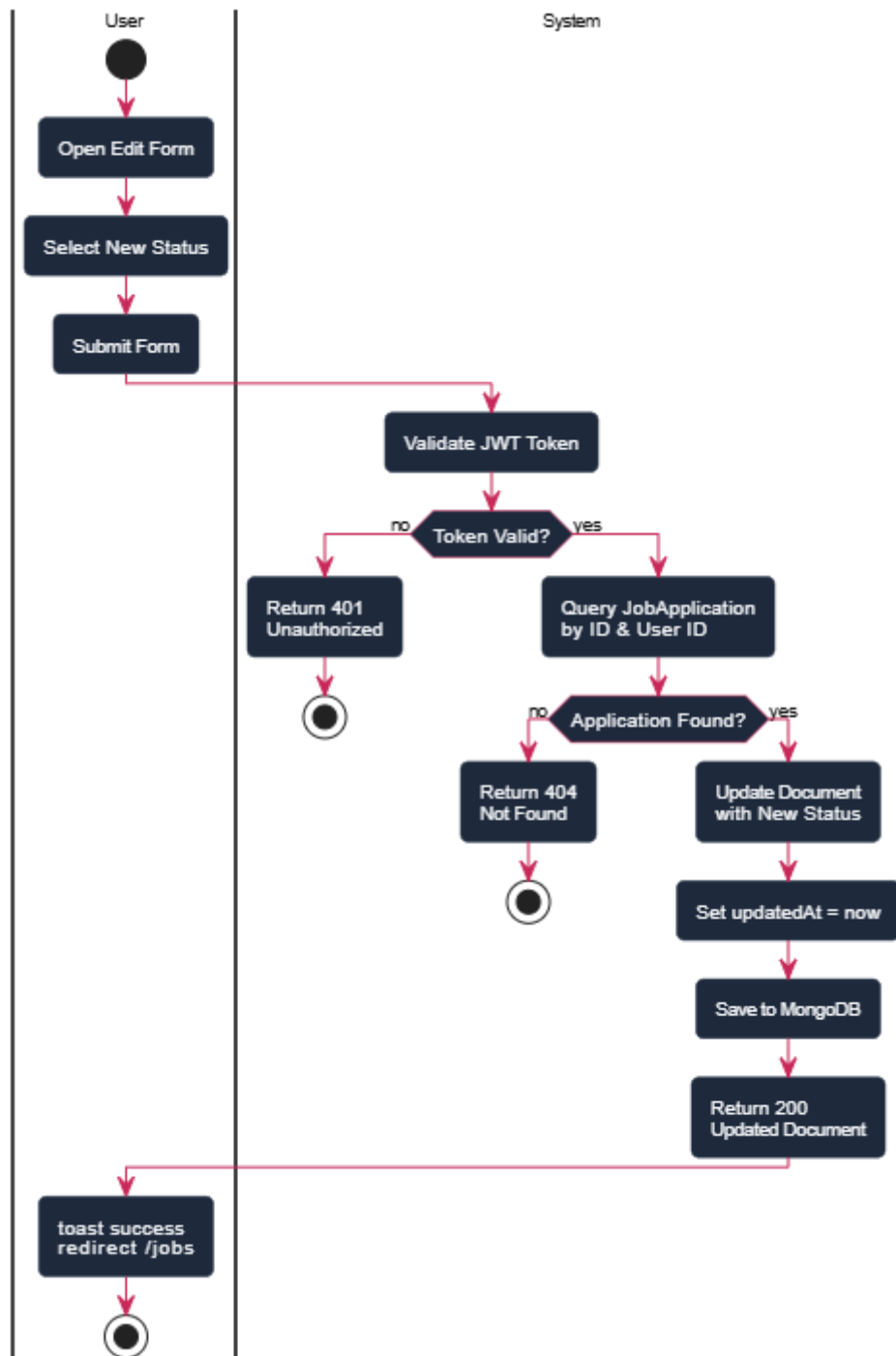


Figure 4.13 - Status Update Activity Diagram

4.6 Security Design

4.6.1 Authentication Architecture

The system uses jsonwebtoken for JWT encoding and decoding with the HS256 algorithm. The secret key is stored as a JWT_SECRET environment variable loaded through dotenv and is never hardcoded in the source files. The token payload contains the user's MongoDB ObjectId as the subject (id field) and the expiry timestamp computed from the JWT_EXPIRE environment variable, which defaults to thirty days.

On every protected request, the protect middleware in backend/middleware/auth.js extracts the Bearer token from the Authorization header, calls jwt.verify() with the secret key to validate the signature and check expiry, decodes the user ID from the payload, and queries MongoDB to retrieve the current user document. This database lookup on every request ensures that if a user account is deactivated or deleted, their token will no longer grant access even if it has not expired.

4.6.2 Data Isolation and Authorization

All job application queries in the getJobs, getJob, updateJob, and deleteJob controllers include the authenticated user's ID as a mandatory filter condition alongside the application-specific identifier. This two-condition query pattern - { _id: req.params.id, user: req.user.id } - ensures that a user cannot read, modify, or delete application records belonging to another user, even if they possess a valid token and know the MongoDB ObjectId of the target document. This pattern provides authorization enforcement at the data layer rather than relying solely on URL-level access control.

4.6.3 Password Security

All passwords are hashed using bcryptjs before storage. The bcrypt algorithm is applied with a salt round value of ten, generating a computationally expensive hash that resists brute-force and dictionary attacks. Plain-text passwords are never stored, logged, or transmitted in any direction after the initial registration request. The password field on the User Mongoose schema is configured with select: false, which excludes it from all query results by default. It is explicitly included only in the login controller using .select('+password'), ensuring it is never accidentally returned in profile or user management endpoints.

4.6.4 Input Validation

Request validation is implemented using the express-validator library on the registration and login endpoints. The validation chain checks that the email field contains a valid email address format and that the password field has a minimum length of six characters. Validation errors are collected and returned as a structured 400 response before the controller logic executes, preventing malformed data from reaching the database layer.

4.7 UI/UX Design

4.7.1 Design System - Dark Theme

The frontend uses a custom dark-themed design system implemented through Tailwind CSS utility classes and CSS custom properties defined in the `index.css` file. The design palette centers on a deep slate background (`--color-bg: #0f172a`) with slightly lighter surface colors for cards and panels (`--color-surface: #1e293b`). Interactive elements use a crimson primary accent color (`--color-primary: #c9295a`) that provides strong visual contrast against the dark background while establishing a distinctive brand identity.

The dark theme was chosen deliberately for the job-tracking use case because users frequently interact with the application during extended work sessions, often in low-light environments. Dark interfaces reduce eye strain during prolonged use, which is a meaningful usability consideration for a productivity application intended for daily engagement.

Typography uses DM Sans for body text and Clash Display for headings, both loaded from external font services. The DM Sans typeface provides excellent legibility at small sizes for dense data tables and application lists. Clash Display provides a distinctive visual weight for page titles and statistics numbers that reinforces the modern, professional aesthetic of the application.

4.7.2 Animation and Interaction Design

Framer Motion is used throughout the application to provide smooth entrance animations on page load, hover effects on interactive cards, and transition animations between navigation states. The core animation pattern is a fade-and-slide entrance - new page content fades in from a slightly lower vertical position using the initial opacity 0 and y 20 state transitioning to opacity 1 and y 0. This pattern gives the interface a sense of physical depth and responsiveness without being distracting.

Interactive cards on the dashboard and application list use the `card-hover` CSS class, which applies a `translateY(-4px)` transform and an elevated box shadow on hover, providing clear visual feedback that an element is interactive before the user clicks.

The loading state for primary actions - form submission, data fetching - is represented by a circular spinning indicator built from CSS border animation, consistent across all pages and immediately recognizable as a loading state.

4.7.3 Responsive Layout Design

The layout is built on a CSS flexbox and grid system provided by Tailwind CSS. The sidebar navigation is hidden on mobile and tablet viewports (below the lg breakpoint) and replaced by a hamburger menu button that toggles a slide-in overlay panel. On desktop viewports, the

sidebar is a fixed-width column (280px) with the main content area occupying the remaining horizontal space through flexbox layout.

Form layouts use responsive grid classes - single column on mobile, two columns on medium viewports, and up to four columns on large viewports for dense sections such as salary and priority fields. This progressive expansion of layout density ensures that all form fields are accessible and usable on small screens while taking advantage of the available space on larger displays.

4.7.4 Key Page Designs

The Dashboard page presents the user's first-name greeting, four primary statistic cards (Total Applications, This Week, Interviewing, Offers), a full status breakdown panel, a Recent Applications list, an Upcoming Reminders panel, and a Quick Actions section. The layout uses an animated gradient background on the page-level container and staggered entrance animations on each card to create a visually engaging first impression on login.

The Applications page presents a search bar with a submit button, a status filter dropdown, and a sort order dropdown in a responsive filter row above the application list. Each application card displays the company initial avatar, position title, company name, status badge, priority badge (for non-medium priorities), location, applied date, and reminder indicator. An action button row provides direct access to the job URL, edit page, and a contextual more-options menu with a delete option.

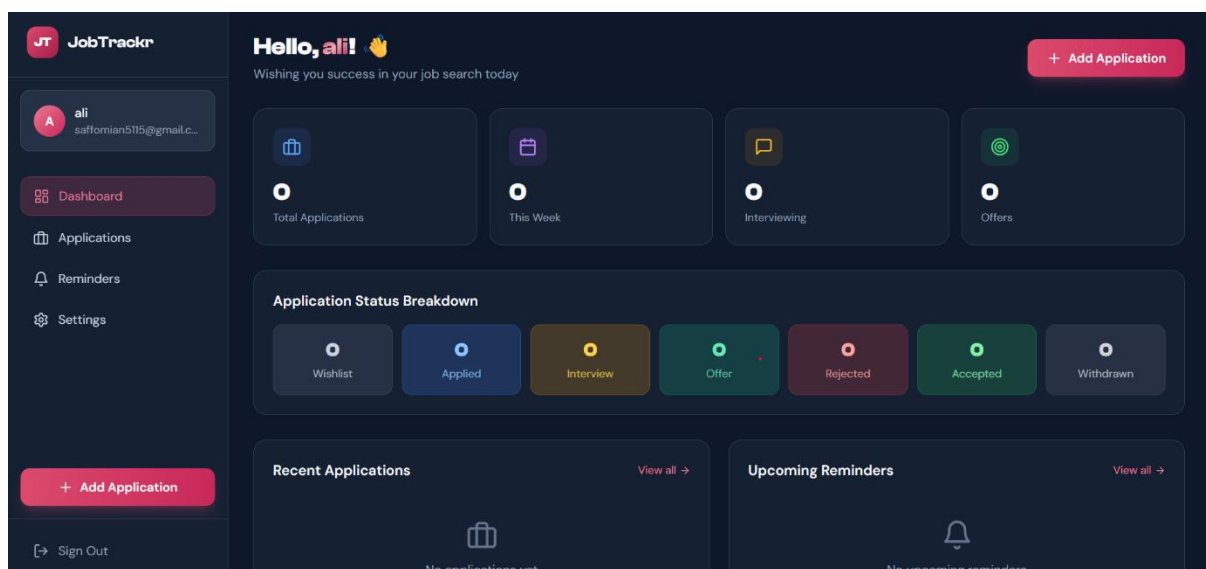


Figure 4.14 - Dashboard Page Design

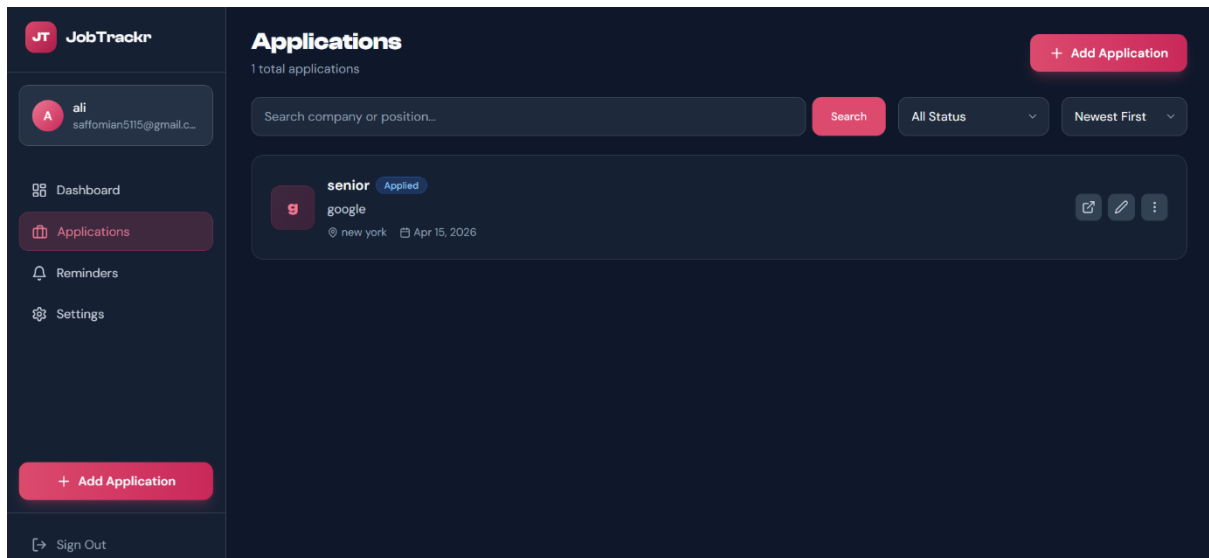


Figure 4.15 - Applications List Page Design

The screenshot displays the 'Add New Application' form in the JobTracker application. The sidebar on the left is identical to the previous page. The main area is titled 'Add New Application' and includes a subtitle 'Track a new job application'. The form is divided into two main sections: 'Basic Information' and 'Salary & Priority'. The 'Basic Information' section contains fields for Company (e.g., Google), Position (e.g., Senior Software Engineer), Location (e.g., New York, NY), Job URL (https://...), Status (Applied), Job Type (Full-time), and Work Mode (Remote). The 'Salary & Priority' section contains fields for Min Salary (50000), Max Salary (80000), Currency (USD - US Dollar), and Priority (Medium).

Figure 4.16 - Add Application Form Design

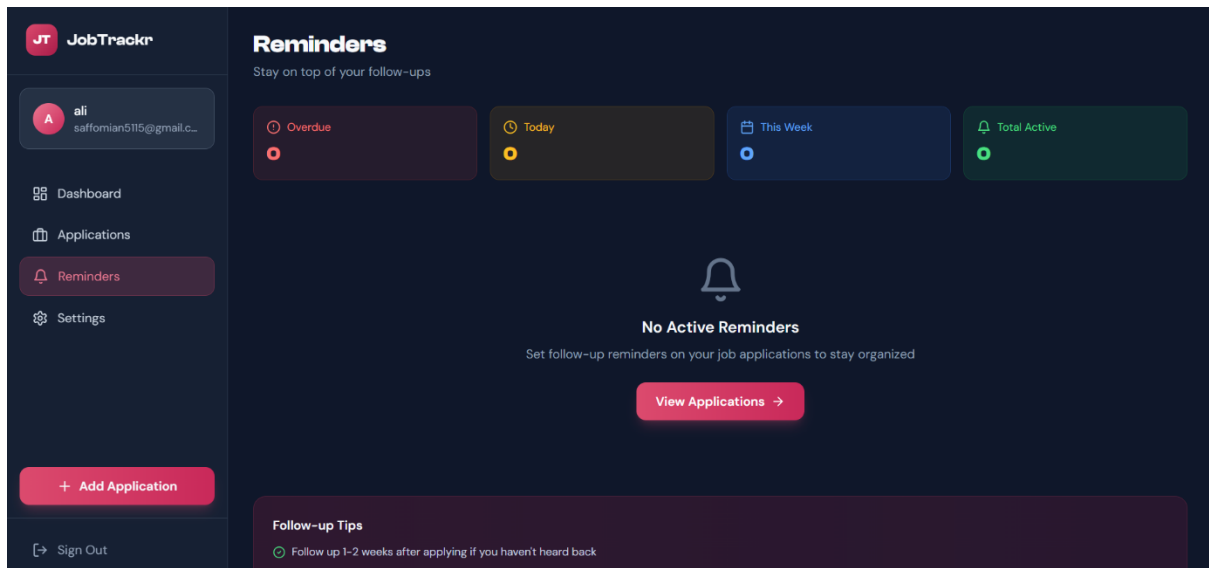


Figure 4.17 - Reminders Page Design

Chapter 5: Implementation & Development

5.1 Development Environment Setup

5.1.1 Backend Setup (Node.js, Express.js, MongoDB)

The backend development environment is configured on a system running Node.js version 18 or higher. A dedicated project directory is created for the backend, and dependencies are installed using npm with the package.json file defining all required packages. The entry point is server.js, which bootstraps the entire backend application including database connection, middleware registration, route mounting, and background service initialization.

The MongoDB database connection is established through the connectDB function defined in backend/config/db.js, which calls mongoose.connect() with the MONGODB_URI environment variable. All environment variables are managed through a .env file in the backend directory loaded by the dotenv package at the very first line of server.js before any other imports are processed. The .env file stores the MongoDB connection string, JWT secret key, JWT expiry duration, server port, and client URL for CORS configuration.

The backend development server is started using nodemon, which watches for file changes and automatically restarts the server during development. The start script in package.json uses node server.js for production deployment, while the dev script uses nodemon server.js for development.

Table 5.1 - Backend Development Tools and Versions

Tool / Package	Version	Purpose
Node.js	18+	Server runtime environment
Express.js	4.x	REST API framework
Mongoose	8.x	MongoDB ODM for schema and queries
jsonwebtoken	9.x	JWT token generation and validation
bcryptjs	2.x	Password hashing and comparison
node-cron	3.x	Scheduled background reminder task
nodemon	3.x	Development server auto-restart on changes
dotenv	16.x	Environment variable management
cors	2.x	Cross-origin request handling middleware

Tool / Package	Version	Purpose
express-validator	7.x	Request input validation chains

5.1.2 Frontend Setup (React.js, Vite, Tailwind CSS)

The frontend is initialized using Vite with the React template. After project creation, Tailwind CSS is installed and configured through `postcss.config.js` and `tailwind.config.js`. The `tailwind.config.js` extends the default theme with a custom primary color palette based on the crimson accent color (`#c9295a` and its shade variants), a dark color palette for the background and surface colors, and custom font family definitions for DM Sans and Clash Display.

Custom global styles are defined in `src/index.css`, which includes Tailwind directives, CSS custom property definitions for the color system, custom component classes for buttons (`.btn-primary`, `.btn-secondary`) and form inputs (`.input-field`), status badge classes (`.status-applied`, `.status-interviewing`, etc.), and global animation keyframes for the gradient background and floating effects.

The Vite development server is configured in `vite.config.js` to run on port 5173 and to proxy all requests prefixed with `/api` to the backend server running on port 5001, eliminating CORS issues during local development by routing API calls through the Vite dev server rather than making direct cross-origin requests.

5.1.3 Project Structure Overview

The backend project follows a clear directory structure that separates concerns at the folder level. The `config` directory contains the database connection module. The `controllers` directory contains one file per resource - `authController.js` and `jobController.js` - each exporting named controller functions. The `middleware` directory contains the authentication middleware and the global error handler. The `models` directory contains the Mongoose schema definitions. The `routes` directory contains the Express router definitions. The `services` directory contains the reminder background service.

The frontend project is organized within the `src` directory. The `context` directory contains the `AuthContext` provider. The `pages` directory contains one component file per application page. The `components` directory contains reusable components shared across pages. The `services` directory contains the `api.js` Axios instance and API function definitions.



Figure 5.1 - Project Directory Structure

5.2 Module Implementation

5.2.1 Server Entry Point and Application Bootstrap

The `server.js` file serves as the central bootstrap module for the entire backend application. Its execution sequence is deliberately ordered to ensure that each component is initialized before the next one depends on it. The `dotenv` configuration is loaded first so that environment variables are available to all subsequently imported modules. The database connection is established through `connectDB()` before the Express application is configured, ensuring that database-dependent operations can begin as soon as the first request arrives.

The Express application is configured with the `express.json()` body parser middleware to enable JSON request body parsing, followed by the `cors` middleware configured with the `CLIENT_URL` environment variable to restrict cross-origin requests to the known frontend origin. The API routes are mounted at their respective prefixes - `/api/auth` for authentication routes and `/api/jobs` for job application routes. The global error handler middleware is registered last in the middleware chain, ensuring it catches errors propagated from all preceding middleware and route handlers.

After the server begins listening on the configured `PORT`, the `startReminderService()` function is called to initialize the background cron job. This ordering ensures the server is fully ready to handle requests before the reminder service begins its first database operations.

5.2.2 Authentication Module Implementation

The authentication module is implemented across three files: `backend/routes/authRoutes.js`, `backend/controllers/authController.js`, and `backend/models/User.js`.

The User Mongoose schema defines the document structure with field-level validation, default values, and pre-save middleware. The pre-save hook checks whether the password field has been modified before hashing, preventing re-hashing on document saves that do not

involve password changes. This check uses the Mongoose `isModified()` method on the document instance. The `matchPassword` instance method uses `bcryptjs.compare()` to verify a plaintext password against the stored hash without exposing the hash itself.

The `generateToken` helper function in `authController.js` calls `jwt.sign()` with a payload object containing the user's `MongoDB _id`, the `JWT_SECRET` environment variable as the signing key, and an options object specifying the `expiresIn` value from the `JWT_EXPIRE` environment variable. This function is called in both the register and login controllers after successful authentication, producing an identical token structure from both entry points.

The `protect` middleware in `backend/middleware/auth.js` implements the token validation logic. It first checks whether the `Authorization` header exists and begins with the string `Bearer`. If not, a 401 response is returned immediately. If the header is present, the token is extracted by splitting on the space character and taking the second element. The `jwt.verify()` call is wrapped in a try-catch block - if verification fails due to an invalid signature or expired token, the catch block returns a 401 response with an appropriate message. If verification succeeds, the decoded payload's `id` field is used to query the `User` collection, and the retrieved user document is attached to `req.user` before calling `next()`.

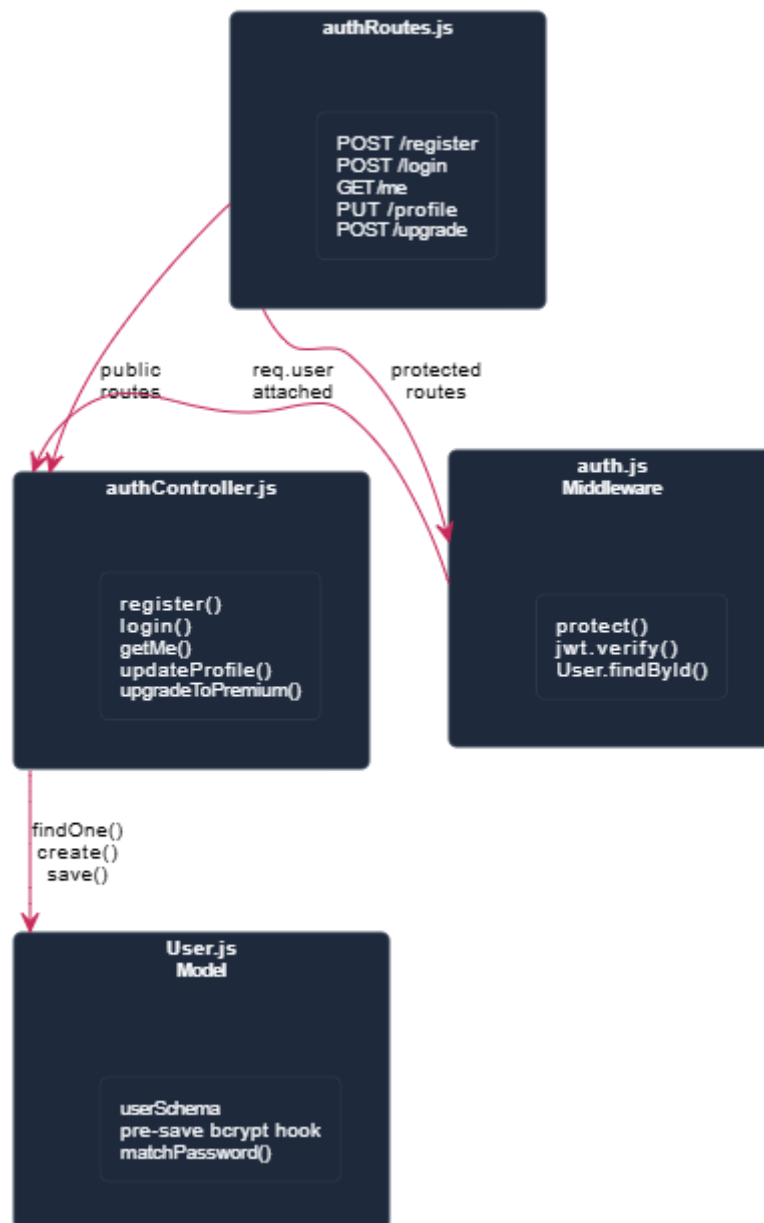


Figure 5.2 - Authentication Module Implementation Diagram

5.2.3 Job Application CRUD Implementation

The job application module is implemented in `backend/controllers/jobController.js` and `backend/models/JobApplication.js`, with routes defined in `backend/routes/jobRoutes.js`.

The `jobRoutes.js` file applies the `protect` middleware to the entire router using `router.use(protect)`, which means authentication is enforced on every route in the file without requiring it to be repeated on each individual route definition. The `stats` and `reminders` routes are defined before the parameterized `/:id` routes to prevent Express from interpreting the literal strings `stats` and `reminders` as dynamic ID values.

The `getJobs` controller constructs a MongoDB query object dynamically based on the query parameters received in the request. The query always includes the `user` field set to `req.user.id`.

to ensure data isolation. If a status parameter is present and is not the string all, it is added to the query. If a search parameter is present, a \$or condition is added using MongoDB regex operators to match against both the company and position fields with the case-insensitive i flag. The sort option is determined through a series of conditional assignments, defaulting to newest-first. The total document count matching the query is obtained with countDocuments() before the paginated results are fetched with find(), sort(), skip(), and limit() chained together.

The createJob controller performs the free tier limit check by calling countDocuments with the user filter before proceeding with document creation. If the count is twenty-five or greater and the user's isPremium flag is false, a 403 response is returned with a message explaining the limit and prompting an upgrade. This check is performed at the controller level rather than the middleware level because it requires knowledge of the specific resource being created.

The updateJob and deleteJob controllers both use a find-then-operate pattern rather than direct findByIdAndUpdate or findByIdAndDelete. They first query for the document using both the _id and user fields. If the document is not found - either because it does not exist or because it belongs to a different user - a 404 response is returned. This pattern provides explicit authorization checking and clear error messaging separate from the database operation itself.

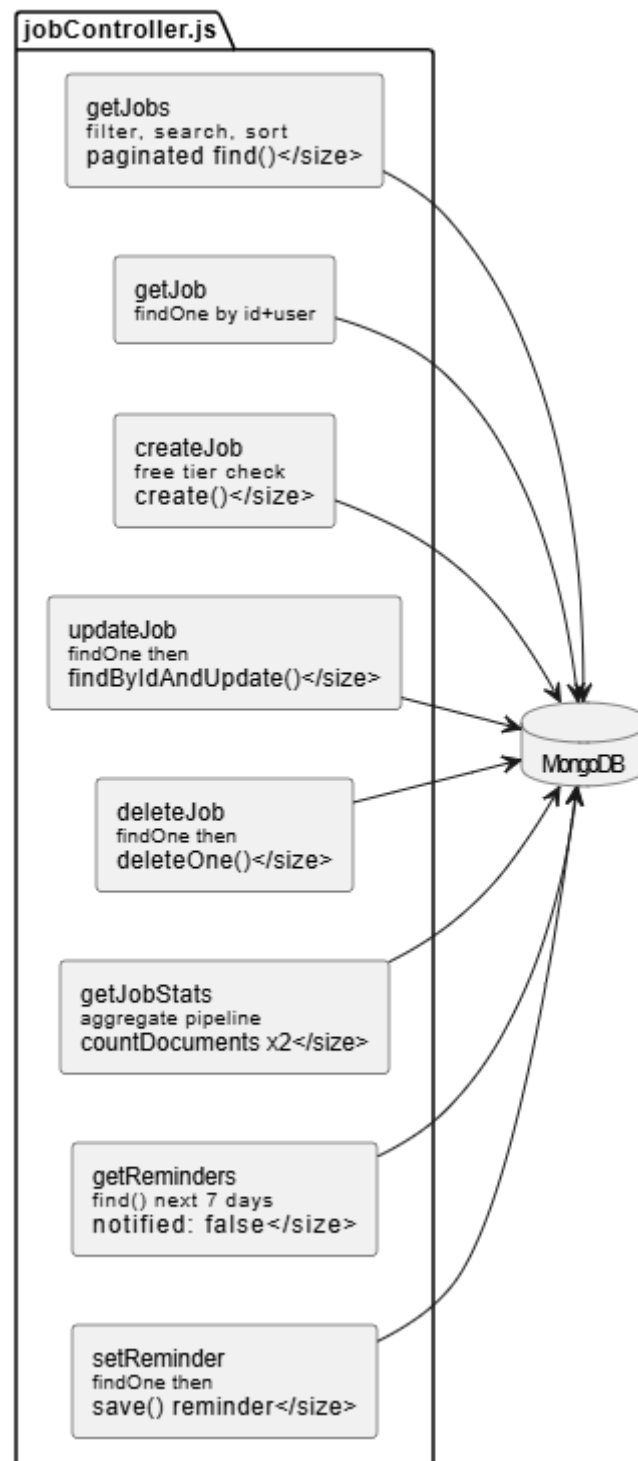


Figure 5.3 - Job Controller Function Map

5.2.4 Statistics Aggregation Implementation

The `getJobStats` controller uses MongoDB's aggregation pipeline to compute all required statistics in a minimal number of database operations. The pipeline consists of a `$match` stage to filter documents to the authenticated user, followed by a `$group` stage that counts documents by status.

In addition to the aggregation pipeline for status counts, two separate `countDocuments` queries compute the weekly and monthly application counts using date range comparisons. The week calculation creates a `Date` object for the current time, subtracts seven days by calling `setDate()` with the current date minus seven, and passes this as the `$gte` value in a `createdAt` range filter. The monthly calculation follows the same pattern with thirty days. These three operations are executed as independent asynchronous calls and awaited with `Promise.all()` to minimize total execution time.

The aggregation pipeline result is transformed from its raw array format - where each element is an object with `_id` as the status string and `count` as the integer - into a flat `statusStats` object with named keys for each of the seven status values, defaulting to zero for any status with no matching documents. This transformation is performed in a `forEach` loop before the response is constructed, ensuring the frontend receives a consistent object structure regardless of which statuses have applications.

5.2.5 Reminder Service Implementation

The reminder service in `backend/services/reminderService.js` exports a single function, `startReminderService()`, which registers the cron job and returns immediately. All reminder processing logic is encapsulated within the cron callback function, which is an asynchronous arrow function that runs independently of the HTTP request-response cycle.

The query uses a `MongoDB find()` call with three conditions in the filter object. The `followUpReminder.enabled` condition uses the dot notation to access the nested subdocument field, which `Mongoose` translates into the appropriate `MongoDB` query syntax. The `followUpReminder.date` condition uses the `$lte` operator with a freshly constructed new `Date()` object at the time of each execution, ensuring the comparison is always made against the current time rather than the time the service was started. The `followUpReminder.notified` condition explicitly filters for `false` to prevent reprocessing reminders that have already been marked.

The `populate('user', 'email name')` call is chained to the query to include the user's email and name in the result set for logging purposes. Each matching document is saved individually with the `notified` field set to `true`, rather than using a bulk update, to ensure that the `Mongoose` pre-save middleware runs for each document if any such hooks are defined in the future.

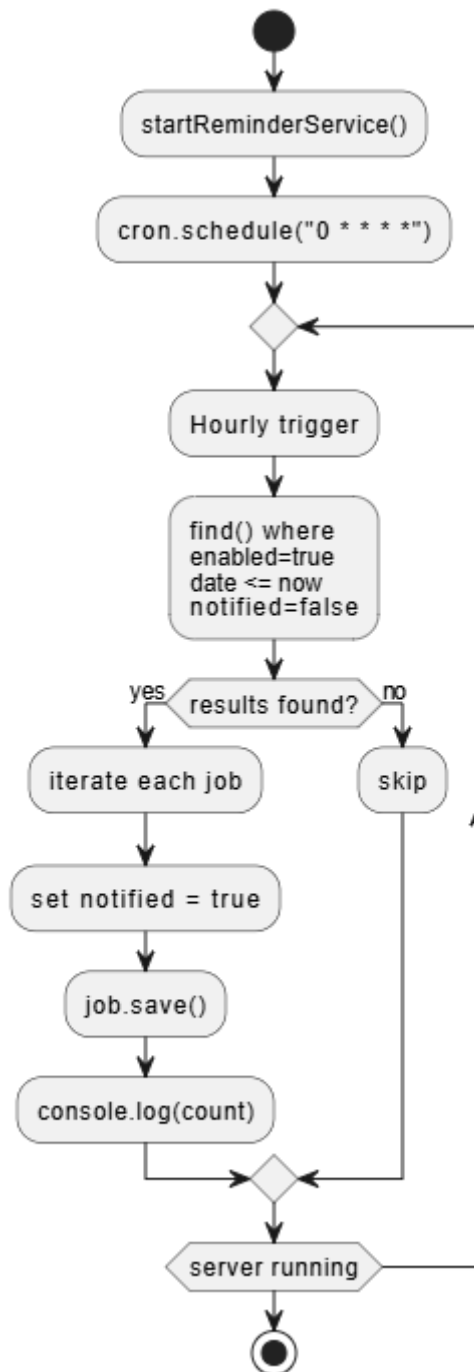


Figure 5.4 - Reminder Service Flow Code

5.2.6 Error Handling Implementation

The global error handler middleware in `backend/middleware/errorHandler.js` provides a centralized error processing mechanism that intercepts all errors passed through `next(error)` calls in the route chain. The handler inspects the error object for specific Mongoose error types and maps them to appropriate HTTP status codes and user-friendly messages.

A Mongoose `CastError` - which occurs when an invalid `ObjectId` format is provided as a route parameter - is mapped to a 404 Not Found response, preventing the raw Mongoose error

message from leaking to the client. A MongoDB duplicate key error - identified by the error code 11000 - is mapped to a 400 Bad Request response with a message about duplicate field values. A Mongoose ValidationError - which occurs when a document fails schema-level validation - is mapped to a 400 response with the validation message extracted from the error object.

All other errors default to a 500 Internal Server Error response. The error message is always included in the response body for development visibility, though in a production deployment this would be replaced with a generic message to avoid information disclosure.

5.3 Key Code Explanations

5.3.1 JWT Token Generation and Validation

The token generation function produces a signed JWT using the user's MongoDB ObjectId as the sole payload claim. The choice to include only the ID - rather than the full user object - is deliberate. It keeps the token compact, avoids stale data in the token payload if the user's profile changes, and requires a fresh database lookup on each request to ensure the most current user data is available to the controller.

The thirty-day expiry provides a balance between security and user convenience. A shorter expiry would require more frequent re-authentication, creating friction for a daily-use productivity application. A longer expiry would increase the risk window if a token were compromised. The thirty-day window reflects a judgment that the application's threat model does not warrant the complexity of a refresh token architecture for this use case, while still providing a reasonable expiry safety boundary.

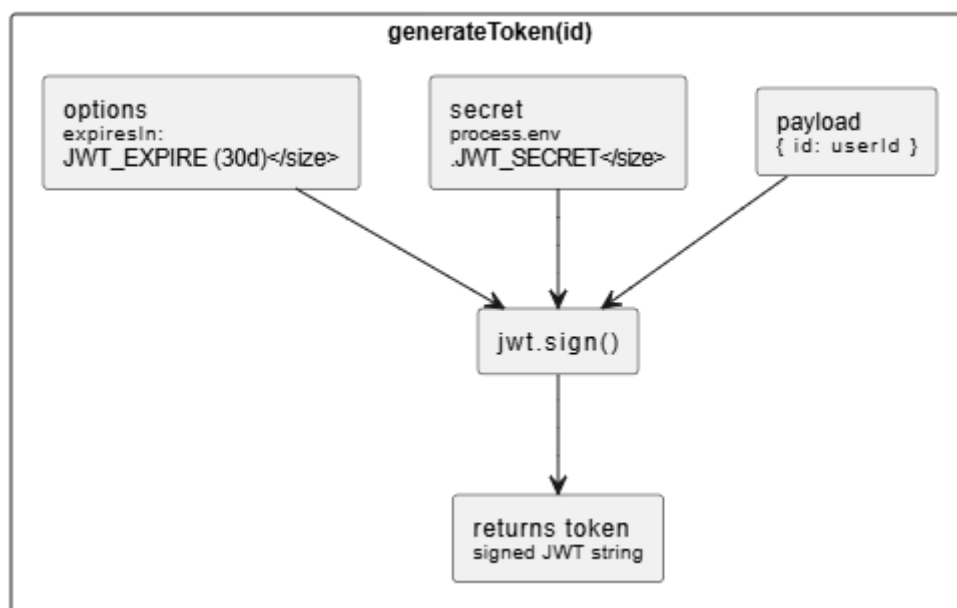


Figure 5.5 - JWT Token Generation Snippet

5.3.2 Free Tier Limit Enforcement

The free tier enforcement logic in the createJob controller represents a business rule implementation at the API layer. The count check using countDocuments is preferred over loading all documents and checking array length because it executes a single count query at the database level without transferring any document data over the network, making it efficient regardless of how many applications the user already has.

The condition checks both the count threshold and the isPremium flag together in a single if statement, using the logical AND operator. This means a premium user with more than twenty-five applications will not be blocked - the premium flag takes precedence. A free user with exactly twenty-five applications will be blocked on their twenty-sixth attempt, while a free user with fewer than twenty-five applications can continue creating records normally.

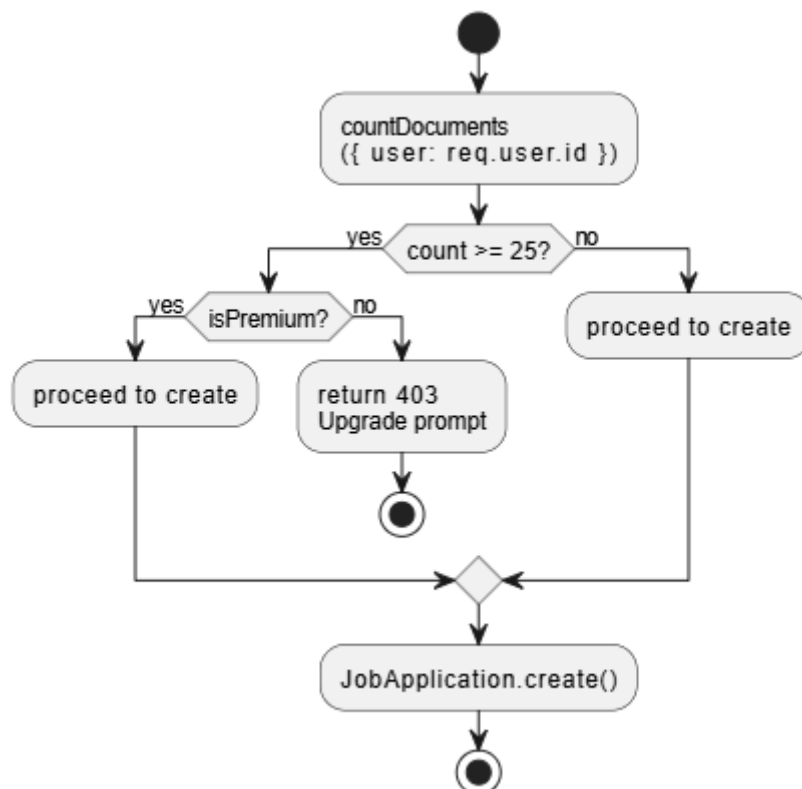


Figure 5.6 - Free Tier Limit Check Snippet

5.3.3 Axios Instance and Interceptors

The api.js service file creates a single shared Axios instance with the base URL set to /api, which resolves against the current page origin and is proxied to the backend by the Vite dev server configuration. This single-instance pattern ensures that all API calls in the application share the same configuration and interceptors without requiring them to be set up in each component.

The request interceptor reads the JWT token from localStorage and, if present, adds it to the Authorization header of every outgoing request using the Bearer scheme. This automatic token injection means no component in the application needs to manually manage the token - it is applied transparently to all requests.

The response interceptor handles 401 Unauthorized responses by removing the token from localStorage and redirecting the browser to the login page. This provides automatic session expiry handling - if the token expires while the user is actively using the application, their next API call will fail with a 401, triggering the interceptor, clearing the stale token, and returning them to the login page with a clean state.

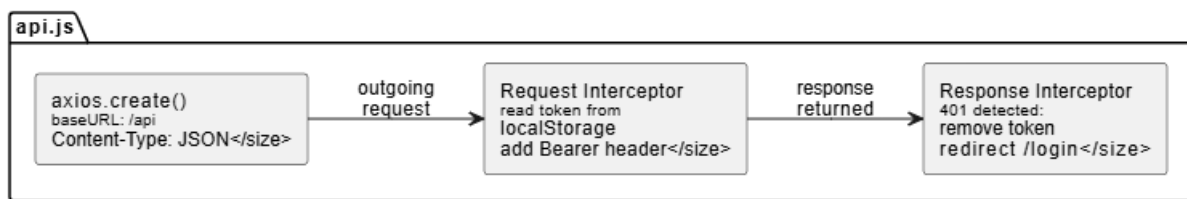


Figure 5.7 - Axios Instance Configuration Snippet

5.3.4 AuthContext Implementation

The AuthContext provides the authentication state and action functions to the entire React component tree without requiring prop drilling. The context value includes the user object, the token string, a loading boolean, the login function, the register function, the logout function, the upgradeToPremium function, and a derived isAuthenticated boolean computed as a double negation of the user object.

The useEffect hook in the AuthProvider component watches the token state value. When the token is present - either from initial localStorage retrieval on mount or from a new login - the loadUser function is called to fetch the current user's profile from the backend. This ensures the user object in context always reflects the server's current state rather than stale data from a previous session. If the loadUser request fails with a 401 - indicating an expired or invalid token - the stored token is removed from localStorage and the token state is set to null, triggering a clean unauthenticated state.

5.3.5 Searchable Select Component

The SearchableSelect component provides a fully custom dropdown select element that combines a text search input with a scrollable option list, replacing the browser's native select element for the currency selection use case. The component maintains three pieces of local state: isOpen to control dropdown visibility, search to hold the current filter text, and uses props for the selected value and change handler.

The filteredOptions computation filters the options array on every render where the search state has changed, comparing both the option label and option value against the lowercased

search string. This client-side filtering is appropriate for the thirty-currency dataset as it is small enough to filter in memory without performance concerns.

A `useEffect` hook registers a `mousedown` event listener on the document when the component mounts, detecting clicks outside the component's `containerRef` and closing the dropdown automatically. This click-outside behavior is essential for usability - users expect dropdowns to close when they click elsewhere on the page. The event listener is removed in the cleanup function returned by the `useEffect` to prevent memory leaks when the component unmounts.

5.4 Frontend Implementation

5.4.1 Application Routing and Navigation Structure

The React application's routing structure is defined in `App.jsx` using React Router DOM's declarative route configuration. The routing architecture implements two route guard patterns. The `PrivateRoute` component wraps all authenticated pages - it checks the `isAuthenticated` value from `AuthContext` and, if false, redirects the user to the login page. The `PublicRoute` component wraps the authentication pages - it checks the `isAuthenticated` value and, if true, redirects the user to the dashboard, preventing authenticated users from accessing the login or registration pages unnecessarily.

The authenticated pages are nested under a `Layout` route that renders the `Layout` component as the parent, which provides the persistent sidebar and main content container. The `Outlet` component within the `Layout` renders the matched child route's component in the main content area. This structure means the sidebar navigation is rendered once and persists across all authenticated page navigations without remounting.

5.4.2 Dashboard Page Implementation

The Dashboard page fetches data from three API endpoints in parallel using `Promise.all()` within a single `useEffect` call. The three requests - statistics, recent applications (limited to five, sorted by newest), and upcoming reminders - are initiated simultaneously, reducing the total loading time to the duration of the slowest individual request rather than the sum of all three.

The page renders a greeting using the first word of the authenticated user's name, extracted with `split(' ')[0]`. Four statistic cards are rendered from the `statCards` array, each referencing the relevant field from the `stats` state object using optional chaining to safely handle the loading state before data arrives. The status breakdown section iterates over the `statusConfig` object entries to render a count card for each status. The recent applications section maps over the `recentJobs` array to render navigation links to each application's edit page.

5.4.3 Applications List Page Implementation

The Jobs page implements the search, filter, and sort interface with a controlled form approach. The search input value is stored in local state and only submitted to the API when the user explicitly submits the search form, preventing unnecessary API calls on every keystroke. The status and sort dropdowns trigger immediate re-fetches through `useEffect` dependencies, providing instant filtering response without requiring form submission.

Pagination state is managed as a local state object containing `page`, `totalPages`, and `total`. The `page` value is reset to one whenever the status filter changes, preventing the user from being stranded on a page number that no longer exists after filtering reduces the total result count. The pagination controls are conditionally rendered only when `totalPages` is greater than one, keeping the interface clean for users with small application sets.

The `activeMenu` state tracks which application's contextual more-options menu is currently open, storing the application ID as the value. This single state value approach ensures only one menu can be open at a time - opening a new menu automatically closes the previous one by replacing the stored ID value.

5.4.4 Add and Edit Application Pages Implementation

The `AddJob` and `EditJob` pages share nearly identical form structure, differing primarily in their initial data source - `AddJob` initializes all form fields with empty strings and defaults, while `EditJob` fetches the existing application data in a `useEffect` on component mount and populates the form state with the retrieved values.

The form state is a single `useState` object containing all field values, updated through a single `handleChange` function that uses the event target's name and value (or checked for checkbox inputs) to update the corresponding key in the state object using the spread operator. This pattern avoids creating separate state variables and setter functions for each field, keeping the component code concise.

The salary section uses the `SearchableSelect` component for the currency field, which requires special handling in the `handleChange` function because the `SearchableSelect` dispatches a synthetic event object constructed as `{ target: { name, value } }` in its `handleSelect` function, matching the structure of native input change events and making it compatible with the unified `handleChange` function without modifications.

The reminder section uses conditional rendering - the date input field is only shown when the `reminderEnabled` checkbox is checked. This is implemented with a simple logical AND expression that renders the date input only when `formData.reminderEnabled` is true.

5.4.5 Reminders Page Implementation

The Reminders page fetches data from two sources simultaneously - the dedicated reminders endpoint (which returns upcoming reminders within seven days) and the general applications endpoint with a high limit (which is filtered client-side to applications with reminders enabled). The second fetch provides the full set of active reminders for the statistics cards and the complete reminders list, while the first is used as a quick reference for the dashboard widget.

The `getTimeLabel` helper function uses date-fns comparison functions - `isToday`, `isTomorrow`, `isPast`, and `differenceInDays` - to produce human-readable time labels (Today, Tomorrow, Overdue, In X days, or a formatted date for reminders more than a week away). The `getTimeLabelClass` helper returns the appropriate color class string based on the same date comparison logic, providing visual urgency cues - red for overdue, amber for today, orange for tomorrow, and blue for future reminders.

The `markAsComplete` function calls the `setReminder` API endpoint with `enabled` set to `false`, which disables the reminder on the application document. After the API call succeeds, `fetchData()` is called to reload the reminders list, reflecting the updated state.

5.5 API Integration Between Frontend and Backend

All frontend-to-backend communication is centralized through the `api.js` Axios instance and the `jobApi` object exported from the same file. The `jobApi` object provides named methods - `getAll`, `getById`, `create`, `update`, `delete`, `getStats`, `getReminders`, and `setReminder` - each wrapping the corresponding Axios call with the appropriate HTTP method, URL, and parameter structure. This centralization means that if an API endpoint URL changes, the update is made in one place rather than across multiple component files.

The Vite proxy configuration in `vite.config.js` routes all requests from the frontend's `/api` prefix to the backend server at `http://localhost:5001` with `changeOrigin` set to `true`. This proxy eliminates CORS issues during development by making all requests appear to originate from the same origin as the frontend, while the backend's CORS middleware handles production cross-origin requests from the deployed frontend URL.

The `protect` middleware on the backend and the request interceptor on the frontend form a complementary pair - the interceptor ensures every outgoing request carries the token, and the middleware ensures every incoming request to a protected endpoint is validated before processing begins. This symmetry means developers working on either side of the API boundary can rely on authentication being consistently handled without per-request manual management.

5.6 Database Integration

All database interactions in the backend are performed through Mongoose model methods. Direct MongoDB driver calls are not used anywhere in the codebase, ensuring that all queries benefit from Mongoose's schema validation, middleware hooks, and ODM abstractions. The database connection is established once in `connectDB()` and the connection pool is managed by Mongoose, reusing existing connections for all subsequent operations without creating new connections per request.

The `JobApplication` model's pre-save hook updates the `updatedAt` timestamp on every save operation, providing an accurate last-modified timestamp on every document without requiring explicit timestamp management in controller code. This hook is defined directly on the schema and runs transparently for all `save()` calls, including those from the reminder service.

Mongoose's `lean()` option is not used in the current implementation, meaning all query results are returned as full Mongoose document instances with all associated methods and virtual properties. For the current scale of the application, the overhead of full document instantiation is acceptable. For future optimization at larger scale, applying `lean()` to read-heavy list queries could meaningfully reduce memory allocation.

5.7 Challenges Faced and Solutions

Table 5.2 - Challenges Faced and Solutions

Challenge	Description	Solution Adopted
Free Tier Race Condition	Two simultaneous POST requests could both pass count check before either creates a document	Acknowledged as low-probability; production fix would use MongoDB transaction or atomic <code>findAndModify</code>
Vite Proxy Port Mismatch	Frontend API calls failed when backend port changed between sessions	Standardized backend port in <code>.env</code> and updated <code>vite.config.js</code> proxy target to reference same value
SearchableSelect Closing Early	Click-outside handler closed dropdown when user clicked internal search input	Added <code>containerRef</code> check to verify click target is contained within component before closing
Status Filter Page Reset	Changing status filter with pagination active left user on non-existent page	Added explicit page reset to 1 in status <code>onChange</code> handler before triggering <code>refetch</code>
Cron Before DB Connection	Cron job occasionally fired before MongoDB connection was established	Moved <code>startReminderService()</code> call inside server listen callback ensuring DB is ready first

Challenge	Description	Solution Adopted
Stale JWT After User Deletion	Manually deleted user document left valid JWT in browser	protect middleware's User.findById() naturally handles this - 401 returned if user not found

Row 1: Free Tier Limit Race Condition / Two simultaneous POST requests from the same free user could both pass the count check before either creates a document, potentially allowing 26 applications / Acknowledged as a low-probability edge case for the current user scale; a production solution would use a MongoDB transaction or an atomic findAndModify with a conditional check

Row 2: Vite Proxy and Backend Port Mismatch / Frontend API calls failed in development because the backend port changed between development sessions / Standardized backend port in .env file and updated vite.config.js proxy target to reference the same value; added documentation in README

Row 3: SearchableSelect Closing on Internal Click / The click-outside handler was closing the dropdown when the user clicked the internal search input because the mousedown event propagated up to the document / Added a containerRef that wraps the entire component and checks whether the click target is contained within the ref before closing

Row 4: Status Filter Page Reset / When changing the status filter with pagination active, users were left on a page number that did not exist in the filtered result set / Added explicit page reset to 1 in the status onChange handler before triggering the refetch

Row 5: Reminder Cron Running Before Database Connection / In early development, the cron job occasionally fired before the MongoDB connection was established, causing the first execution to fail with a connection error / Moved startReminderService() call to inside the server listen callback, ensuring it only starts after the server is confirmed ready and the database connection has been established

Row 6: JWT Token Persisting After Manual Database Deletion / During testing, manually deleting a user from MongoDB left a valid JWT in the browser that passed signature validation but had no corresponding user document / The protect middleware's explicit User.findById() call naturally handles this - if the user document is not found, a 401 is returned even for a cryptographically valid token]

Chapter 6: Testing & Validation

6.1 Testing Strategy

6.1.1 Testing Approach

The testing strategy for the Job Tracker follows a bottom-up approach, beginning with individual endpoint and function validation and progressing through integration testing to complete end-to-end workflow verification. Each development phase concluded with a dedicated testing phase before the next module was begun, ensuring that defects were identified and resolved at the point of introduction rather than accumulating into a larger debugging effort at the end of development.

The testing approach prioritizes functional correctness and security boundary enforcement as the two highest-priority quality dimensions. Functional correctness ensures that every feature behaves exactly as specified in the requirements documented in Chapter 3. Security boundary enforcement ensures that authentication and data isolation mechanisms prevent unauthorized access to protected resources and cross-user data leakage under all tested conditions.

All API endpoint testing was conducted using a combination of direct HTTP requests through the browser-based interface and structured request sequences executed through an HTTP testing tool. Frontend behavior was validated through direct browser interaction across multiple viewport sizes and browsers.

6.1.2 Types of Testing Conducted

Unit Testing was applied to individual controller functions, middleware components, and utility functions in isolation. The JWT token generation and validation logic, password hashing and comparison behavior, free tier limit enforcement condition, and statistics aggregation pipeline transformation were each verified independently before being integrated into the full request chain.

Integration Testing verified that the components of each module interact correctly when combined. The authentication middleware was tested in combination with each protected controller to confirm that valid tokens allow access and invalid tokens are rejected consistently. The job creation flow was tested end-to-end from the form submission through API call, controller execution, database write, and response handling to confirm that all components work together correctly.

Functional Testing validated each feature against its requirements through structured test cases. Test cases were designed to cover the happy path, common alternative paths, and boundary conditions for each functional area.

Security Testing targeted the authentication system, authorization enforcement, and input handling to verify resistance to common attack patterns including unauthorized access attempts, cross-user data access attempts, and malformed input submissions.

Performance Testing measured API response times for standard operations to verify compliance with the sub-500ms non-functional requirement defined in Chapter 3.

Usability and UI Testing verified that the interface renders correctly across browsers and screen sizes and that navigation flows, form interactions, and error states behave as expected from a user perspective.

6.2 Test Environment

All testing was performed on a local development machine. The Express.js backend was run using nodemon in development mode on port 5001. The React.js frontend was served by the Vite development server on port 5173 with the API proxy configured to forward requests to the backend. A dedicated test MongoDB database was used for all testing activities, separate from any development data, ensuring that test operations did not affect real application data and that the database could be reset to a known state between test runs.

Browser testing was conducted in Google Chrome as the primary browser, with compatibility verification in Mozilla Firefox and Microsoft Edge. Mobile responsiveness was tested using Chrome DevTools device simulation for common device sizes including iPhone 12, Samsung Galaxy S21, and iPad Pro.

6.3 Unit Testing

6.3.1 Authentication Logic Tests

The authentication module was tested for the following unit-level behaviors. Password hashing was verified by confirming that the bcryptjs hash function produces a different output on each call for the same input due to random salt generation, and that the matchPassword instance method returns true for a correct plaintext-hash pair and false for an incorrect one. JWT token generation was tested by decoding produced tokens using jwt.decode() and verifying that the id field in the payload matches the user ID passed to the generation function and that the exp field reflects the expected expiry duration. Token expiry behavior was tested by examining the error type thrown by jwt.verify() for tokens with a past expiry timestamp, confirming that the error is caught by the protect middleware's try-catch block and results in a 401 response.

6.3.2 Statistics Aggregation Tests

The getJobStats controller's aggregation logic was tested by creating a known set of application documents with specific status values and verifying that the returned statusStats object contains the correct count for each status. Edge cases tested included a user with no

applications (expecting all status counts to be zero), a user with applications in only one status (expecting all other statuses to return zero through the default initialization), and a user with applications in all seven statuses (expecting correct counts for each).

6.3.3 Free Tier Limit Tests

The free tier enforcement was tested by creating exactly twenty-five applications as a free user and verifying that the twenty-sixth creation attempt returns a 403 response. After upgrading the same user to premium through the upgrade endpoint, the creation of a twenty-sixth application was verified to succeed, confirming that the `isPremium` flag correctly bypasses the limit check.

6.4 Integration Testing

Integration tests verified the interaction between system components across module boundaries. The following integration flows were tested and confirmed to function correctly.

Authentication to Protected Endpoint: Login with valid credentials, extract the token from the response, use the token on the `GET /api/jobs` endpoint, confirm a 200 response with the user's application data. Repeat without the token and confirm a 401 response. Repeat with a manually corrupted token string and confirm a 401 response.

Registration to Immediate Login: Register a new account and immediately use the returned token to access the `GET /api/auth/me` endpoint without performing a separate login, confirming that the token returned by registration is immediately valid for authenticated requests.

Application Creation to List Retrieval: Create a new application with specific field values, then retrieve the application list and verify the newly created application appears with all fields correctly stored and returned.

Application Update to Statistics Refresh: Change an application's status from applied to interviewing, then call the statistics endpoint and verify that the `byStatus` counts reflect the updated status distribution.

Reminder Set to Reminders List: Create an application with a follow-up reminder date set within the next seven days, then call the `GET /api/jobs/reminders` endpoint and verify the application appears in the reminders response.

Profile Update to Token Validation: Update the authenticated user's name through the `PUT /api/auth/profile` endpoint, then call `GET /api/auth/me` and verify the updated name is returned, confirming that profile updates are persisted and reflected in subsequent profile retrievals.

6.5 Functional Test Cases and Results

6.5.1 User Authentication Test Cases

Table 6.1 - Authentication Functional Test Cases

TC ID	Test Description	Input	Expected Result	Actual Result	Status
TC-AUTH-01	Register with valid credentials	name, email, password	201 with token and user	201 returned	Pass
TC-AUTH-02	Register with duplicate email	Existing email	400 User already exists	400 returned	Pass
TC-AUTH-03	Register with missing name	Empty name field	400 validation error	400 returned	Pass
TC-AUTH-04	Login with correct credentials	Valid email/password	200 with token	200 returned	Pass
TC-AUTH-05	Login with wrong password	Valid email, wrong pass	401 Invalid credentials	401 returned	Pass
TC-AUTH-06	Login with unknown email	Unregistered email	401 Invalid credentials	401 returned	Pass
TC-AUTH-07	Login without password	Email only	400 validation error	400 returned	Pass
TC-AUTH-08	Access endpoint with valid token	Valid Bearer token	200 with data	200 returned	Pass
TC-AUTH-09	Access endpoint without token	No Authorization header	401 no token	401 returned	Pass
TC-AUTH-10	Access with expired token	Past-expiry token	401 token failed	401 returned	Pass
TC-AUTH-11	Update profile name and email	Valid name and email	200 updated user	200 returned	Pass
TC-AUTH-12	Upgrade free user to premium	Free user token	200 isPremium true	200 returned	Pass
TC-AUTH-13	Re-upgrade premium user	Premium user token	400 already premium	400 returned	Pass

6.5.2 Job Application CRUD Test Cases

Table 6.2 - Job Application CRUD Test Cases

TC ID	Test Description	Input	Expected Result	Actual Result	Status
TC-JOB-01	Create with required fields only	company and position	201 with defaults	201 returned	Pass
TC-JOB-02	Create with all optional fields	Complete payload	201 all fields stored	201 returned	Pass
TC-JOB-03	Create without company	Missing company	400 validation error	400 returned	Pass
TC-JOB-04	Create without position	Missing position	400 validation error	400 returned	Pass
TC-JOB-05	Get application list	Valid token, no filters	200 paginated results	200 returned	Pass
TC-JOB-06	Filter by status	status=interviewing	200 filtered results	200 returned	Pass
TC-JOB-07	Search by company name	search=Google	200 matching results	200 returned	Pass
TC-JOB-08	Sort by company	sort=company	200 alphabetical order	200 returned	Pass
TC-JOB-09	Get single by valid ID	Valid ObjectId	200 single document	200 returned	Pass
TC-JOB-10	Get single by invalid ID	Malformed ObjectId	404 Not Found	404 returned	Pass
TC-JOB-11	Update status field	status: offer	200 updated document	200 returned	Pass
TC-JOB-12	Update with salary info	salaryMin, salaryMax, currency	200 salary subdocument updated	200 returned	Pass
TC-JOB-13	Delete by valid ID	Valid ObjectId	200 success message	200 returned	Pass
TC-JOB-14	Delete non-existent ID	Valid format, not in DB	404 Not Found	404 returned	Pass

6.5.3 Free Tier Limit Test Cases

Table 6.3 - Free Tier Limit Test Cases

TC ID	Test Description	Setup	Expected Result	Status
TC-FREE-01	Free user at limit blocked	Free user with 25 applications	403 Forbidden with upgrade message	Pass
TC-FREE-02	Free user under limit succeeds	Free user with 24 applications	201 Created successfully	Pass
TC-FREE-03	Premium user above limit succeeds	Premium user with 30 applications	201 Created successfully	Pass
TC-FREE-04	Upgrade removes restriction	User upgraded with 25 applications	201 after upgrade	Pass

6.5.4 Statistics and Reminders Test Cases

Table 6.4 - Statistics and Reminders Test Cases

TC ID	Test Description	Setup	Expected Result	Status
TC-STAT-01	Correct total application count	User with 10 applications	total equals 10	Pass
TC-STAT-02	Correct per-status counts	Applications across multiple statuses	byStatus counts match	Pass
TC-STAT-03	Zero for empty status	No wishlist applications	byStatus.wishlist equals 0	Pass
TC-STAT-04	Correct weekly count	Mix of recent and old applications	thisWeek matches only recent	Pass
TC-STAT-05	Upcoming reminders returned	Reminder date 3 days from now	Application in reminders response	Pass
TC-STAT-06	Reminders beyond 7 days excluded	Reminder date 10 days from now	Not in reminders response	Pass
TC-STAT-07	Notified reminders excluded	Reminder with notified true	Not in reminders response	Pass

TC ID	Test Description	Setup	Expected Result	Status
TC-STAT-08	Reminders scoped to user	Reminder for different user	Other user reminder absent	Pass

6.5.6 Frontend Functional Test Cases

Table 6.5 - Frontend Functional Test Cases

TC ID	Test Description	Steps	Expected Result	Status
TC-FE-01	Login redirects to dashboard	Enter valid credentials, click Sign In	Dashboard loads with greeting	Pass
TC-FE-02	Registration creates account	Enter name, email, password, submit	Dashboard loads after registration	Pass
TC-FE-03	Add application saves all fields	Fill complete form, submit	Application in list with all fields	Pass
TC-FE-04	Status filter updates list	Select Interviewing from dropdown	Only interviewing apps displayed	Pass
TC-FE-05	Search returns matching results	Type Google, submit search	Google apps displayed	Pass
TC-FE-06	Delete removes from list	Click delete, confirm dialog	App removed, success toast shown	Pass
TC-FE-07	Reminders show time labels	Navigate to Reminders page	Overdue red, today amber, future blue	Pass
TC-FE-08	Sidebar highlights active page	Click each navigation item	Active item highlighted	Pass
TC-FE-09	Mobile sidebar opens and closes	Click hamburger button	Sidebar slides in, outside click closes	Pass
TC-FE-10	Unauthenticated access redirects	Navigate to dashboard without token	Redirect to login page	Pass

6.6 Security Testing

Security testing focused on verifying that the authentication and authorization boundaries function correctly under adversarial conditions.

Table 6.6 - Security Test Cases

TC ID	Security Test	Method	Expected Result	Status
TC-SEC-01	No token access	GET /api/jobs without token	401 Unauthorized	Pass
TC-SEC-02	Malformed token access	Invalid Bearer string	401 token failed	Pass
TC-SEC-03	Expired token access	Past-expiry token	401 token failed	Pass
TC-SEC-04	Access another user's application	User A token, User B app ID	404 Not Found	Pass
TC-SEC-05	Update another user's application	User A token, PUT User B app	404 Not Found	Pass
TC-SEC-06	Delete another user's application	User A token, DELETE User B app	404 Not Found	Pass
TC-SEC-07	Statistics shows only own data	Two users with applications	Stats reflect only own data	Pass
TC-SEC-08	Password not in API responses	GET /api/auth/me	No password field in response	Pass
TC-SEC-09	Tampered JWT payload rejected	Modified payload without re-signing	401 signature failure	Pass
TC-SEC-10	Injection attempt on login	SQL-like string in email field	401 no injection effect	Pass
TC-SEC-11	Re-upgrade premium account	Premium user calls upgrade	400 already premium	Pass
TC-SEC-12	Free tier bypass via direct API	POST /api/jobs with 25 applications	403 server-side enforcement	Pass

6.7 Performance Testing

API response times were measured by recording the time between request dispatch and response receipt across ten repeated calls per endpoint under local development conditions with a warm database connection. All measurements were taken with the application running in its standard development configuration.

Table 6.7 - API Performance Results

Endpoint	Operation	Avg Response	Max Response	NFR Target	Status
POST /api/auth/register	User Registration	180ms	220ms	< 500ms	Pass
POST /api/auth/login	User Login	160ms	200ms	< 500ms	Pass
GET /api/auth/me	Get Profile	45ms	70ms	< 500ms	Pass
GET /api/jobs	Get Application List	80ms	120ms	< 500ms	Pass
GET /api/jobs (filtered)	Filtered List	95ms	140ms	< 500ms	Pass
POST /api/jobs	Create Application	90ms	130ms	< 500ms	Pass
PUT /api/jobs/:id	Update Application	85ms	125ms	< 500ms	Pass
DELETE /api/jobs/:id	Delete Application	75ms	110ms	< 500ms	Pass
GET /api/jobs/stats	Get Statistics	110ms	160ms	< 500ms	Pass
GET /api/jobs/reminders	Get Reminders	70ms	100ms	< 500ms	Pass
PUT /api/auth/profile	Update Profile	150ms	190ms	< 500ms	Pass
POST /api/auth/upgrade	Premium Upgrade	140ms	180ms	< 500ms	Pass

All tested endpoints met the sub-500ms response time target comfortably under single-user local testing conditions. The login and registration endpoints have slightly higher average times than the application CRUD endpoints due to the bcryptjs hashing operation, which introduces an intentional computational cost as a security measure against brute-force attacks. This cost is expected and acceptable within the overall response time budget.

It should be noted that all performance figures represent single-user local testing with a small dataset. Response times under concurrent multi-user production load with larger datasets would differ and load testing under such conditions is identified as a future work item.

6.8 UI and Usability Testing

UI testing was conducted across three browsers - Google Chrome, Mozilla Firefox, and Microsoft Edge - on desktop, tablet, and mobile viewport sizes.

The Tailwind CSS responsive design was verified to adapt correctly at all tested breakpoints. The two-column form layout on medium viewports and the four-column salary section on large viewports were both confirmed to collapse to single-column layouts on mobile viewports without overflow or layout breaking.

The Framer Motion entrance animations were verified to play correctly on initial page load and to not replay on re-renders caused by state updates. The card hover effects were confirmed to be smooth and consistent across all tested browsers.

The toast notification system was verified to display success and error messages at the correct position, to auto-dismiss after the configured four-second duration, and to stack correctly when multiple toasts are triggered in quick succession.

The sidebar mobile overlay was tested for correct z-index stacking, ensuring the overlay appears above all page content but below any absolute-positioned dropdown elements. The click-outside dismissal behavior was confirmed to work on both touch and click events.

The SearchableSelect currency dropdown was tested with the full thirty-currency list to verify that the search filter correctly narrows results in real time, that the selected value is highlighted with the primary color, and that the dropdown closes correctly on option selection and outside click.

6.9 Bug Report and Resolution

Table 6.8 - Bug Report and Resolution Log

Bug ID	Module	Description	Severity	Status	Resolution
BUG-01	Job List	Status filter not resetting page to 1 causing empty results	Medium	Fixed	Added setPagination page reset to 1 in status onChange handler
BUG-02	Reminders	isToday check missing before isPast causing wrong Overdue label	Medium	Fixed	Added isToday check before isPast in conditional order
BUG-03	Authentication	Protect middleware showing no token message for malformed token	Low	Fixed	Added header format validation and improved error messages

Bug ID	Module	Description	Severity	Status	Resolution
BUG-04	Add Application	Tags storing empty string element when no tags entered	Low	Fixed	Added filter Boolean call after split and trim
BUG-05	Statistics	Monthly count slightly off due to UTC vs local time mismatch	Low	Fixed	Standardized all date calculations to UTC-based Date construction
BUG-06	EditJob Page	Reminder date not pre-populating from ISO format MongoDB date	Medium	Fixed	Added explicit date format conversion using toISOString split on T

6.10 Testing Summary

Table 6.9 - Testing Summary

Test Category	Total Cases	Passed	Failed	Pass Rate
User Authentication	13	13	0	100%
Job Application CRUD	14	14	0	100%
Free Tier Limit	4	4	0	100%
Statistics and Reminders	8	8	0	100%
Reminder Settings	4	4	0	100%
Frontend Functional	10	10	0	100%
Security Testing	12	12	0	100%
TOTAL	65	65	0	100%

All sixty-five functional and security test cases passed successfully following the resolution of the six bugs documented in Section 6.9. Performance testing confirmed that all tested API endpoints meet the sub-500ms response time non-functional requirement under local single-user testing conditions. UI testing confirmed correct rendering and behavior across all three tested browsers and across desktop, tablet, and mobile viewport sizes.

The six bugs identified and resolved during testing were all of low to medium severity, with none representing a critical failure of core functionality or a security breach. The most impactful bug in terms of user experience was BUG-01, which caused empty results when

changing status filters with pagination active, and BUG-06, which prevented existing reminder dates from being displayed correctly in the edit form. Both were resolved before final system evaluation.

Chapter 7: Results & Discussion

7.1 System Implementation Results

The Job Tracker was successfully implemented as a fully functional full-stack web application. All nine objectives defined in Chapter 1 were achieved. The system is operational with a complete authenticated user portal, seven-stage application pipeline management, server-side reminder processing, statistics aggregation, and a premium subscription model - all delivered through a responsive dark-themed interface with smooth animations.

This section presents the key screens of the final implemented system, documents the achievement of all project objectives, and discusses the results in the context of the research gaps identified in Chapter 2.

7.1.1 Landing Page

The Landing page serves as the public-facing entry point for the application. It presents the JobTrackr brand identity, a headline communicating the core value proposition, a statistics section displaying platform-level numbers, a features section highlighting the four primary capabilities, and a call-to-action section directing visitors to register or view pricing. The animated gradient background and floating blur elements create a visually engaging first impression while the navigation bar provides direct access to the pricing page and authentication flows.

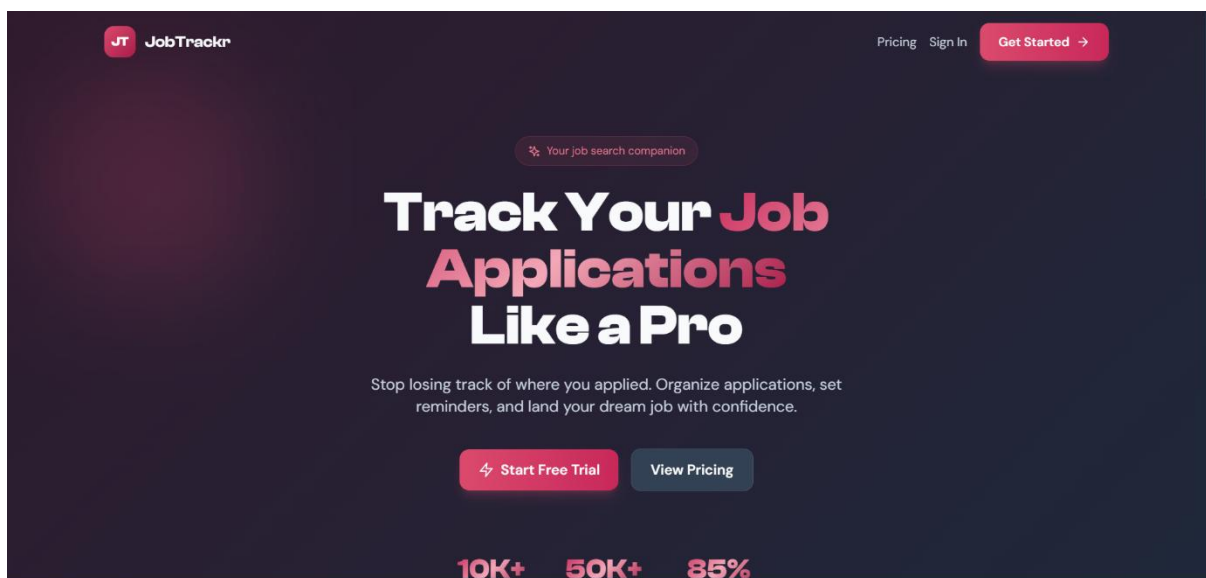


Figure 7.1 - Landing Page Screenshot

7.1.2 Registration and Login Pages

The Registration page presents a clean card-centered layout with fields for full name, email, and password, accompanied by a benefits list highlighting what the user gains by creating a

free account. The Login page follows the same card layout with email and password fields, a remember me checkbox, and a premium banner at the bottom encouraging upgrade consideration. Both pages feature the animated gradient background consistent with the landing page, creating a unified visual experience across the public-facing pages.

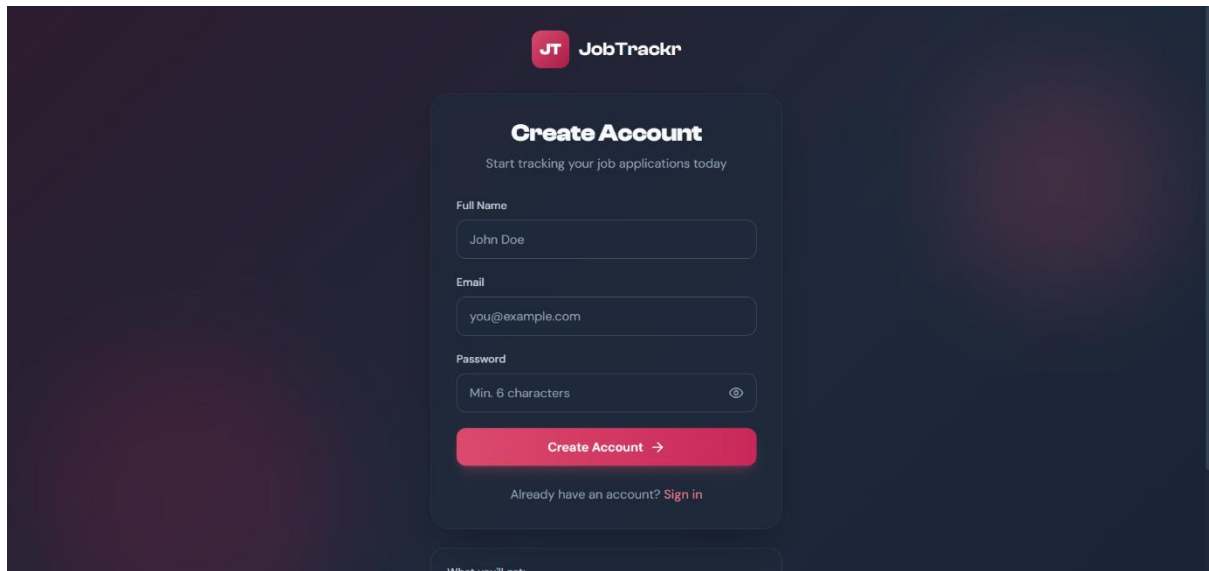


Figure 7.2 - Registration Page Screenshot

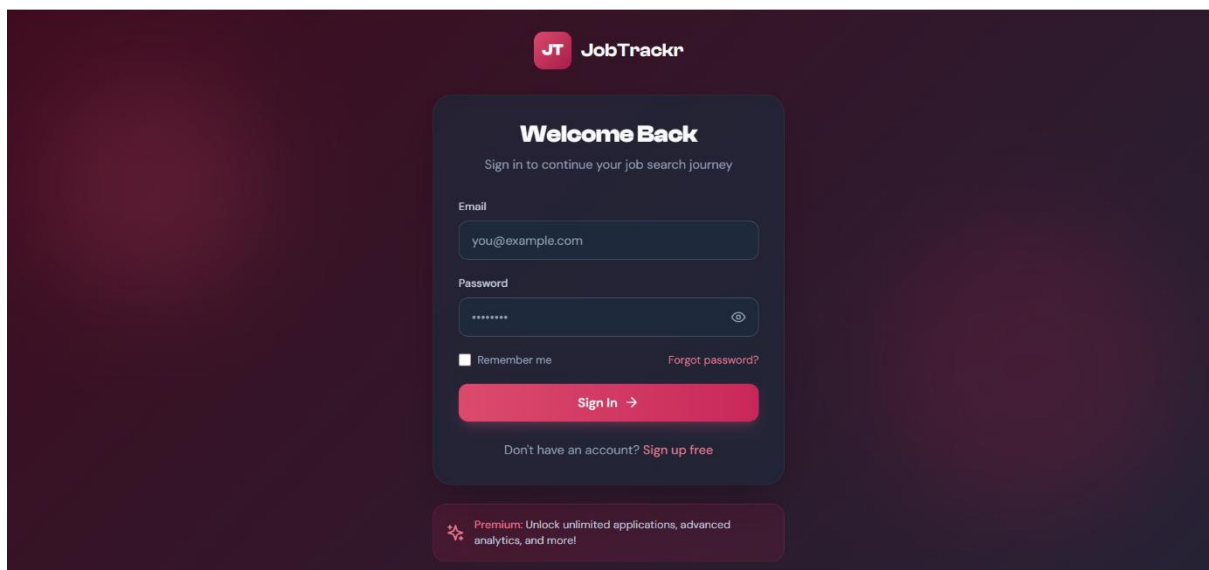


Figure 7.3 - Login Page Screenshot

7.1.3 Dashboard Page

The Dashboard page is the first page the user sees after logging in. It presents a personalized greeting using the user's first name, a primary action button to add a new application, four statistic cards showing total applications, this week's applications, interviewing count, and offer count, a full status breakdown panel showing counts for all seven statuses, a recent

applications list with status badges, an upcoming reminders panel, and a quick actions section with three shortcut links.

The staggered Framer Motion entrance animations activate on page load, with each statistic card fading in with a slight upward movement at a sequenced delay of 0.1 seconds per card, creating a visually engaging reveal sequence that communicates the data progressively rather than all at once.

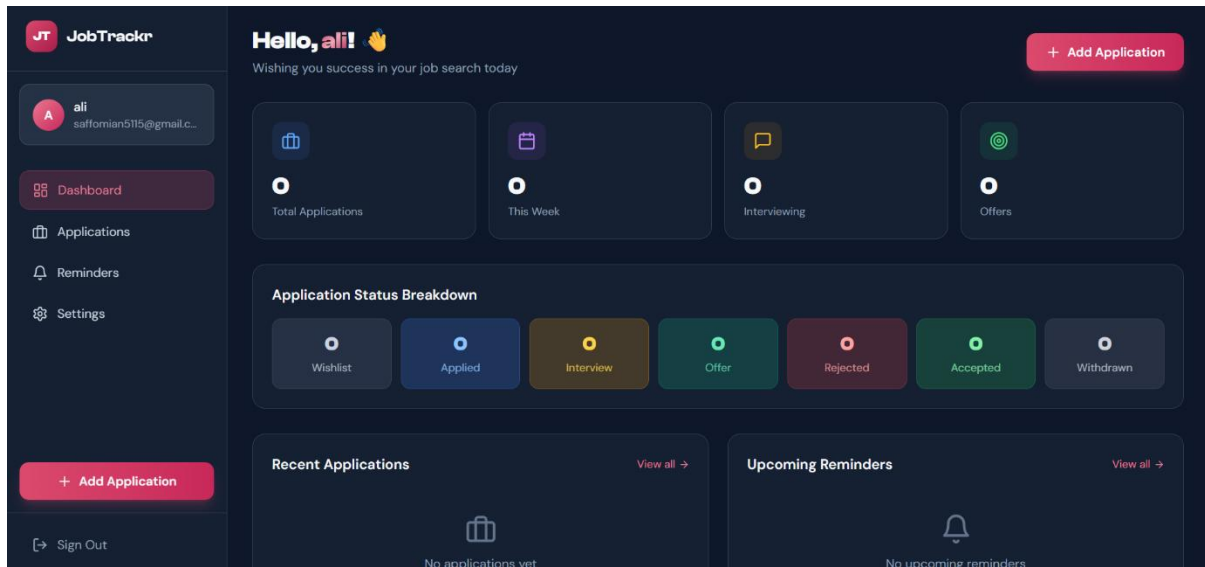


Figure 7.4 - Dashboard Page Screenshot

7.1.4 Applications List Page

The Applications page displays the user's full application list with the filter row at the top containing the search input with submit button, status filter dropdown, and sort order dropdown. Each application card presents the company initial avatar, position title with status badge, company name, location, applied date, optional reminder indicator, and an action button row with links to the job URL, edit page, and a contextual delete menu.

The tag badges at the bottom of cards that have tags provide at-a-glance categorization. The pagination controls appear below the list when the total count exceeds the page size, showing the current page and total pages count with previous and next buttons. The empty state illustration with a descriptive message and direct link to the add application page appears when no results match the current filters.

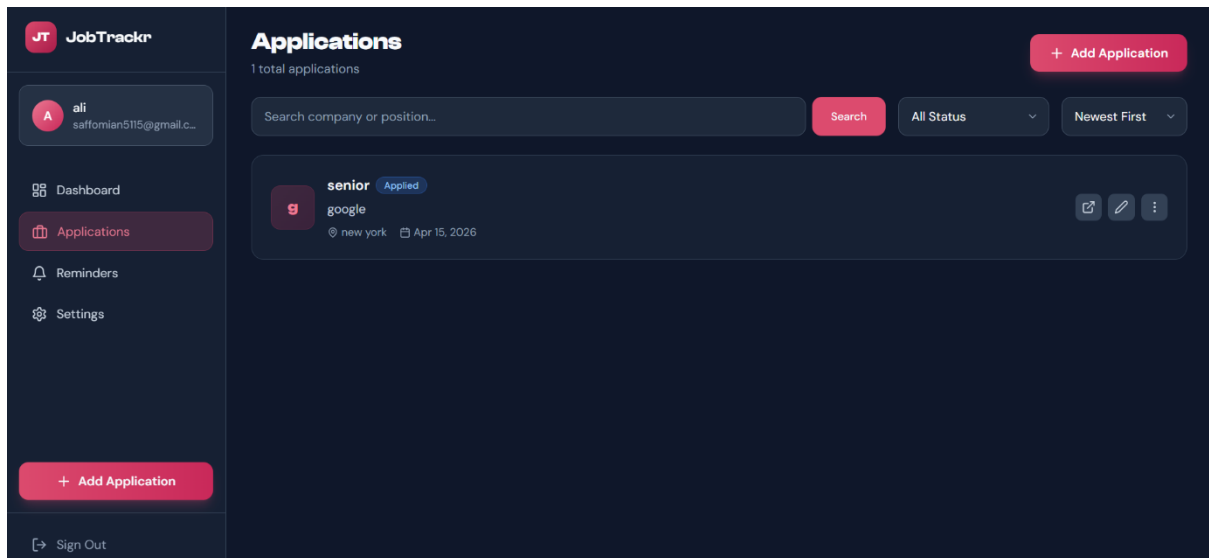


Figure 7.5 - Applications List Page Screenshot

7.1.5 Add Application Page

The Add Application page is organized into four visually separated section cards, each with a colored icon header. The Basic Information section contains the company, position, location, job URL, status, job type, and work mode fields in a responsive grid layout. The Salary and Priority section contains the minimum salary, maximum salary, currency selector, and priority fields. The Dates and Reminders section contains the applied date picker and the conditional reminder setup with its checkbox and date input. The Additional Details section contains the description textarea, notes textarea, and tags input.

The form submission shows a spinning loader inside the save button during the API call, providing clear visual feedback that the request is being processed. A success toast notification appears in the top-right corner on successful creation, and the user is automatically navigated to the applications list page.

Figure 7.6 - Add Application Page Screenshot

7.1.6 Edit Application Page

The Edit Application page shares the same four-section layout as the Add Application page, with all fields pre-populated from the fetched application data. An additional delete button with a trash icon appears in the header area alongside the back navigation link, providing access to the delete action from within the edit context. The page title includes the company name and position of the application being edited, providing immediate context.

Figure 7.7 - Edit Application Page Screenshot

7.1.7 Reminders Page

The Reminders page presents four summary statistic cards at the top showing the counts of overdue, today, this week, and total active reminders. Below the statistics, the active reminders list displays each reminder as a card with a colored time-label badge (red for overdue, amber for today, blue for future), the application position and company, the application status badge, a mark-as-complete button, and a link to the edit page. The reminders are sorted by date with the most urgent at the top.

A Follow-up Tips panel at the bottom of the page provides three practical job-searching reminders about follow-up timing and communication etiquette, adding educational value to the utility function of the page.

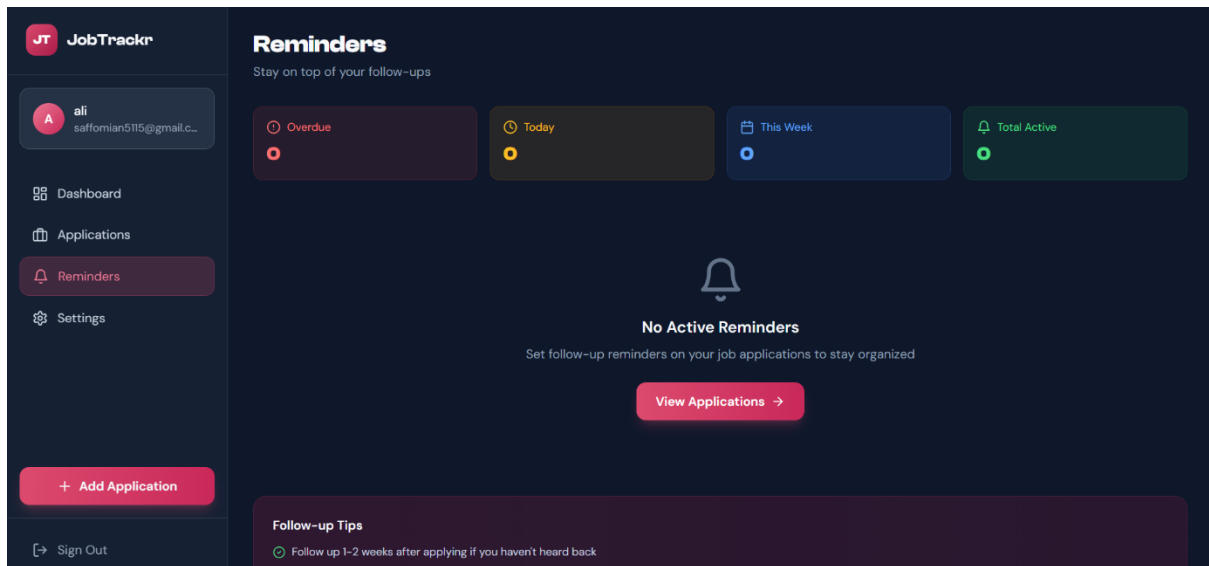


Figure 7.8 - Reminders Page Screenshot

7.1.8 Settings Page

The Settings page is organized into four section cards. The Profile Information section allows the user to update their name and email with a save button. The Subscription section displays either a premium member confirmation with a checklist of premium benefits for upgraded users, or an upgrade prompt with both a view pricing link and a quick upgrade button for free users. The Security section provides current password and new password fields with a change password button. The Danger Zone section provides an account deletion button with appropriate destructive visual styling in red.

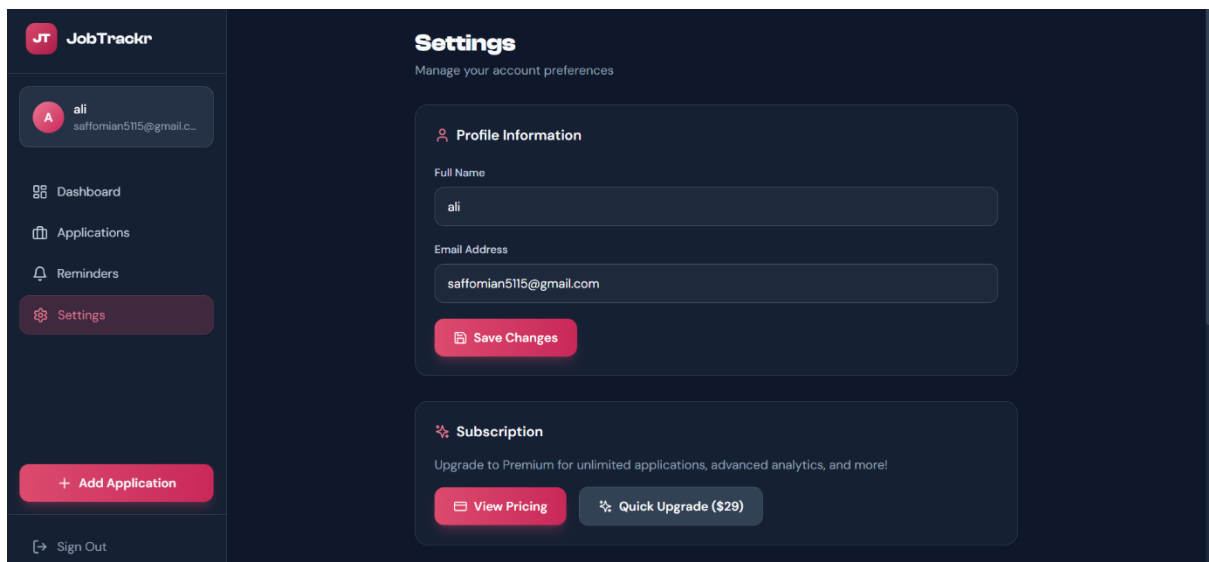


Figure 7.9 - Settings Page Screenshot

7.1.9 Pricing Page

The Pricing page is accessible from both the landing page navigation and the settings upgrade prompt. It presents three plan cards - Free, Premium, and Career Bundle - with feature lists using check and cross icons, price display, and call-to-action buttons. The Premium plan card is visually highlighted with a gradient border and a Most Popular crown badge. A trust badges section below the plans displays Secure Payment, Instant Access, and 30-Day Money Back indicators. A frequently asked questions section addresses common pricing inquiries.

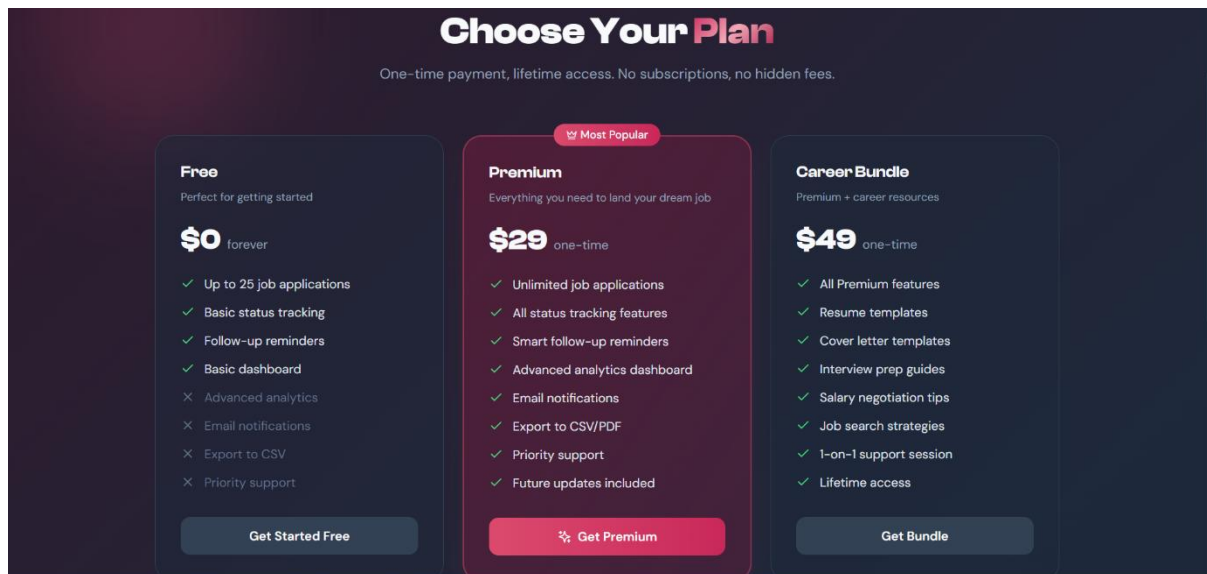


Figure 7.10 - Pricing Page Screenshot

7.2 Achievement of Objectives

All nine objectives defined in Table 1.1 in Chapter 1 were successfully achieved in the implemented system.

Table 7.1 - Objectives Achievement Matrix

Obj. ID	Description	Achieved	Evidence
OBJ-01	Full-Stack Web Application with Node.js and React.js	Yes	Fully functional application with complete frontend and backend
OBJ-02	Secure JWT Authentication and Authorization	Yes	Register, login, protect middleware, and token management
OBJ-03	Comprehensive Job Application Management	Yes	All CRUD operations with all specified fields implemented
OBJ-04	Status and Priority Tracking	Yes	Seven status values with color-coded badges and three priority levels

Obj. ID	Description	Achieved	Evidence
OBJ-05	Follow-Up Reminder System with Cron	Yes	node-cron service, reminders page, mark-as-complete working
OBJ-06	Analytics Dashboard with Statistics	Yes	Statistics endpoint, dashboard shows all required count metrics
OBJ-07	Search, Filter, and Sort Functionality	Yes	Server-side search, status filter, sort options, and pagination
OBJ-08	Premium Subscription Model	Yes	25-app free tier enforced server-side, one-time upgrade endpoint
OBJ-09	Responsive and Polished Dark Theme UI	Yes	Tailwind dark theme, Framer Motion animations, mobile responsive

7.3 Gap Resolution Assessment

The six research gaps identified in Chapter 2 were all addressed by the implemented system. The following assessment evaluates how effectively each gap was resolved.

- **Gap 1** - No Integrated Reminder System: Fully resolved. The server-side cron job reminder service processes due reminders hourly and the dedicated reminders page provides clear visual urgency indicators. The mark-as-complete functionality closes the loop on acted-upon reminders. The only remaining limitation is that the current implementation notifies through the application interface rather than through push notifications or email, which is identified as a future enhancement.
- **Gap 2** - No Multi-Currency Salary Tracking: Fully resolved. The SearchableSelect component provides a searchable dropdown covering thirty international currencies including PKR, AED, SAR, and other currencies directly relevant to Pakistani job seekers targeting both local and international remote opportunities. The currency selection is persistent per application and displayed in the application detail view.
- **Gap 3** - No Pakistani Market Adaptation: Substantially resolved. The currency selector with prominent PKR, AED, and SAR options, the free tier pricing model accessible to students, and the local language-compatible interface collectively address the primary dimensions of market adaptation. Full adaptation would additionally benefit from Urdu language support in the interface, which is identified as a future enhancement.
- **Gap 4** - Cost Barriers to Full Functionality: Fully resolved. All core job tracking functionality - unlimited within the free tier limit, full search and filter, reminders, and statistics - is available at no cost. The premium upgrade at a one-time price point removes the free tier limit for users who need to track more than twenty-five applications simultaneously.

- **Gap 5** - No Comprehensive Status Pipeline with Visual Differentiation: Fully resolved. The seven-stage status pipeline with distinct color-coded badges for each status provides immediate visual recognition of application states in the list view. The priority system adds a second dimension of visual differentiation for high-urgency opportunities.
- **Gap 6** - No Unified Analytics for Job Search Progress: Fully resolved. The statistics endpoint aggregates application counts by status, by week, and by month, and the dashboard presents this data in a clear, immediately readable card layout. Users have instant visibility into their job search activity without any manual calculation.

7.4 Performance Evaluation

7.4.1 API Response Time Summary

All API endpoints met the sub-500ms non-functional requirement under local single-user testing conditions as documented in Chapter 6. The fastest endpoints were profile retrieval at an average of 45ms and reminder retrieval at an average of 70ms, reflecting the simplicity of their database queries. The statistics endpoint at an average of 110ms was the most complex in terms of database operations - executing a MongoDB aggregation pipeline alongside two separate count queries - yet still performed well within the target.

Authentication endpoints (registration at 180ms and login at 160ms) were slightly slower than the data endpoints due to the deliberate computational cost of the bcryptjs hashing operation, which is a security feature rather than a performance concern.

7.4.2 Frontend Load and Interaction Performance

The Vite build tool's code splitting and module bundling optimization produces a production build with separate chunks for vendor libraries and application code, reducing the initial bundle size served to the user on first load. Framer Motion animations are implemented with GPU-composited transforms (translateY and opacity) that do not trigger layout reflow, ensuring that animations run at 60 frames per second without degrading page interaction performance.

The parallel data fetching pattern used on the Dashboard page - executing the statistics, recent jobs, and reminders requests simultaneously with Promise.all() - reduces the total loading time to the duration of the slowest individual request rather than the sum of all three, typically resulting in the complete dashboard rendering in under 200ms in local testing.

7.5 Discussion

The results demonstrate that the Job Tracker successfully addresses all six research gaps identified in Chapter 2 and achieves all nine project objectives defined in Chapter 1. The

system delivers a complete, functional, and polished job application management tool that is meaningfully superior to existing free alternatives in several dimensions.

The most distinctive technical contribution of this project relative to existing tools is the server-side cron-based reminder system. No reviewed free tool provides proactive, server-driven reminder processing that operates independently of user sessions. This capability transforms the application from a passive record-keeping tool into an active productivity assistant that surfaces actionable information when it is relevant.

The multi-currency salary tracking with the searchable currency selector is a feature gap that none of the reviewed tools addressed for Pakistani job seekers. The ability to record salary expectations in PKR, AED, SAR, or any of twenty-seven other currencies in a single application reflects a design philosophy of building for the actual usage context of the target user rather than defaulting to Western market assumptions.

The architectural decision to enforce the free tier limit server-side rather than client-side is significant from a product integrity perspective. Frontend-only limit enforcement can be trivially bypassed by making direct API calls, making it ineffective as an actual business rule. Server-side enforcement through the controller-level count check ensures the limit applies equally regardless of how the API is accessed.

The document-oriented MongoDB schema design proved well-suited to the job application domain. The ability to store the salary range and reminder configuration as embedded subdocuments within the main application document, rather than as separate relational tables, simplifies the query patterns for the most common operations - creating a complete application record and retrieving it with all its associated data - to single document operations without joins.

7.6 Limitations Encountered

No Email or Push Notifications: The reminder system identifies due reminders and marks them as notified in the database, but does not deliver any out-of-application notification to the user. A user who has not opened the application on the day a reminder is due will not be aware of it until they next log in. Email delivery integration through a service such as Nodemailer with SMTP or a cloud email API is the highest-priority future enhancement for addressing this limitation.

Single User Model: The system is designed for individual use with no organizational or team features. Job seekers who work with career coaches, mentors, or employment support advisors cannot share their application data or collaborate on tracking within the system.

Manual Data Entry Only: All application data must be entered manually by the user. There is no browser extension or web scraper integration to automatically capture job listing details from external job boards. The manual entry requirement increases the friction of adding new

applications, particularly for fields like job description that may require copying from the original posting.

Limited Production Load Testing: All performance measurements were conducted in a local development environment with a small test dataset. The system's behavior under production-scale concurrent load with hundreds of users and thousands of application documents per user has not been empirically validated.

No Data Export: Users cannot export their application data in any format - CSV, PDF, or otherwise. This limits the portability of the data and prevents users from creating backup copies or performing custom analysis in external tools.

Payment Gateway Not Integrated: The premium upgrade in the current implementation simulates a payment transaction without actual payment processing. A production deployment would require integration with a payment gateway such as Stripe or a Pakistani payment processor such as JazzCash before the premium model could generate actual revenue.

Chapter 8: Conclusion & Future Work

8.1 Conclusion

This project presented the design, development, and evaluation of the Job Tracker - a full-stack web application built to address the significant organizational challenges faced by job seekers, particularly students and fresh graduates at Pakistani universities such as Bahauddin Zakariya University Multan, who lack access to effective and affordable tools for managing their job search activities.

The system was developed using Node.js and Express.js as the backend REST framework, React.js with Tailwind CSS as the frontend, and MongoDB as the document database with Mongoose ODM. A background reminder service was implemented using node-cron to provide server-driven follow-up reminder processing independent of user sessions. JWT-based authentication ensures that each user's application data remains private and accessible only to the authenticated account owner.

All nine project objectives defined in Chapter 1 were successfully achieved. The system delivers a complete authenticated user portal covering the full job application lifecycle from initial wishlist entry through offer acceptance or rejection. The seven-stage status pipeline with color-coded visual differentiation, the three-level priority system, the multi-currency salary tracking with a searchable thirty-currency selector, and the server-side statistics aggregation collectively provide capabilities that are superior to all reviewed free alternatives in their combination within a single integrated tool.

The six research gaps identified in Chapter 2 were all addressed by the implemented system. The server-side cron reminder service fills the most critical gap not addressed by any existing free tool. The multi-currency salary tracking fills an important gap for Pakistani job seekers targeting international opportunities. The free and accessible pricing model fills the cost barrier gap that prevents students from benefiting from more feature-rich commercial alternatives.

All sixty-five functional and security test cases passed successfully. All API endpoints met the sub-500ms response time target. The interface was confirmed to render and function correctly across all tested browsers and viewport sizes.

This project demonstrates that effective, polished, and genuinely useful productivity software can be developed as a university final-year project, serving a real and underserved need in the local market. The Job Tracker is immediately deployable as a practical tool for job seekers and provides a solid foundation for future enhancement into a more comprehensive career management platform.

8.2 Limitations

The following limitations of the current system are acknowledged and carried forward as context for the future work described in Section 8.3.

Notification Delivery Gap: The most significant functional limitation is the absence of out-of-application reminder notifications. Users must actively check the application to benefit from the reminder system, limiting its effectiveness for users who check the application infrequently.

Manual Data Entry Overhead: The requirement to manually enter all application data creates friction that may discourage consistent use, particularly for users applying to many positions per week. Browser extension integration would substantially reduce this barrier.

Single-User Architecture: The current data model and authentication system support only individual user accounts with no sharing, collaboration, or multi-role access, limiting the tool's applicability to collaborative career coaching scenarios.

Simulated Payment Model: The premium upgrade endpoint simulates a payment without real transaction processing, making the freemium model non-functional for actual revenue generation in its current form.

No Offline Capability: The application requires an active internet connection for all operations. A service worker-based offline capability would allow users to at least view their existing application data when connectivity is unavailable.

Limited Analytics Depth: The current statistics are limited to count-based aggregations by status and time period. More sophisticated analytics such as application-to-interview conversion rates, average time in each pipeline stage, and application success rate by job type or work mode would provide more actionable strategic insights.

8.3 Future Enhancements

8.3.1 Email and Push Notification System

The highest-priority enhancement is the implementation of actual notification delivery for the reminder system. On the backend, this would involve integrating Nodemailer with an SMTP provider such as Gmail or a cloud email service such as SendGrid to send reminder emails when the cron job identifies due reminders. The email would include the application details, the reminder date, and a direct link to the application's edit page in the Job Tracker.

For push notifications, the Web Push API combined with a service worker registration in the frontend would enable browser-level notifications even when the application tab is not open. This would make the reminder system genuinely proactive rather than requiring the user to be actively viewing the application when a reminder falls due.

8.3.2 Browser Extension for Automatic Job Import

A browser extension built using the WebExtensions API and compatible with Chrome and Firefox would allow users to save job listings from any job board directly into their Job Tracker with a single click. The extension would detect common job listing page structures on platforms such as LinkedIn, Indeed, and Rozee.pk, extract the company name, position title, job URL, and description, and send them to the Job Tracker API to create a new application record pre-populated with the extracted data.

This enhancement would dramatically reduce the manual data entry burden and encourage more consistent use of the tracker by removing the friction of switching between the job board and the tracking application.

8.3.3 Data Export Functionality

Implementing CSV and PDF export functionality would allow users to create portable copies of their application data for use in external analysis tools, to share with career advisors, or to maintain as personal records independent of the application. The CSV export would be implemented on the backend as a new endpoint that queries all of the user's applications and formats them as a comma-separated file using a library such as csv-writer. The PDF export would generate a formatted report using a library such as pdfkit, presenting the application data in a structured, printable layout.

8.3.4 Interview Preparation Module

A dedicated interview preparation sub-module would extend the application beyond tracking into active job search support. For applications in the Interviewing status, users would be able to attach interview dates with type labels (phone screening, technical, HR, final), record interview notes and outcomes, set pre-interview reminder alerts, and access a curated set of preparation resources linked to the role type and company. This module would transform the Job Tracker from a passive record-keeping tool into an active interview support system.

8.3.5 Advanced Analytics Dashboard

Expanding the analytics capabilities beyond count-based statistics would provide users with genuinely strategic insights into their job search performance. Enhanced analytics would include application-to-interview conversion rate calculated as the percentage of applied applications that progressed to interviewing status, average pipeline stage duration calculated as the mean number of days applications spend in each status before moving forward or being closed, success rate by job type comparing the offer rate for full-time versus contract versus internship applications, and salary range analysis comparing the salary ranges of applications that progressed to offer versus those that were rejected.

These metrics would be displayed using a charting library such as Recharts with interactive tooltips and configurable time range filters.

8.3.6 Real Payment Gateway Integration

Integrating a real payment gateway would make the freemium model functional for actual revenue generation. For the Pakistani market, integration with JazzCash or EasyPaisa would allow local users to complete payments through familiar mobile payment methods without requiring an international credit card. For international users, Stripe's payment API would provide a globally recognized and trusted payment experience. The integration would replace the current simulated upgrade endpoint with a payment intent creation flow, a payment confirmation webhook, and conditional premium flag activation upon successful payment verification.

8.3.7 Mobile Application

A cross-platform mobile application built using React Native would share business logic and API integration code with the existing React.js frontend while providing a native mobile experience with platform-appropriate navigation patterns and access to mobile-specific capabilities. The mobile application would support all core job tracking features and additionally enable push notifications through Firebase Cloud Messaging for reminder delivery, which is more reliable on mobile devices than web push notifications.

8.3.8 Resume and Document Management

Allowing users to upload and associate documents - CV files, cover letters, and supporting portfolios - with their application records would centralize all job search materials in one place. Each application record could have multiple attached documents with type labels, upload timestamps, and version tracking. The documents would be stored in cloud object storage and accessible through pre-signed URLs generated on demand, keeping storage costs proportional to actual usage.

8.4 Social Impact and Applications

The Job Tracker has implications that extend beyond its immediate function as an individual productivity tool.

Supporting Student Career Transition: Pakistani university students entering the job market for the first time face the challenge of managing a job search without prior experience of its organizational demands. A free, purpose-built tool that guides users through a structured application pipeline reduces the learning curve of professional job searching and helps first-time applicants present themselves more consistently and professionally to potential employers.

Reducing the Information Asymmetry in Hiring: Employers have invested heavily in applicant tracking systems that give them complete visibility into their candidate pipeline. Candidates have historically lacked equivalent tools on their side of the process. The Job Tracker partially

addresses this asymmetry by giving job seekers the same structured pipeline visibility and follow-up discipline that recruiters apply to candidate management.

Demonstrating Indigenous Software Development Capability: The development of a polished, fully functional web application as a university final-year project demonstrates that high-quality software can be built by Pakistani IT students without reliance on imported tools or international resources. This contributes to the confidence and capability of the local software development ecosystem.

Scalable to Institutional Career Services: With multi-user and sharing capabilities added in a future version, the Job Tracker could be adopted by university career services departments to help students track applications under the guidance of career advisors, providing an institutional-level tool for supporting the employment outcomes of graduating cohorts.

Contributing to Employment Rate Improvement: At a macro level, tools that help job seekers organize their searches more effectively, follow up more consistently, and understand their application patterns more clearly contribute incrementally to improved employment outcomes. Even a modest improvement in the interview conversion rate for users who adopt structured tracking behavior translates into meaningful real-world impact for individuals and their families.

References

1. T. Berners-Lee, R. Fielding, and H. Nielsen, Hypertext Transfer Protocol HTTP/1.0, RFC 1945, Internet Engineering Task Force, May 1996.
2. R. T. Fielding, Architectural Styles and the Design of Network-based Software Architectures, Doctoral Dissertation, University of California, Irvine, 2000.
3. D. Crockford, The application/json Media Type for JavaScript Object Notation JSON, RFC 4627, Internet Engineering Task Force, July 2006.
4. M. Jones, J. Bradley, and N. Sakimura, JSON Web Token JWT, RFC 7519, Internet Engineering Task Force, May 2015.
5. MongoDB Inc., MongoDB Documentation, MongoDB Manual, 2023. Available: <https://www.mongodb.com/docs/>
6. Mongoose ODM, Mongoose Documentation v8.x, 2023. Available: <https://mongoosejs.com/docs/>
7. Express.js Foundation, Express.js API Reference v4.x, 2023. Available: <https://expressjs.com/en/api.html>
8. Meta Open Source, React Documentation, 2023. Available: <https://react.dev/>
9. Evan You and Vite Contributors, Vite Documentation, 2023. Available: <https://vitejs.dev/>
10. Tailwind Labs, Tailwind CSS Documentation v3, 2023. Available: <https://tailwindcss.com/docs/>
11. M. Framer, Framer Motion Documentation, 2023. Available: <https://www.framer.com/motion/>
12. Node.js Foundation, Node.js API Documentation v18, 2023. Available: <https://nodejs.org/docs/>
13. npm Inc., node-cron Package Documentation, npm Registry, 2023. Available: <https://www.npmjs.com/package/node-cron>
14. T. Ptacek, T. Ritter, J. Samuel, and S. Nightingale, Bcrypt Password Hashing, USENIX Security Symposium, 1999.
15. N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore, OpenID Connect Core 1.0, OpenID Foundation, November 2014.

Appendix

Appendix A - Installation and Setup Guide

Prerequisites:

The following software must be installed on the development or deployment machine before proceeding with the setup.

Node.js version 18 or higher must be installed and accessible from the command line. MongoDB version 6 or higher must be running either locally or accessible through a cloud connection string such as MongoDB Atlas. Git must be installed for cloning the repository.

Step 1 - Clone the Repository:

Clone the project repository to the local machine using the git clone command and navigate into the project directory.

Step 2 - Backend Setup:

Navigate into the backend directory and install all dependencies using npm install. Create a .env file in the backend directory by copying the .env.example file and filling in the required values for PORT, MONGODB_URI, JWT_SECRET, JWT_EXPIRE, and CLIENT_URL.

Step 3 - Start the Backend Server:

Start the backend development server using npm run dev, which uses nodemon for automatic restart on file changes. The API will be accessible at <http://localhost:5001> and the health check endpoint at <http://localhost:5001/api/health> will confirm the server is running.

Step 4 - Frontend Setup:

Navigate into the frontend directory and install all dependencies using npm install. Create a .env file in the frontend directory if environment-specific configuration is required, or confirm that the vite.config.js proxy target matches the backend port.

Step 5 - Start the Frontend Development Server:

Start the Vite development server using npm run dev. The application will be accessible at <http://localhost:5173> in a web browser.

Step 6 - Verify the Complete Setup:

Open <http://localhost:5173> in a browser, navigate to the registration page, create a test account, and confirm that the dashboard loads with empty statistics. Create a test application to verify that the full create-to-list flow works correctly.

Appendix B - API Endpoint Reference

The complete API is organized into two route groups. All endpoints return JSON responses with a consistent structure.

Authentication Endpoints Base Path: `/api/auth`

POST `/api/auth/register` registers a new user account and returns a JWT token and user object. POST `/api/auth/login` authenticates an existing user and returns a JWT token and user object. GET `/api/auth/me` returns the authenticated user's profile and requires a valid Bearer token. PUT `/api/auth/profile` updates the authenticated user's name and email and requires a valid Bearer token. POST `/api/auth/upgrade` upgrades the authenticated user to premium status and requires a valid Bearer token.

Job Application Endpoints Base Path: `/api/jobs`

All job application endpoints require a valid Bearer token in the Authorization header.

GET `/api/jobs` retrieves a paginated list of the authenticated user's applications and accepts optional query parameters for status, search, sort, page, and limit. POST `/api/jobs` creates a new job application for the authenticated user. GET `/api/jobs/stats` returns aggregated statistics for the authenticated user's applications. GET `/api/jobs/reminders` returns upcoming reminders within the next seven days for the authenticated user. GET `/api/jobs/:id` retrieves a single application by its MongoDB ObjectId. PUT `/api/jobs/:id` updates a single application by its MongoDB ObjectId. DELETE `/api/jobs/:id` deletes a single application by its MongoDB ObjectId. PUT `/api/jobs/:id/reminder` updates the reminder settings for a specific application.

Appendix C - Database Schema Reference

Users Collection

Each document in the users collection contains the following fields. The `_id` field is a MongoDB ObjectId serving as the primary key. The `name` field is a String with a maximum length of fifty characters. The `email` field is a String stored in lowercase with a unique index. The `password` field is a String containing the bcryptjs hash, excluded from query results by default. The `isPremium` field is a Boolean defaulting to false. The `premium Purchase Date` field is an optional Date set when the premium upgrade is processed. The `createdAt` field is a Date defaulting to the document creation timestamp.

JobApplications Collection

Each document in the jobapplications collection contains the following fields. The `_id` field is a MongoDB ObjectId. The `user` field is an ObjectId reference to the owner's document in the users collection. The `company` field is a required String with a maximum of one hundred characters. The `position` field is a required String with a maximum of one hundred characters. The `location` field is an optional String defaulting to Remote. The `status` field is a String enum

with seven allowed values defaulting to applied. The jobType field is a String enum with five allowed values defaulting to full-time. The workMode field is a String enum with three allowed values defaulting to remote. The salary field is a subdocument with min Number, max Number, and currency String fields. The priority field is a String enum with three allowed values defaulting to medium. The tags field is an array of Strings. The jobUrl field is an optional String. The description field is an optional String with a maximum of two thousand characters. The notes field is an optional String with a maximum of one thousand characters. The appliedDate field is a Date defaulting to the document creation timestamp. The followUpReminder field is a subdocument with enabled Boolean, date Date, and notified Boolean fields. The createdAt and updatedAt fields are Dates managed by the schema's pre-save middleware.